

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1

HỌC PHẦN PHÁT TRIỂN CÁC HỆ THỐNG THÔNG MINH



BÁO CÁO ASSIGNMENT 02

**ĐỀ TÀI: TỪ BIỂU DIỄN DỮ LIỆU ĐẾN
HỆ THỐNG THÔNG MINH CÓ THỂ TRIỂN KHAI**

Lớp:	D23CTPM01
Họ và tên:	Nguyễn Duy Nghĩa
Mã sinh viên:	B23DCCN600
Giảng viên hướng dẫn:	PGS.TS Trần Đình Quế
Học kỳ:	Học kỳ 1 năm học 2026 – 2027

Hà Nội, 2026

Mục lục

Mã nguồn dự án và nguồn dữ liệu	1
Chương I. Giới thiệu bài toán	2
1.1. Bối cảnh	2
1.2. Mục tiêu của bài tập	2
1.3. Quy trình phát triển hệ thống thông minh	3
1.4. Tổng quan về ba ứng dụng	5
1.4.1. Ứng dụng dự đoán bệnh tiểu đường	5
1.4.2. Ứng dụng dự đoán giá nhà	5
1.4.3. Ứng dụng phân tích hành vi và khám phá sở thích khách hàng thương mại điện tử	6
Chương II. Cơ sở lý thuyết về biểu diễn dữ liệu	7
2.1. Tổng quan về học máy và hệ thống thông minh	7
2.2. Bài toán phân loại và hồi quy	7
2.2.1. Bài toán phân loại	7
2.2.2. Bài toán hồi quy	7
2.3. Khái niệm biểu diễn dữ liệu	7
2.4. Vector đặc trưng	8
2.5. Ma trận đặc trưng	8
2.6. Tensor và biểu diễn dữ liệu nhiều chiều	9
2.7. Biểu diễn dữ liệu văn bản	9
2.8. Mã hoá dữ liệu phân loại	10
2.9. Kiểu dữ liệu và miền giá trị	11
2.10. Tính nhất quán của biểu diễn dữ liệu	11
2.11. Rò rỉ dữ liệu	12
Chương III. Phương pháp tiền xử lý dữ liệu	14
3.1. Tìm hiểu dữ liệu	14
3.2. Làm sạch dữ liệu	15
3.2.1. Xử lý giá trị thiếu	15
3.2.2. Xử lý dữ liệu trùng lặp	15
3.2.3. Xử lý giá trị không hợp lệ	16
3.2.4. Phân tích ngoại lệ	16
3.3. Mã hoá biến phân loại	16

3.4. Chuẩn hoá dữ liệu số	16
3.5. Chia tập dữ liệu	17
3.6. Ngăn ngừa rò rỉ dữ liệu	17
3.7. Pipeline tiền xử lý	17
3.8. Nguyên tắc tái lập quá trình tiền xử lý	18
Chương IV. Các độ đo đánh giá mô hình	20
4.1. Đối với bài toán Hồi quy	20
4.2. Đối với bài toán Phân lớp	21
Chương V. Phương pháp xây dựng mô hình	23
5.1. Tổng quan về xây dựng mô hình	23
5.2. Mô hình Baseline	23
5.3. Các mô hình cho bài toán hồi quy	24
5.4. Các mô hình cho bài toán phân loại	25
Chương VI. Phương pháp lưu trữ và triển khai mô hình	26
6.1. Tổng quan	26
6.2. Lưu trữ mô hình	26
6.3. Tính nhất quán giữa huấn luyện và dự đoán	27
6.4. Triển khai mô hình dưới dạng Web Service	28
6.5. Web Application	30
6.5.1. Sáu thành phần giao diện phục vụ khả năng diễn giải	30
6.6. Triển khai trên ứng dụng mobile	32
6.7. Kiến trúc hoàn chỉnh của hệ thống	33
6.8. Thông tin tái lập thực nghiệm	33
6.8.1. Các bước tái lập	34
Chương VII. Ứng dụng 1 — Dự đoán bệnh tiểu đường	36
7.1. Mô tả bài toán	36
7.2. Giới thiệu tập dữ liệu	36
7.3. Khảo sát và tìm hiểu dữ liệu	37
7.4. Làm sạch dữ liệu	38
7.4.1. Giá trị thiếu bị mã hoá thành số 0	38
7.4.2. Phân tích ngoại lệ	40
7.5. Biểu diễn dữ liệu	40
7.5.1. Vì sao chỉ dùng 5 trong 8 đặc trưng	43
7.5.2. Pipeline tiền xử lý	44

7.6. Phân tích khám phá dữ liệu (EDA)	45
7.7. Xây dựng mô hình	47
7.7.1. Chia tập Train, Validation và Test	47
7.7.2. Mô hình cơ sở	48
7.7.3. Năm mô hình phân loại	48
7.8. Đánh giá mô hình	49
7.8.1. Diễn giải ma trận nhầm lẫn	51
7.8.2. Phân tích sai số	52
7.9. Lựa chọn mô hình	53
7.10. Triển khai hệ thống	53
7.10.1. Giao diện Web	55
7.10.2. Giao diện Mobile	56
Chương VIII. Ứng dụng 2 — Dự đoán giá nhà	59
8.1. Mô tả bài toán	59
8.2. Giới thiệu tập dữ liệu	59
8.3. Khảo sát và tìm hiểu dữ liệu	60
8.4. Làm sạch dữ liệu	62
8.4.1. Vì sao xử trùng lặp là bước quan trọng nhất	62
8.4.2. Bốn cải tiến so với Assignment 01	63
8.5. Biểu diễn dữ liệu	63
8.5.1. Rút biến vị trí từ cột địa chỉ	63
8.5.2. Năm đặc trưng phái sinh	65
8.5.3. Ma trận đặc trưng cuối cùng	66
8.6. Phân tích khám phá dữ liệu (EDA)	67
8.7. Xây dựng mô hình	69
8.7.1. Chia tập và mô hình cơ sở	69
8.7.2. Năm mô hình hồi quy	70
8.8. Đánh giá mô hình	70
8.8.1. Phân tích phần dư	71
8.9. Lựa chọn mô hình	72
8.10. Triển khai hệ thống	73
8.10.1. Giao diện Web	75
8.10.2. Giao diện Mobile	76
Chương IX. Ứng dụng 3 — Phân tích hành vi khách hàng và khám phá sở thích	79

9.1. Mô tả bài toán	79
9.2. Giới thiệu tập dữ liệu	79
9.3. Biểu diễn khách hàng	80
9.4. Làm sạch dữ liệu	80
9.4.1. Vấn đề nghiêm trọng nhất là rò rỉ nhãn	80
9.4.2. Trùng lặp và bình luận rỗng	81
9.5. Biểu diễn dữ liệu văn bản	81
9.5.1. Bốn bước trên một bình luận thật	82
9.5.2. Tensor embedding $E \in \mathbb{R}^{(B \times T \times d)}$	84
9.5.3. Vì sao triển khai bằng TF-IDF chứ không bằng embedding	85
9.5.4. Hai biểu diễn song song	87
9.6. Khám phá dữ liệu (EDA)	88
9.7. Xây dựng mô hình	90
9.8. Đánh giá mô hình	92
9.8.1. Câu hỏi trung tâm — văn bản có cải thiện dự đoán không?	93
9.9. Khám phá sở thích và hành vi khách hàng	95
9.9.1. Ứng dụng trong thương mại điện tử	95
9.9.2. Phân tích sai số	96
9.10. Lựa chọn mô hình	97
9.11. Triển khai hệ thống	97
9.11.1. Giao diện Web	99
9.11.2. Giao diện Mobile	100
Chương X. So sánh ba hệ thống thông minh	103
10.1. So sánh dữ liệu và bài toán	103
10.2. So sánh biểu diễn dữ liệu	103
10.3. So sánh chất lượng dữ liệu và tiền xử lý	104
10.3.1. Các bước tiền xử lý chung	104
10.3.2. Các bước tiền xử lý riêng	105
10.4. So sánh mô hình và đánh giá	105
10.5. So sánh triển khai	106
10.6. Tổng hợp và bài học rút ra	107
10.6.1. Sáu bài học chính	107
10.7. Thách thức kỹ thuật lớn nhất	108
10.8. Vấn đề biểu diễn dữ liệu quan trọng nhất	108
10.9. Bài học về triển khai	109

10.10. Hướng cải tiến	109
Chương XI. Mở rộng — Tích hợp đồ thị tri thức Neo4j	110
11.1. Luồng hoạt động	110
11.1.1. Lược đồ đồ thị	111
11.1.2. Truy vấn Cypher	113
11.2. Kết quả thực nghiệm	113
11.2.1. Kiểm chứng nguyên tắc “không ảnh hưởng tới dự đoán”	115
11.2.2. Giá trị mang lại và hạn chế	115

Mã nguồn dự án và nguồn dữ liệu

Toàn bộ mã nguồn, notebook, mô hình đã huấn luyện và bản báo cáo này được công bố công khai tại kho mã nguồn dưới đây.

Kho mã nguồn: <https://github.com/Nghaiz/Assignment-02-Intelligent-System>

Ba tập dữ liệu gốc được tải từ Kaggle và giữ nguyên trong thư mục `<ứng-dụng>/data/` của kho mã nguồn:

Ứng dụng 1 — Tiểu đường	Pima Indians Diabetes Database https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database
Ứng dụng 2 — Giá nhà	Vietnam Housing Dataset 2024 https://www.kaggle.com/datasets/nguyentiennhan/vietnam-housing-dataset-2024
Ứng dụng 3 — Hành vi khách hàng	Women's E-Commerce Clothing Reviews https://www.kaggle.com/datasets/nicapotato/womens-ecommerce-clothing-reviews

Chương I. Giới thiệu bài toán

1.1. Bối cảnh

Dữ liệu thực tế được sinh ra dưới nhiều hình thức rất khác nhau: bảng số liệu, giao dịch, văn bản tự do, ảnh, âm thanh. Nhưng hầu hết mô hình học máy chỉ nhận được một thứ duy nhất — một mảng số có kích thước xác định. Khoảng cách giữa hai điều đó là nơi phân lớn công sức của một hệ thống thông minh thực sự được tiêu tốn.

Assignment 02 với chủ đề *From Data Representation to a Deployable Intelligent System* yêu cầu đi trọn con đường ấy, chứ không dừng ở việc huấn luyện một mô hình:

Dữ liệu thô → Tìm hiểu → Làm sạch → Biểu diễn → Học → Đánh giá → Lưu trữ → Triển khai

Điểm nhấn nằm ở chỗ chất lượng của một hệ thống thông minh không do thuật toán quyết định một mình. Nó phụ thuộc vào chất lượng dữ liệu, vào cách dữ liệu được biểu diễn thành số, và vào việc phép đánh giá có trung thực hay không. Báo cáo này sẽ đưa ra bằng chứng định lượng cho cả ba mệnh đề đó, ở ba ứng dụng khác nhau.

Ngoài phần xây dựng mô hình, hệ thống sau khi huấn luyện phải dùng lại được mà không cần huấn luyện lại. Vì vậy mô hình và quy trình tiền xử lý đều được lưu trữ và nạp lại lúc dự đoán. Cả ba ứng dụng đều được triển khai thành một dịch vụ Web kèm giao diện Mobile.

1.2. Mục tiêu của bài tập

Assignment 02 hướng tới bảy mục tiêu, và báo cáo này bám theo đúng thứ tự ấy.

#	Mục tiêu	Được chứng minh ở đâu trong báo cáo
1	Hiểu và phân tích dữ liệu thực tế	Mục 7.3, 8.3, 9.3 — khảo sát cấu trúc và chất lượng từng tập
2	Xây dựng biểu diễn dữ liệu phù hợp cho mô hình	Chương II, và mục 7.5, 8.5, 9.5
3	Tiền xử lý dữ liệu một cách có hệ thống	Chương III, và mục 7.4, 8.4, 9.4
4	Xây dựng và so sánh nhiều mô hình học máy	Chương V; 5 + 5 + 6 mô hình ở mục 7.7, 8.7, 9.7
5	Đánh giá và lựa chọn mô hình phù hợp	Chương IV, và mục 7.8–7.9, 8.8–8.9, 9.8–9.9
6	Lưu trữ mô hình và quy trình tiền xử lý	Chương VI, mục 6.2
7	Triển khai thành hệ thống có khả năng sử dụng	Mục 7.10, 8.10, 9.11 — REST API + Web + Mobile

Mục tiêu thứ hai là mục tiêu trung tâm. Ba ứng dụng được chọn có chủ đích để chúng đòi hỏi ba kiểu biểu diễn khác nhau: dữ liệu bảng thuần số, dữ liệu bảng có biến phân loại, và dữ liệu có văn bản. Nhờ vậy Chương X mới so sánh được chúng một cách có ý nghĩa.

1.3. Quy trình phát triển hệ thống thông minh

Cả ba ứng dụng đều đi theo đúng một quy trình tám bước.



Hình 1. Quy trình phát triển hệ thống thông minh áp dụng cho cả ba ứng dụng.

Bước	Nội dung	Kết quả tạo ra
Dữ liệu thô	Thu thập bộ dữ liệu thực tế từ Kaggle	Tệp CSV trong <code>data/</code>
Tìm hiểu	Kiểm tra cấu trúc, kiểu dữ liệu, chất lượng	Bảng khảo sát, biểu đồ phân bố
Làm sạch	Xử lý thiếu, trùng lặp, giá trị không hợp lệ, ngoại lệ	Bảng dữ liệu sạch
Biểu diễn	Chuyển thành vector, ma trận hoặc tensor	Ma trận đặc trưng X
Học	Huấn luyện nhiều mô hình trên cùng một cách chia dữ liệu	Các mô hình đã huấn luyện
Đánh giá	Đo bằng độ đo phù hợp với loại bài toán	Bảng so sánh mô hình
Lưu trữ	Lưu pipeline tiền xử lý và mô hình được chọn	Tệp <code>.joblib</code>
Triển khai	REST API + giao diện Web + giao diện Mobile	Dịch vụ chạy được

Điểm mấu chốt. Bốn bước đầu — trước khi bất kỳ mô hình nào được huấn luyện — quyết định trần hiệu năng của toàn hệ thống. Chương X sẽ chỉ ra rằng ở cả ba ứng dụng, thay đổi cách biểu diễn dữ liệu tạo ra khác biệt lớn hơn nhiều so với thay đổi thuật toán.

1.4. Tổng quan về ba ứng dụng

	Ứng dụng 1	Ứng dụng 2	Ứng dụng 3
Tên	Dự đoán bệnh tiểu đường	Dự đoán giá nhà	Hành vi và sở thích khách hàng
Loại bài toán	Phân loại nhị phân	Hồi quy	Phân loại nhị phân
Dữ liệu thô	CSV — bảng thuần số	CSV — bảng có biến phân loại	CSV — bảng + văn bản đánh giá
Một quan sát	Một lần khám bệnh nhân	Một tin rao bán bất động sản	Một lượt đánh giá sản phẩm
Số mẫu	768	30,229 → 27,513 sau làm sạch	23,486 → 22,639 sau làm sạch
Biến mục tiêu	Outcome	Price (tỷ VNĐ)	Recommended IND
Số mô hình so sánh	5	5	6
Mô hình được chọn	Random Forest	Gradient Boosting Regressor	Logistic Regression (bảng + văn bản)
Độ đo chính	Recall	MAE	F1 lớp thiểu số / ROC-AUC

1.4.1. Ứng dụng dự đoán bệnh tiểu đường

Dự đoán một bệnh nhân có thuộc lớp dương tính tiểu đường hay không, từ các chỉ số lâm sàng thu được trong một lần khám. Toàn bộ tám đặc trưng đều là số, nên đây là biểu diễn dữ liệu đơn giản nhất trong ba ứng dụng: không cần mã hoá, chỉ cần điền khuyết và chuẩn hoá thang đo.

Thách thức thật của ứng dụng này không nằm ở mô hình mà nằm ở chất lượng dữ liệu: giá trị thiếu được mã hoá bằng số 0, nên mọi hàm kiểm tra tiêu chuẩn đều báo dữ liệu sạch trong khi gần một nửa một cột là dữ liệu bìa.

1.4.2. Ứng dụng dự đoán giá nhà

Ước lượng giá rao bán của một bất động sản nhà ở tại Việt Nam. Khác ứng dụng 1 ở hai điểm: đầu ra là số thực liên tục nên bộ độ đo phải đổi hoàn toàn, và dữ liệu có biến phân

loại nên bắt buộc phải mã hoá one-hot.

Ứng dụng này còn có một cột địa chỉ ở dạng văn bản tự do. Một phép rút chuỗi đơn giản biến nó thành hai biến vị trí có sức dự báo mạnh — minh hoạ trực tiếp cho luận điểm rằng biểu diễn dữ liệu là một phần của lời giải.

1.4.3. Ứng dụng phân tích hành vi và khám phá sở thích khách hàng thương mại điện tử

Từ thông tin khách hàng và nội dung đánh giá họ viết, dự đoán khách hàng có khuyến nghị sản phẩm hay không. Đây là ứng dụng duy nhất có dữ liệu văn bản, nên là nơi trình bày đầy đủ chuỗi biến đổi mà Bài giảng 02 yêu cầu: bình luận $\rightarrow$ token $\rightarrow$ token ID $\rightarrow$ vector / embedding.

Ứng dụng này cũng cho bằng chứng mạnh nhất trong cả báo cáo: cùng một cách chia dữ liệu, mô hình chỉ dùng đặc trưng dạng bảng đạt ROC-AUC khoảng 0.552, trong khi mô hình được đọc thêm văn bản đạt 0.943.

Chương II. Cơ sở lý thuyết về biểu diễn dữ liệu

2.1. Tổng quan về học máy và hệ thống thông minh

Học máy có giám sát tìm một ánh xạ f từ không gian đặc trưng sang không gian mục tiêu, dựa trên một tập ví dụ đã biết đáp án:

$$f: X \rightarrow y, \quad X \in \mathbb{R}^{N \times d}, \quad y \in \mathbb{R}^N \text{ hoặc } \{0, 1\}^N$$

Một **hệ thống thông minh** thì rộng hơn một mô hình. Nó gồm sáu thành phần, và thiếu bất kỳ thành phần nào thì hệ thống không dùng được: dữ liệu, biểu diễn, mô hình đã học, phép đánh giá, phần mềm triển khai, và giao diện tương tác với người dùng. Assignment 02 yêu cầu đủ cả sáu.

2.2. Bài toán phân loại và hồi quy

2.2.1. Bài toán phân loại

Biến mục tiêu nhận giá trị rời rạc. Với phân loại nhị phân, $y \in \{0, 1\}$. Mô hình thường trả về một xác suất $P(y = 1 | x)$ rồi so với một ngưỡng (mặc định 0,5) để ra nhãn. Sai lầm được **đếm**: một dự đoán hoặc đúng, hoặc sai, không có mức độ ở giữa. Vì vậy công cụ đánh giá là ma trận nhầm lẫn.

Ứng dụng 1 và Ứng dụng 3 thuộc loại này.

2.2.2. Bài toán hồi quy

Biến mục tiêu nhận giá trị liên tục, $y \in \mathbb{R}$. Sai lầm được **đo bằng khoảng cách**: dự đoán 5,0 tỷ cho một căn 5,2 tỷ là gần đúng, dự đoán 9,0 tỷ là sai nặng. Không có khái niệm ma trận nhầm lẫn, không có accuracy, không có ROC-AUC.

Ứng dụng 2 thuộc loại này. Sự khác biệt này là lý do Chương IV phải trình bày hai bộ độ đo tách biệt.

2.3. Khái niệm biểu diễn dữ liệu

Bài giảng 02 giới thiệu nguyên tắc:

Dữ liệu thực tế $\rightarrow$ Biểu diễn số $\rightarrow$ Mô hình tính toán
--

Biểu diễn dữ liệu không đơn giản là cách dữ liệu được lưu trong một tệp. Cùng một tệp CSV có thể sinh ra nhiều biểu diễn số khác nhau, và mỗi biểu diễn dẫn tới một kết quả khác nhau. Khi xây dựng một biểu diễn, cần xác định rõ bảy yếu tố:

- **Shape** — kích thước dữ liệu;
- **Dtype** — kiểu dữ liệu;
- **Range** — miền giá trị;
- **Ý nghĩa của từng chiều**;
- **Cách mã hoá biến phân loại**;
- **Cách xử lý giá trị thiếu**;
- **Thứ tự cột** đưa vào mô hình.

Vì sao thứ tự cột thuộc về biểu diễn. Hai ma trận cùng kích thước nhưng thứ tự cột khác nhau là hai biểu diễn khác nhau. Mô hình nhận đầu vào theo *vị trí* chứ không theo tên cột — nếu dịch vụ Web dựng khung dữ liệu sai thứ tự, giá trị BMI sẽ được chuẩn hoá bằng tham số của cột tuổi. Không lỗi nào được ném ra, mô hình vẫn trả về một con số, chỉ là con số sai.

2.4. Vector đặc trưng

Một mẫu dữ liệu được biểu diễn bằng một vector đặc trưng gồm nhiều giá trị số:

$$x = [x_1, x_2, \dots, x_d]^T \in \mathbb{R}^d$$

trong đó x là vector đặc trưng của một mẫu, x_i là giá trị của đặc trưng thứ i , d là số lượng đặc trưng, và T biểu thị phép chuyển vị.

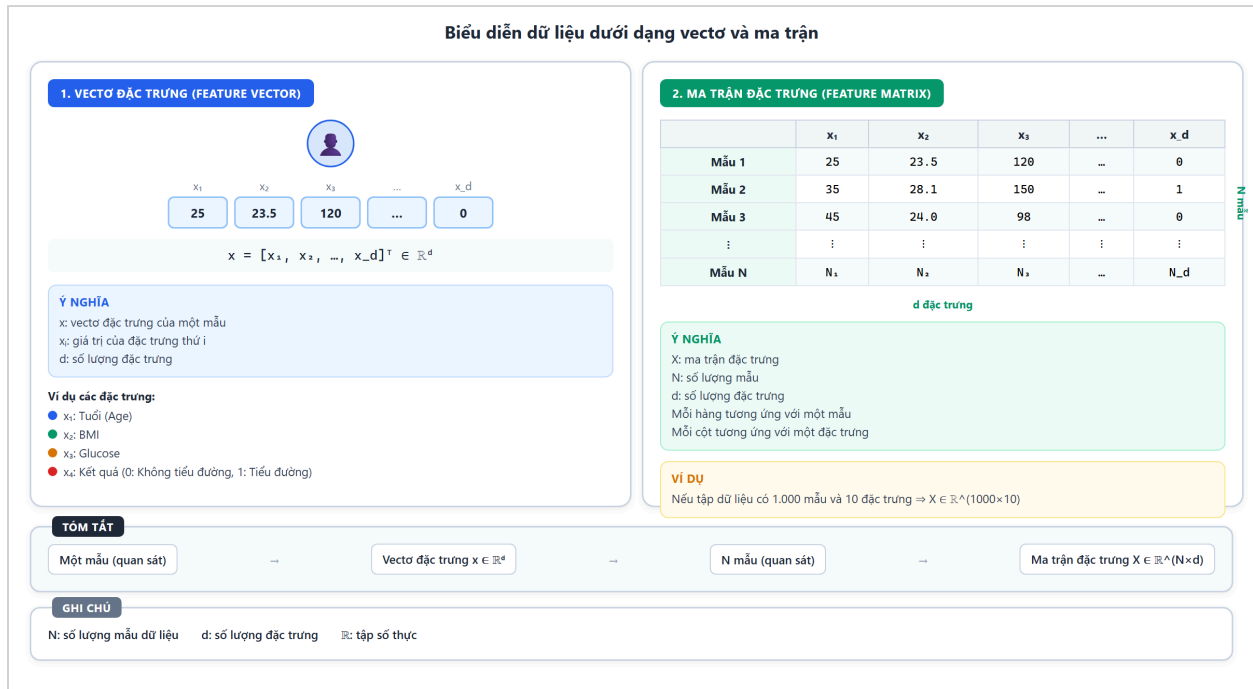
Ví dụ, một bệnh nhân trong Ứng dụng 1 được biểu diễn bởi $x = [\text{Glucose}, \text{BMI}, \text{Age}, \text{Pregnancies}, \text{DiabetesPedigreeFunction}]^T$, còn một căn nhà trong Ứng dụng 2 bởi $x = [\text{Area}, \text{Floors}, \text{Bedrooms}, \text{Bathrooms}, \dots]^T$.

2.5. Ma trận đặc trưng

Khi ghép nhiều vector đặc trưng lại theo hàng, ta được ma trận đặc trưng:

$$X \in \mathbb{R}^{N \times d}$$

với N là số lượng mẫu, d là số lượng đặc trưng; mỗi hàng tương ứng một mẫu, mỗi cột tương ứng một đặc trưng. Nếu tập dữ liệu có 1.000 mẫu và 10 đặc trưng thì $X \in \mathbb{R}^{1000 \times 10}$.



Hình 2. Minh họa biểu diễn dữ liệu dưới dạng vector và ma trận đặc trưng.

2.6. Tensor và biểu diễn dữ liệu nhiều chiều

Tensor là dạng biểu diễn có thể chứa nhiều hơn hai chiều, dùng khi cấu trúc dữ liệu cần nhiều chiều mới thể hiện đủ thông tin.

Với dữ liệu văn bản, nếu có B văn bản trong một lô, mỗi văn bản có T token, và mỗi token được biểu diễn bằng một vector embedding d chiều, thì dữ liệu embedding có dạng:

$$E \in \mathbb{R}^{B \times T \times d}$$

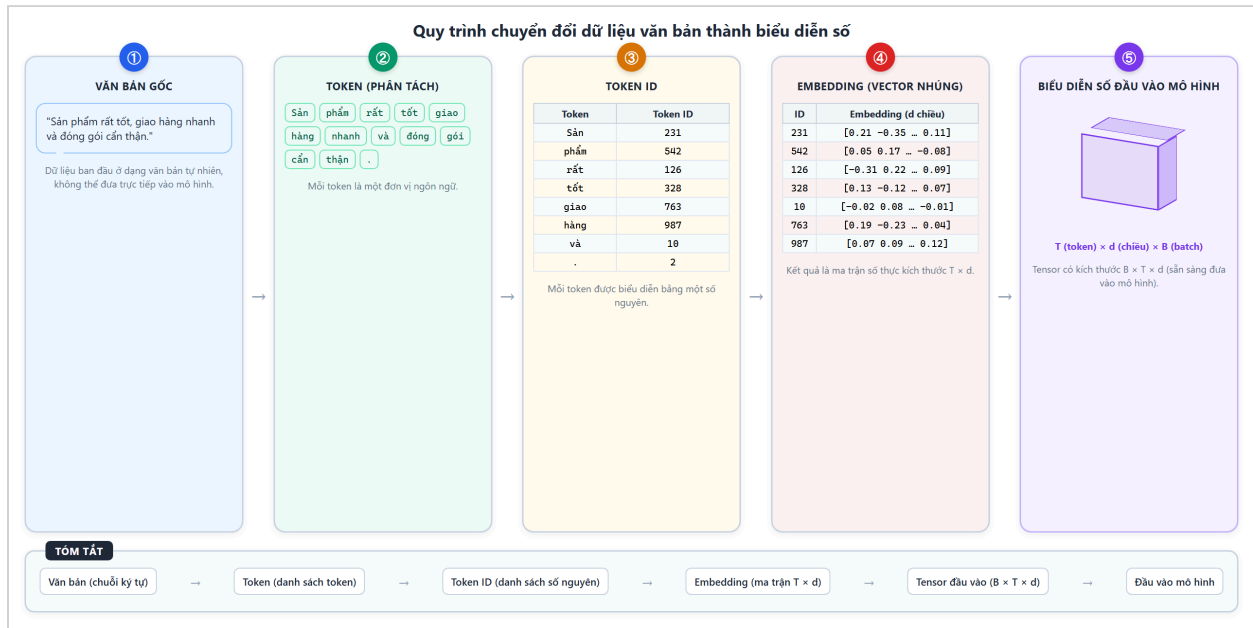
Ứng dụng 3 dựng thật tensor này với $B = 4$, $T = 24$, $d = 16$ — xem mục 9.5.

2.7. Biểu diễn dữ liệu văn bản

Khi bộ dữ liệu chứa bình luận hoặc đánh giá của khách hàng, văn bản phải đi qua chuỗi biến đổi sau trước khi mô hình dùng được:

Văn bản $\rightarrow$ Token $\rightarrow$ Token ID $\rightarrow$ Vector / Embedding

Token là các đơn vị được tạo ra từ văn bản qua quá trình tách token. Tùy phương pháp, một token có thể là một từ, một phần của từ, hoặc một ký hiệu. **Token ID** là số nguyên mà mỗi token được ánh xạ tới. Bước này đưa văn bản về dạng số, nhưng bản thân ID chưa mang ngữ nghĩa: ID 5 không “gần” ID 4 hơn ID 900 về mặt ý nghĩa. **Embedding** là vector số nhiều chiều biểu diễn token trong không gian số, trong đó khoảng cách giữa hai vector mới thực sự phản ánh quan hệ ngữ nghĩa.



Hình 3. Quy trình chuyển đổi dữ liệu văn bản thành biểu diễn số.

Ứng dụng 3 sử dụng **TF-IDF** để triển khai thay vì embedding học từ đầu. TF-IDF cho mỗi văn bản một vector duy nhất $X \in \mathbb{R}^{N \times d}$, mất thứ tự từ nhưng bù lại mỗi chiều tương ứng một từ cụ thể nên **đọc được**. Lý do chọn và cái giá phải trả được phân tích ở mục 9.5.

2.8. Mã hoá dữ liệu phân loại

Các đặc trưng dạng phân loại không thể đưa trực tiếp vào phần lớn mô hình học máy, nên phải chuyển thành số. Hai phương pháp phổ biến:

- **Label Encoding** — ánh xạ mỗi hạng mục thành một số nguyên;
- **One-Hot Encoding** — mỗi hạng mục thành một cột nhị phân riêng.

Ví dụ, tập hạng mục {Urban, Suburban, Rural} qua one-hot trở thành Urban $\rightarrow [1, 0, 0]$, Suburban $\rightarrow [0, 1, 0]$, Rural $\rightarrow [0, 0, 1]$.



Hình 4. So sánh Label Encoding và One-Hot Encoding.

Vì sao báo cáo này dùng one-hot chứ không dùng label encoding. Label encoding áp đặt một thứ tự không có thật lên dữ liệu. Nếu gán Đông = 0, Tây = 1, Nam = 2, mô hình sẽ hiểu rằng Nam “lớn hơn” Tây và khoảng cách Đông → Nam gấp đôi khoảng cách Đông → Tây. Với hướng nhà, điều đó vô nghĩa. One-hot không giả định gì về thứ tự, đổi lại làm tăng số chiều — ở Ứng dụng 2, từ 17 cột lên 111 cột.

2.9. Kiểu dữ liệu và miền giá trị

Biểu diễn dữ liệu không chỉ phụ thuộc kích thước mà còn phụ thuộc kiểu dữ liệu và miền giá trị. Sáu yếu tố cần kiểm tra: Shape, Dtype, Range, Encoding, Ordering (thứ tự đặc trưng), và Preprocessing.

Miền giá trị đặc biệt quan trọng với các mô hình dựa trên khoảng cách. Trong Ứng dụng 1, Glucose có thang 0–200 còn DiabetesPedigreeFunction có thang 0–2,4. Nếu không chuẩn hoá, khoảng cách Euclid giữa hai bệnh nhân gần như chỉ còn phản ánh mỗi chênh lệch glucose — bốn đặc trưng còn lại bị vô hiệu hoá trên thực tế mà không có cảnh báo nào.

2.10. Tính nhất quán của biểu diễn dữ liệu

Biểu diễn dữ liệu phải được duy trì nhất quán từ quá trình huấn luyện đến quá trình triển khai. Các bước như mã hoá, chuẩn hoá hoặc xử lý giá trị thiếu dùng lúc huấn luyện phải được áp dụng *y hệt* khi mô hình nhận dữ liệu mới.

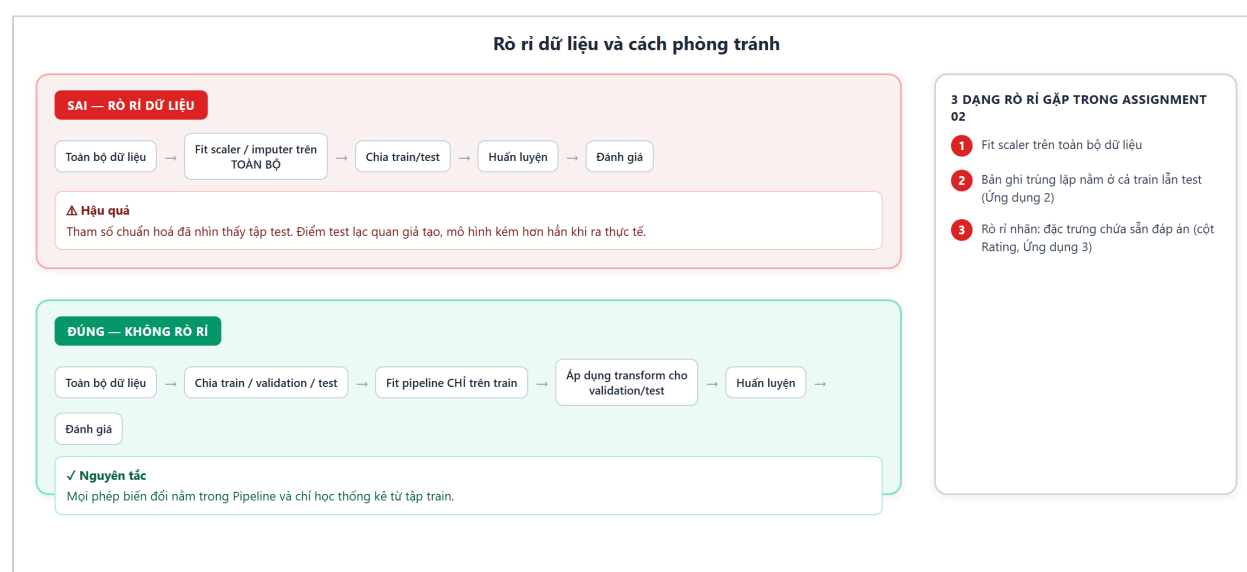
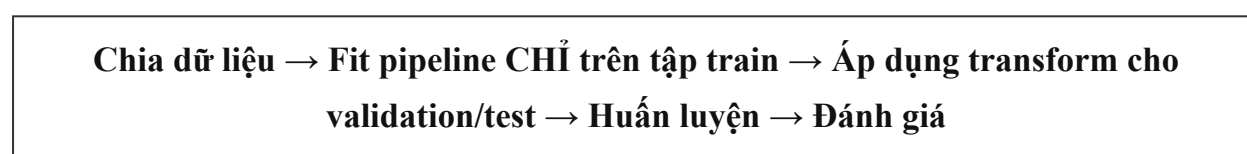
Nếu quá trình triển khai dùng một cách tiền xử lý khác với lúc huấn luyện, dữ liệu đầu vào không còn cùng ý nghĩa với dữ liệu mà mô hình đã học. Đây là lý do cả ba ứng dụng đều đóng gói tiền xử lý thành một đối tượng **Pipeline** duy nhất, lưu ra đĩa, và REST API chỉ nạp lại rồi gọi `transform` — không bao giờ `fit` lại trên dữ liệu người dùng.

2.11. Rò rỉ dữ liệu

Rò rỉ dữ liệu xảy ra khi thông tin không được phép — từ tập kiểm tra, hoặc từ tương lai — ảnh hưởng đến quá trình huấn luyện hoặc tiền xử lý.

Ví dụ kinh điển: dùng toàn bộ dữ liệu để tính tham số của bộ chuẩn hoá *trước khi* chia tập. Khi ấy trung bình và độ lệch chuẩn dùng lúc huấn luyện đã chứa thông tin từ tập kiểm tra, và điểm số cuối cùng lạc quan giả tạo.

Quy trình đúng là:



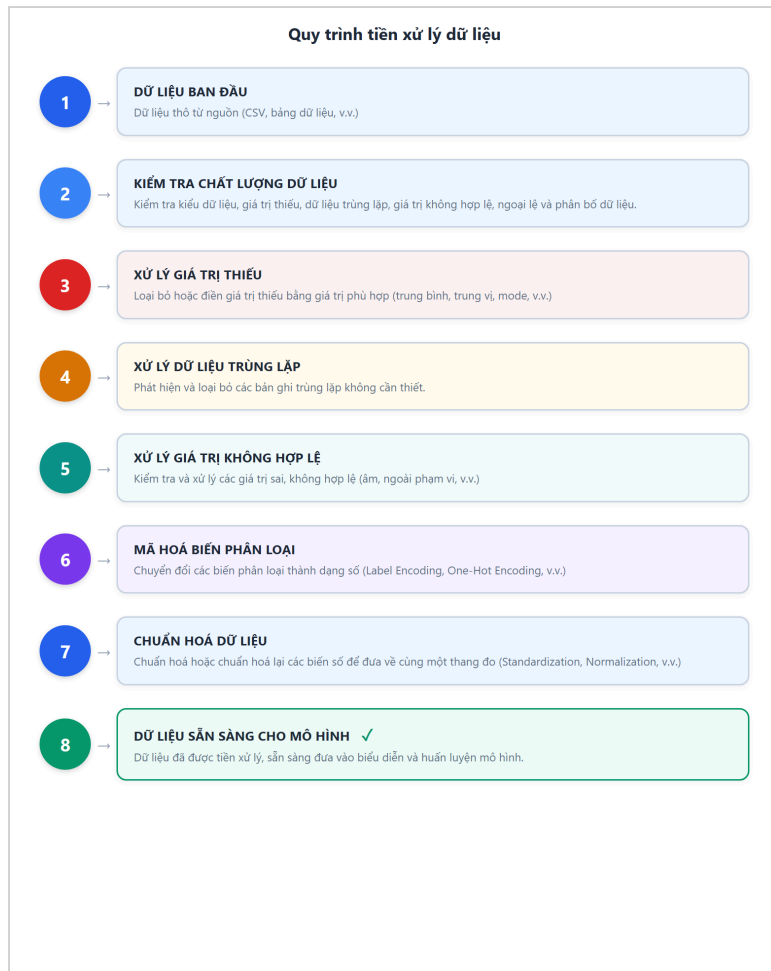
Hình 5. Rò rỉ dữ liệu và quy trình phòng tránh.

Assignment 02 gặp cả ba dạng rò rỉ, mỗi ứng dụng một dạng khác nhau:

Dạng rò rỉ	Gặp ở đâu	Vì sao nguy hiểm	Cách xử lý
Rò rỉ qua tiền xử lý	Cả ba ứng dụng	Tham số chuẩn hoá nhìn thấy tập test	Đưa mọi phép biến đổi vào <code>Pipeline</code> , chỉ fit trên train
Rò rỉ qua bản ghi trùng lặp	Ứng dụng 2	Cùng một căn nhà nằm ở cả train lẫn test; mô hình được chấm điểm trên chính dòng nó đã học thuộc	Khử trùng lặp <i>trước</i> khi chia tập — mục 8.4
Rò rỉ nhãn	Ứng dụng 3	Cột <code>Rating</code> gần như chính là đáp án; điểm số đẹp nhưng mô hình không học được gì	Loại bỏ cột khỏi tập đặc trưng — mục 9.4

Dạng thứ ba khó phát hiện nhất. Rò rỉ qua tiền xử lý bị chặn bằng một quy tắc kỹ thuật đơn giản. Rò rỉ nhãn thì không: nó không vi phạm bất kỳ quy tắc pipeline nào, không có công cụ nào cảnh báo, và biểu hiện duy nhất là điểm số *quá đẹp*. Chỉ có hiểu biết về ý nghĩa từng cột mới phát hiện được.

Chương III. Phương pháp tiền xử lý dữ liệu



Hình 6. Quy trình tiền xử lý dữ liệu tám bước.

3.1. Tìm hiểu dữ liệu

Trước khi xây dựng mô hình, dữ liệu cần được kiểm tra để xác định cấu trúc và chất lượng. Sáu lệnh khảo sát chuẩn được dùng thống nhất ở cả ba ứng dụng:

PYTHON

```
df.shape           # kích thước tập dữ liệu
df.head()          # xem vài bản ghi đầu
df.info()           # kiểu dữ liệu từng cột, số giá trị không rỗng
df.describe()       # thống kê mô tả các cột số
df.isna().sum()     # số giá trị thiếu theo cột
df.duplicated().sum() # số bản ghi trùng lặp
```

Sáu lệnh khảo sát bắt buộc theo yêu cầu đề bài.

Cảnh báo quan trọng. `isna().sum()` chỉ đếm giá trị `NaN`. Nó *không* phát hiện giá trị thiếu được mã hoá bằng một con số hợp lệ về kiểu — chẳng hạn số 0. Ứng dụng 1 là ví dụ trực tiếp: hàm này trả về 0 ở mọi cột, trong khi 48,7% cột `Insulin` là dữ liệu thiếu. Chất lượng dữ liệu là việc của người đọc dữ liệu, không phải của một hàm.

3.2. Làm sạch dữ liệu

3.2.1. Xử lý giá trị thiếu

Cách xử lý	Khi nào dùng	Rủi ro	Dùng ở ứng dụng
Xoá dòng	Tỷ lệ thiếu rất thấp và thiếu ngẫu nhiên	Mất dữ liệu; lệch mẫu nếu thiếu có hệ thống	Ứng dụng 3 (dòng không có bình luận)
Xoá cột	Thiếu trên 80% và cột ít giá trị dự báo	Mất thông tin nếu bản thân sự vắng mặt mang tín hiệu	Ứng dụng 1 (3 cột bị loại)
Điền bằng trung bình	Phân phối gần đối xứng	Bị đuôi dài kéo lệch	Không dùng
Điền bằng trung vị	Phân phối lệch, có ngoại lệ	Làm giảm phương sai thật của cột	Ứng dụng 1, 2, 3 — mọi cột số
Coi là một hạng mục riêng	Biến phân loại; sự vắng mặt mang thông tin	Tăng số chiều one-hot	Ứng dụng 2 (hạng mục “Không rõ”)

Báo cáo này chọn **trung vị** chứ không phải trung bình cho mọi cột số, vì cả ba tập dữ liệu đều có các cột lệch phải mạnh — trung bình sẽ bị đuôi dài kéo lệch trong khi trung vị bền với ngoại lệ.

3.2.2. Xử lý dữ liệu trùng lặp

Bản ghi trùng lặp phải được khử **trước khi chia tập**. Nếu không, một bản ghi có thể nằm cả ở tập train lẫn tập test; mô hình được chấm điểm trên chính dòng nó đã học thuộc, và điểm test bị thổi phồng mà không có dấu hiệu nào lộ ra. Mục 8.4 đo trực tiếp mức thổi phồng này trên dữ liệu thật.

3.2.3. Xử lý giá trị không hợp lệ

Giá trị không hợp lệ là giá trị đúng kiểu dữ liệu nhưng vô nghĩa trong miền ứng dụng: huyết áp bằng 0, diện tích âm, giá bằng 0. Chúng nguy hiểm hơn giá trị thiếu vì không công cụ nào tự phát hiện — chỉ hiểu biết về miền ứng dụng mới nhận ra.

3.2.4. Phân tích ngoại lệ

Ngoại lệ được phát hiện bằng quy tắc IQR: một giá trị nằm ngoài khoảng $[Q1 - 1,5 \cdot IQR, Q3 + 1,5 \cdot IQR]$ được coi là ngoại lệ, với $IQR = Q3 - Q1$.

Phát hiện xong *không* đồng nghĩa với việc phải xoá. Câu hỏi đúng là: đây là lỗi nhập liệu, hay là một quan sát thật mang đúng tín hiệu ta muốn mô hình học? Trong Ứng dụng 1, insulin rất cao là biểu hiện kháng insulin thật — xoá đi là xoá mất chính tín hiệu bệnh lý. Báo cáo này giữ lại ngoại lệ thật và phòng thủ bằng cách điền khuyết theo trung vị và chuẩn hoá thang đo.

3.3. Mã hoá biến phân loại

Hai phương pháp đã trình bày ở mục 2.8. Báo cáo này dùng **one-hot** cho toàn bộ biến phân loại, với tham số `handle_unknown="ignore"`.

Tham số ấy quan trọng cho khâu triển khai: nếu người dùng Web nhập một tỉnh không có trong tập huấn luyện, bộ mã hoá trả về vector toàn 0 thay vì ném lỗi và làm sập API.

3.4. Chuẩn hoá dữ liệu số

`StandardScaler` đưa mỗi cột về trung bình 0 và độ lệch chuẩn 1:

$$z = (x - \mu) / \sigma$$

trong đó μ và σ được tính **chỉ trên tập train**.

Chuẩn hoá là bắt buộc với KNN và SVM vì chúng dựa trên khoảng cách. Với mô hình dạng cây thì không cần — cây chia theo ngưỡng trên từng biến nên miễn nhiễm với thang đo — nhưng để trong pipeline chung vẫn vô hại và giữ cho mọi mô hình dùng đúng một biểu diễn, nhờ đó bảng so sánh mới công bằng.

3.5. Chia tập dữ liệu

Đề bài yêu cầu ba tập độc lập:

$$D = D_{train} \cup D_{validation} \cup D_{test}$$

Báo cáo dùng tỷ lệ **70 / 15 / 15** cho cả ba ứng dụng. Vai trò của từng tập khác nhau và không được lẫn lộn:

- **Train** — học tham số mô hình và tham số tiền xử lý;
- **Validation** — so sánh và chọn mô hình;
- **Test** — chỉ dùng *một lần*, để ước lượng sai số cuối cùng trên dữ liệu chưa từng thấy.

Với hai bài toán phân loại, phép chia được **phân tầng** (`stratify=y`) để giữ tỷ lệ hai lớp gần như đồng nhất ở cả ba tập. Không phân tầng thì tập test có thể có tỷ lệ lớp khác hẳn train, và khi ấy điểm test đo lẫn cả sự lệch phân bố lẫn chất lượng mô hình — không tách được hai thứ.

Vì sao cần tập validation riêng. Nếu chọn mô hình dựa trên chính tập test, tập test trở thành một tập validation trá hình: ta đã dùng nó để ra quyết định, nên nó không còn “chưa từng thấy” nữa và ước lượng cuối cùng lại lạc quan thêm một lần. Đây là một trong bốn cải tiến của Assignment 02 so với Assignment 01 — xem mục 8.4.

3.6. Ngăn ngừa rò rỉ dữ liệu

Ba biện pháp được áp dụng thống nhất ở cả ba ứng dụng: khử trùng lặp trước khi chia tập; đóng gói toàn bộ tiền xử lý vào `Pipeline` và chỉ `fit` trên train; và loại bỏ mọi đặc trưng chứa sẵn thông tin về nhãn.

3.7. Pipeline tiền xử lý

`ColumnTransformer` cho phép áp dụng các phép biến đổi khác nhau cho các nhóm cột khác nhau, rồi ghép kết quả thành một ma trận duy nhất. Đoạn mã dưới là pipeline thật của Ứng dụng 2, lấy nguyên văn từ notebook đã chạy:

PYTHON

```
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

numeric_branch = Pipeline([
    ("impute", SimpleImputer(strategy="median")),
    ("scale", StandardScaler()),
])
categorical_branch = Pipeline([
    ("impute", SimpleImputer(strategy="constant", fill_value="Không rõ")),
    ("onehot", OneHotEncoder(handle_unknown="ignore", sparse_output=False)),
])

preprocessor = ColumnTransformer([
    ("num", numeric_branch, NUM_FEATURES),
    ("cat", categorical_branch, CAT_FEATURES),
], remainder="drop")

preprocessor.fit(X_train)          # CHỈ fit trên train
Xtr = preprocessor.transform(X_train)
Xva = preprocessor.transform(X_val)
Xte = preprocessor.transform(X_test)

feat_names = preprocessor.get_feature_names_out()
print("Số chiều TRƯỚC mã hoá :", X_train.shape[1], "cột "
      f"({len(NUM_FEATURES)} số + {len(CAT_FEATURES)} phân loại)")
print("Số chiều SAU mã hoá   :", Xtr.shape[1], "cột")
print(f"    → one-hot làm tăng {Xtr.shape[1] - len(NUM_FEATURES)} cột từ "
      f"{len(CAT_FEATURES)} biến phân loại")
print("\nHình dạng đầu vào mô hình:")
print("  Xtr:", Xtr.shape, "| Xva:", Xva.shape, "| Xte:", Xte.shape)
print("  dtype:", Xtr.dtype)
print("\n20 tên cột đầu tiên sau biến đổi:")
print(list(feat_names[:20]))
```

Pipeline hai nhánh của Ứng dụng 2 — nhánh số và nhánh phân loại.

Lợi ích không chỉ là gọn mã. Khi gọi `preprocessor.fit(X_train)`, mọi bộ điền khuyết, bộ chuẩn hoá và bộ mã hoá bên trong **chỉ nhìn thấy tập train**. Đây là biện pháp chống rò rỉ đáng tin cậy nhất vì nó đúng theo cấu trúc chứ không phụ thuộc vào việc người viết có nhớ hay không.

3.8. Nguyên tắc tái lập quá trình tiền xử lý

Toàn bộ pipeline được lưu ra đĩa bằng `joblib` cùng với mô hình. Lúc triển khai, dịch vụ nạp lại chính đối tượng đó và gọi `transform`. Ba nguyên tắc bắt buộc:

1. **Không bao giờ `fit` lại** trên dữ liệu người dùng.
2. **Giữ đúng thứ tự cột** — thứ tự được lưu trong `metadata.json` và dịch vụ dựng khung dữ liệu theo đúng nó.
3. **Kiểm chứng sau khi nạp lại** — cả ba notebook đều có một phép khẳng định rằng artifact nạp từ đĩa cho kết quả trùng khớp hoàn toàn với mô hình trong bộ nhớ trên toàn bộ tập test.

Chương IV. Các độ đo đánh giá mô hình

Loại bài toán quyết định bộ độ đo. Dùng nhầm bộ độ đo là một lỗi nghiêm trọng hơn dùng nhầm thuật toán, vì nó khiến ta chọn sai mô hình mà vẫn tưởng mình đang tối ưu đúng thứ.



Hình 7. Các độ đo đánh giá cho bài toán phân loại và hồi quy.

4.1. Đối với bài toán Hồi quy

Độ đo	Công thức	Đơn vị	Ý nghĩa trong bài toán giá nhà
MAE	$(1/N) \sum y_i - \hat{y}_i $	tỷ VNĐ	Sai lệch trung bình. Con số dễ giải thích nhất cho người dùng cuối.
MSE	$(1/N) \sum (y_i - \hat{y}_i)^2$	(tỷ VNĐ) ²	Phạt nặng các cú trượt lớn. Không đọc trực tiếp được vì đơn vị bị bình phương.
RMSE	$\sqrt{MSE}$	tỷ VNĐ	Đưa MSE về lại đơn vị gốc. So RMSE với MAE cho biết sai số rải đều hay tập trung ở vài cú trượt lớn.
R ²	$1 - SS_{res} / SS_{tot}$	không đơn vị	Tỷ lệ phương sai giá được giải thích. 1 là hoàn hảo, 0 ngang mô hình cơ sở, âm là tệ hơn cả việc luôn đoán giá trung bình.

Cách đọc cặp MAE và RMSE. RMSE luôn lớn hơn hoặc bằng MAE. Khoảng cách giữa hai con số cho biết phân bố sai số: nếu RMSE xấp xỉ MAE thì sai số rải khá đều; nếu RMSE lớn hơn MAE nhiều thì có một nhóm nhỏ dự đoán sai rất nặng đang kéo chỉ số lên. Ứng dụng 2 rơi vào trường hợp thứ hai, và mục 8.8 truy ra nhóm ấy nằm ở hai đầu phổ giá.

4.2. Đối với bài toán Phân lớp

Mọi độ đo phân lớp đều bắt nguồn từ bốn ô của ma trận nhầm lẫn: TP (dương tính thật), TN (âm tính thật), FP (dương tính giả) và FN (âm tính giả).

Độ đo	Công thức	Trả lời câu hỏi gì	Khi nào gây hiểu lầm
Accuracy	$(TP + TN) / (TP + TN + FP + FN)$	Tỷ lệ dự đoán đúng nói chung	Khi hai lớp mất cân bằng. Ở Ứng dụng 3, luôn đoán “khuyến nghị” đã đạt 81.9% mà không học gì.
Precision	$TP / (TP + FP)$	Trong các trường hợp bị báo động, bao nhiêu phần trăm là báo động thật	Khi FN mới là loại sai lầm đắt tiền
Recall	$TP / (TP + FN)$	Trong các trường hợp thật sự dương tính, bắt được bao nhiêu phần trăm	Khi FP đắt; có thể nâng Recall lên 1 bằng cách đoán tất cả là dương tính
F1	$2 \cdot (P \cdot R) / (P + R)$	Trung bình điều hoà của Precision và Recall	Che giấu việc một trong hai thành phần rất kém
ROC - AUC	diện tích dưới đường cong ROC	Khả năng phân tách hai lớp ở mọi ngưỡng	Có thể trông đẹp ngay cả khi ngưỡng 0,5 cho kết quả thực tế tệ

Ma trận nhầm lẫn phải được diễn giải, không chỉ hiển thị. Ý nghĩa của bốn ô phụ thuộc hoàn toàn vào bài toán, và chính điều đó quyết định độ đo nào là quan trọng nhất:

Ứng dụng	FN nghĩa là gì	FP nghĩa là gì	Cái nào đắt hơn	Độ đo chính
1 — Tiểu đường	Bệnh nhân có bệnh bị bỏ sót	Người khỏe bị cảnh báo nhầm, phải làm thêm một xét nghiệm	FN đắt hơn nhiều	Recall
3 — Thương mại điện tử	Khách hài lòng bị đoán là không hài lòng — chỉ gây rà soát thừa	Phản hồi xấu bị bỏ lọt, sản phẩm lỗi tiếp tục được bán	FP đắt hơn	F1 lớp thiểu số

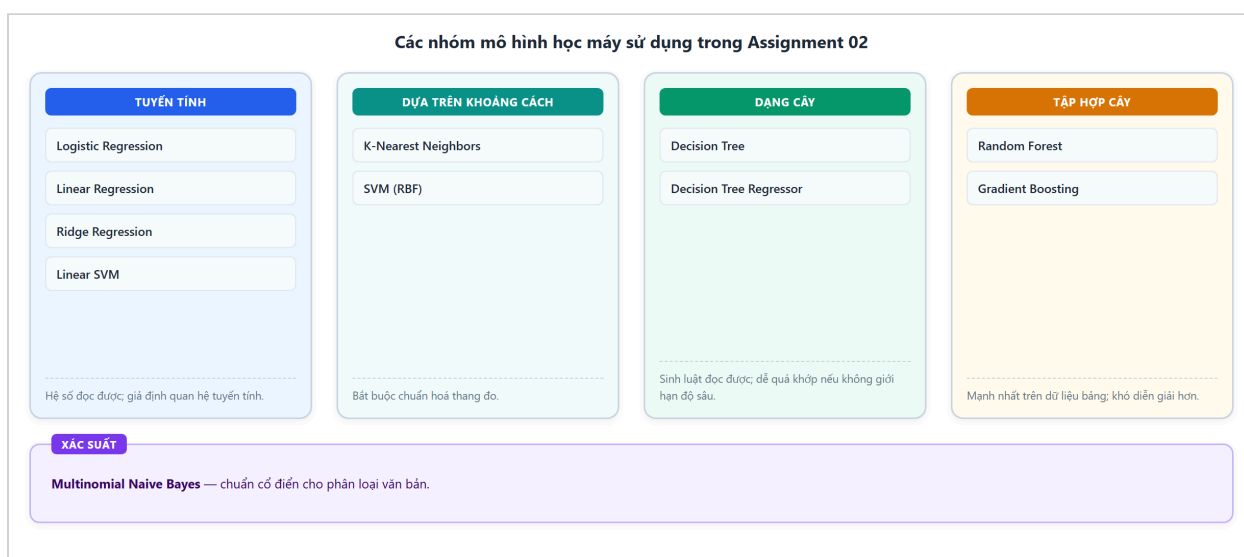
Hai ứng dụng cùng là phân loại nhị phân nhưng ưu tiên ngược nhau. Đây là lý do báo cáo không dùng một độ đo duy nhất cho mọi bài toán, và là lý do `class_weight="balanced"` được bật ở những mô hình hỗ trợ — nó nhân trọng số lỗi của lớp thiểu số lên, đẩy mô hình về phía ưu tiên đúng với bài toán.

Chương V. Phương pháp xây dựng mô hình

5.1. Tổng quan về xây dựng mô hình

Assignment 02 yêu cầu so sánh năm mô hình cho bài toán tiểu đường, năm mô hình cho bài toán giá nhà và sáu mô hình cho ứng dụng thương mại điện tử — tổng cộng 16 mô hình.

Nguyên tắc so sánh: mọi mô hình trong cùng một ứng dụng được huấn luyện trên **cùng một cách chia dữ liệu** và **cùng một biểu diễn** (trừ khi bản thân biểu diễn mới là thứ đang được so sánh, như ở Ứng dụng 3). Chỉ khi đó chênh lệch điểm số mới quy được về chất lượng mô hình chứ không lẫn yếu tố nào khác.



Hình 8. Các nhóm mô hình học máy được sử dụng.

5.2. Mô hình Baseline

Trước khi huấn luyện bất kỳ mô hình nào, phải biết ngưỡng vô nghĩa nằm ở đâu.

- **Phân loại** — `DummyClassifier(strategy="most_frequent")`: luôn đoán lớp đa số.
- **Hồi quy** — `DummyRegressor(strategy="mean")`: luôn đoán giá trị trung bình; theo định nghĩa có $R^2 = 0$.

Mô hình nào không vượt được baseline thì không có giá trị, dù điểm số trông cao. Baseline của Ứng dụng 3 đạt tới 81.9% accuracy — cao hơn nhiều mô hình thật ở các bài toán khác, mà vẫn hoàn toàn vô dụng.

5.3. Các mô hình cho bài toán hồi quy

Mô hình	Nguyên lý	Ưu điểm	Hạn chế
Linear Regression	Tìm siêu phẳng cực tiểu tổng bình phương sai số	Hệ số đọc được trực tiếp thành “mỗi m ² thêm bao nhiêu tỷ”	Chỉ bắt được quan hệ tuyến tính
Ridge Regression	Hồi quy tuyến tính có phạt L2 trên hệ số	Ổn định hoá hệ số khi one-hot sinh ra nhiều cột tương quan	Vẫn là mô hình tuyến tính
Decision Tree Regressor	Chia không gian đặc trưng bằng các ngưỡng liên tiếp	Bắt được phi tuyến và tương tác giữa các biến	Dễ quá khớp; dự đoán bậc thang, không mượt
Random Forest Regressor	Trung bình hoá nhiều cây trên các mẫu bootstrap	Giảm mạnh phương sai; bền với ngoại lệ	Khó diễn giải; tệp mô hình lớn
Gradient Boosting Regressor	Các cây học tuần tự trên phần dư của cây trước	Thường chính xác nhất trên dữ liệu bảng	Huấn luyện lâu; nhạy với siêu tham số

Giới hạn chung của mô hình dạng cây trong bài toán hồi quy. Mỗi lá trả về trung bình của các mẫu rơi vào lá đó, nên mô hình **không bao giờ dự đoán vượt ra ngoài khoảng giá đã thấy lúc huấn luyện**. Hệ quả là hiện tượng co về trung bình: căn rễ bị đoán đắt lên, căn đất bị đoán rẻ đi. Mục 8.8 chỉ ra hiện tượng này trên dữ liệu thật và giải thích vì sao giao diện Web phải hiển thị khoảng dao động thay vì một con số duy nhất.

5.4. Các mô hình cho bài toán phân loại

Mô hình	Nguyên lý	Ưu điểm	Hạn chế
Logistic Regression	Ánh xạ tổ hợp tuyến tính qua hàm sigmoid thành xác suất	Chuẩn tham chiếu trong y học; hệ số diễn giải được thành tỷ số chênh	Chỉ tạo ra ranh giới tuyến tính
K-Nearest Neighbors	Bỏ phiếu theo k mẫu gần nhất	Không tham số; không giả định dạng phân phối	Bắt buộc chuẩn hoá; chậm khi dữ liệu lớn
Decision Tree	Chia dữ liệu theo ngưỡng trên từng đặc trưng	Sinh ra luật đọc được, chuyên gia kiểm tra được	Dễ quá khớp nếu không giới hạn độ sâu
Random Forest	Tập hợp nhiều cây trên mẫu bootstrap	Giảm phương sai; cho tầm quan trọng đặc trưng	Hộp đen so với cây đơn
SVM (RBF)	Tìm siêu phẳng biên lớn trong không gian ánh xạ phi tuyến	Bắt được ranh giới phi tuyến phức tạp	Bắt buộc chuẩn hoá; chi phí bậc hai theo số mẫu
Linear SVM	Siêu phẳng biên lớn trong không gian gốc	Rất mạnh trên dữ liệu văn bản thưa nhiều chiều	Không cho xác suất trực tiếp
Multinomial Naive Bayes	Xác suất Bayes với giả định độc lập giữa các đặc trưng	Chuẩn cổ điển cho phân loại văn bản; huấn luyện gần như tức thì	Yêu cầu đặc trưng không âm; giả định độc lập không đúng thực tế

Xử lý mất cân bằng lớp. Cả Ứng dụng 1 (tỷ lệ 1,87 : 1) và Ứng dụng 3 (4,5 : 1) đều mất cân bằng. Tham số `class_weight="balanced"` được bật ở Logistic Regression, Decision Tree, Random Forest và SVM. Nó nhân trọng số lỗi của lớp thiểu số theo tỷ lệ nghịch với tần suất, khiến mô hình chịu thiệt nhiều hơn khi bỏ sót lớp hiếm — đúng với ưu tiên đã phân tích ở mục 4.2.

Chương VI. Phương pháp lưu trữ và triển khai mô hình

6.1. Tổng quan

Giai đoạn triển khai nối thực nghiệm học máy với một hệ thống phần mềm dùng được. Bốn thành phần bắt buộc:

Bộ tiền xử lý đã lưu + Mô hình đã lưu + REST API + Giao diện Web và Mobile

Phân biệt quan trọng: **huấn luyện** $\neq$ **suy luận**. Mô hình được huấn luyện một lần trong giai đoạn thực nghiệm rồi lưu lại. Dịch vụ Web và ứng dụng Mobile chỉ thực hiện suy luận trên mô hình đã lưu, không bao giờ huấn luyện lại.

6.2. Lưu trữ mô hình

Bốn thứ phải được lưu, theo đúng yêu cầu đề bài:

Thành phần	Tệp	Nội dung
Tiền xử lý + biến đổi đặc trưng	<code>preprocessor.r.joblib</code>	Toàn bộ Pipeline: điền khuyết, chuẩn hoá, one-hot, TF-IDF — kèm mọi tham số đã học từ tập train
Mô hình đã huấn luyện	<code><tên_mô_hình>.joblib</code>	Cả bộ mô hình được lưu, để giao diện Web cho phép người dùng đổi mô hình và so sánh trực tiếp
Cấu hình mô hình	<code>metadata.json</code>	Thứ tự cột, danh sách hạng mục hợp lệ, nhãn lớp, mô hình được chọn, toàn bộ điểm số trên tập test

PYTHON

```
import joblib

joblib.dump(preprocessor, MODEL_DIR / "preprocessor.joblib")
joblib.dump(best_model, MODEL_DIR / "random_forest.joblib")
```

Lưu artifact bằng joblib.

Vì sao thứ tự cột phải được lưu. `preprocessor.transform` nhận dữ liệu theo **vị trí**, không theo tên cột. Nếu REST API dựng khung dữ liệu theo thứ tự khác lúc huấn luyện, giá trị BMI sẽ được chuẩn hoá bằng trung bình và độ lệch chuẩn của tuổi. Không lỗi nào được ném ra và mô hình vẫn trả về một con số — chỉ là con số sai. `metadata.json` tồn tại để chặn đúng lỗi này.

6.3. Tính nhất quán giữa huấn luyện và dự đoán

Quy trình tiền xử lý lúc dự đoán phải trùng khớp quy trình lúc huấn luyện. Cụ thể, dịch vụ triển khai **không được** tạo mới scaler, encoder, imputer hay bộ vector hoá từ dữ liệu người dùng, mà phải nạp lại pipeline đã học từ tập train.

Cả ba notebook đều kết thúc bằng một phép kiểm chứng: nạp artifact từ đĩa, chạy trên toàn bộ tập test, và khẳng định kết quả trùng khớp hoàn toàn với mô hình đang nằm trong bộ nhớ. Nếu phép khẳng định này sai thì artifact đã hỏng và dịch vụ Web sẽ trả về kết quả khác với thực nghiệm.

PYTHON

```

loaded_pre = joblib.load(MODEL_DIR / "preprocessor.joblib")
loaded_model = joblib.load(MODEL_DIR / f"{best_id}.joblib")
meta = json.loads((MODEL_DIR / "metadata.json").read_text(encoding="utf-8"))

patients = [
    {"Glucose": 168, "BMI": 38.2, "Age": 52, "Pregnancies": 6,
     "DiabetesPedigreeFunction": 0.85},
    {"Glucose": 92, "BMI": 22.4, "Age": 24, "Pregnancies": 0,
     "DiabetesPedigreeFunction": 0.19},
]

for i, p in enumerate(patients, 1):
    frame = pd.DataFrame([p], columns=meta["feature_columns"]) # đúng thứ tự đã
    Lưu
    vec = loaded_pre.transform(frame) # transform, KHÔNG
    fit
    cls = int(loaded_model.predict(vec)[0])
    prob = float(loaded_model.predict_proba(vec)[0][cls])
    print(f"--- Bệnh nhân {i} ---")
    print("    Đầu vào thô   :", p)
    print("    Vectors chuẩn hoá:", np.round(vec[0], 4))
    print(f"    Dự đoán         : lớp {cls} - {meta['class_labels'][str(cls)]}")
    print(f"    Độ tin cậy       : {prob*100:.2f}%\n")

# Kiểm chứng: artifact nạp lại phải khớp mô hình trong bộ nhớ.
assert np.array_equal(loaded_model.predict(loaded_pre.transform(X_test)),
                      best_model.predict(Xte)), "Artifact nạp lại KHÔNG khớp!"
print("✓ Artifact nạp lại cho kết quả trùng khớp hoàn toàn trên 116 mẫu test.")
print("✓ Sẵn sàng cho REST API (api/REST_API.py).")

```

Kiểm thử suy luận của Ứng dụng 1 — chứng minh artifact tự nó chạy được.

6.4. Triển khai mô hình dưới dạng Web Service

Cả ba ứng dụng dùng **Flask**. Mỗi ứng dụng là một dịch vụ độc lập chạy trên cổng riêng, phoi ra các điểm cuối sau:

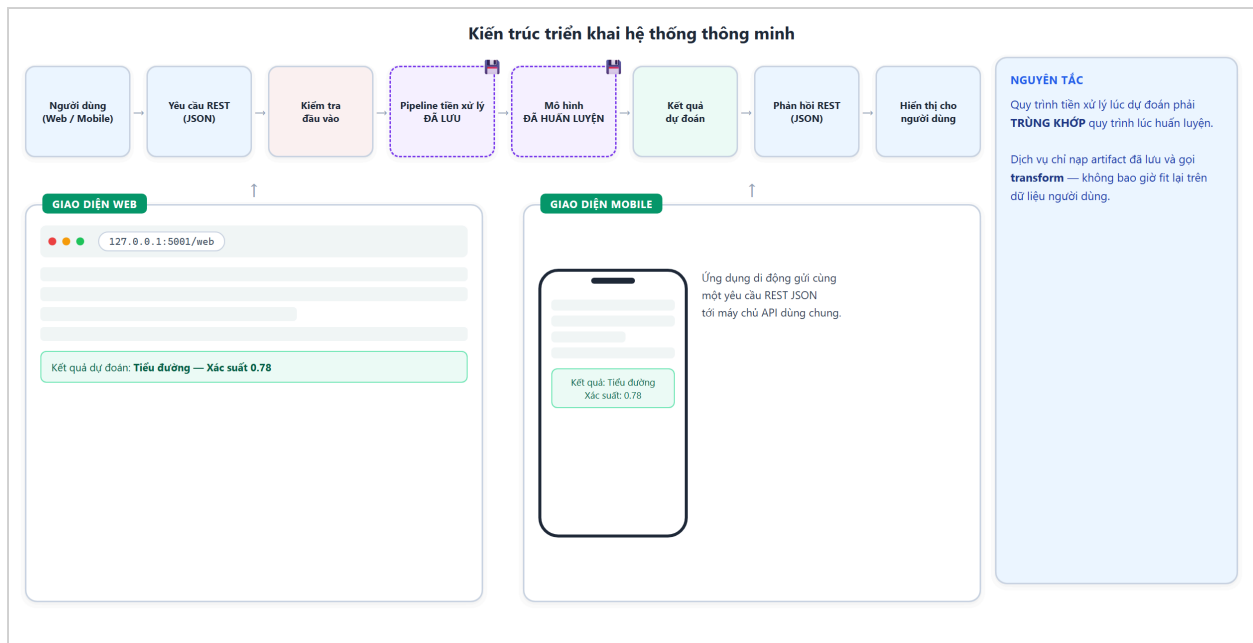
Điểm cuối	Phương thức	Chức năng
/, /web	GET	Trả về giao diện Web
/mobile	GET	Trả về giao diện Mobile
/health	GET	Kiểm tra dịch vụ sống và liệt kê mô hình khả dụng
/metadata	GET	Trả về <code>metadata.json</code> để giao diện tự động biểu mẫu
/models	GET	Liệt kê từng mô hình đã nạp kèm nhãn hiển thị, điểm số trên tập test và cờ đánh dấu mô hình tốt nhất — đọc thẳng từ <code>metadata.json</code> , không mô hình nào bị liệt kê cứng trong mã
/ <code><ứng-dụng>/v1/predict</code>	POST	Nhận JSON, trả về dự đoán

Luồng xử lý của một yêu cầu dự đoán:

JSON đầu vào → Kiểm tra hợp lệ → Tiền xử lý đã lưu → Mô hình → JSON đầu ra

Bước kiểm tra hợp lệ trả về mã 400 kèm thông báo tiếng Việt khi dữ liệu sai, thay vì để ngoại lệ lan xuống tầng mô hình và sinh ra mã 500.

Phần thân JSON của `POST /<ứng-dụng>/v1/predict` còn nhận thêm một trường tùy chọn, `model`, cho phép người gọi chỉ định mô hình nào sẽ chạy dự đoán thay vì mặc định. Quy tắc chọn mô hình ở cả ba dịch vụ đều là `model_id = payload.get('model')` or `BEST_MODEL` — một yêu cầu không kèm trường này, đúng hình dạng JSON từ trước khi tính năng chọn mô hình tồn tại, vẫn chạy đúng mô hình tốt nhất như cũ. Tham số mới vì vậy **tương thích ngược hoàn toàn**: không client cũ nào cần sửa để tiếp tục hoạt động đúng.



Hình 9. Kiến trúc triển khai hoàn chỉnh của hệ thống.

6.5. Web Application

Giao diện Web là một trang HTML tự chứa, nạp `/metadata` khi khởi động để tự dựng danh sách mô hình và các danh sách hạng mục — nhờ vậy khi notebook được chạy lại và tập hạng mục thay đổi, giao diện tự cập nhật theo mà không cần sửa mã.

Theo yêu cầu đề bài, mỗi giao diện đều có: biểu mẫu nhập liệu với kiểu điều khiển phù hợp, kiểm tra dữ liệu phía trình duyệt, nút dự đoán, kết quả dự đoán, độ tin cậy hoặc xác suất, và một câu diễn giải ngắn.

6.5.1. Sáu thành phần giao diện phục vụ khả năng diễn giải

Trả về đúng một nhãn hoặc một con số là chưa đủ cho một hệ thống được gọi là “thông minh”: người dùng cần thấy dự đoán ấy dựa trên điều gì và đáng tin tới đâu, nếu không giao diện chỉ là một lớp vỏ bọc quanh một hộp đen. Vì vậy cả ba giao diện Web đều bổ sung sáu thành phần dưới đây, tất cả đều dựng động từ `/metadata` — không mô hình, điểm số hay danh sách hạng mục nào bị gở cứng trong HTML.

#	Thành phần	Cách hoạt động	Vì sao cần thiết
1	Hộp chọn mô hình	Nạp danh sách mô hình và mô hình tốt nhất từ <code>/metadata</code> , chọn sẵn mô hình tốt nhất kèm nhãn “khuyến nghị”; người dùng có thể đổi sang mô hình khác rồi dự đoán lại trên cùng dữ liệu vừa nhập.	Biến bảng so sánh mô hình ở Chương VII–IX từ một bảng số liệu tĩnh thành một thực nghiệm người dùng tự làm được: đổi mô hình, thấy kết quả đổi theo.
2	Bảng so sánh mô hình	Đọc trường <code>test_metrics</code> trong <code>/metadata</code> , hiển thị đúng bộ độ đo theo loại bài toán (Accuracy/Precision/Recall/F1/ROC-AUC cho phân loại; MAE/RMSE/R ² cho hồi quy) và tô đậm hàng của mô hình đang chọn.	Đưa đúng con số đã dùng để chọn mô hình ở Chương VII–IX ra giao diện, để người dùng tự đọc bằng chứng thay vì tin lời khẳng định “mô hình này tốt nhất”.
3	Thanh độ tin cậy / thước đo khoảng giá	Hai bài toán phân loại hiển thị một thanh ngang theo phần trăm xác suất mô hình gán cho nhãn dự đoán, đổi màu khi rủi ro cao hoặc không khuyến nghị. Ứng dụng giá nhà — hồi quy, không có xác suất — thay bằng một vạch định vị giá dự đoán trong đoạn [giá thấp, giá cao].	Một con số trần trụi ngụ ý một sự chắc chắn không có thật. Hiển thị độ tin cậy hoặc khoảng dao động buộc người xem đọc kết quả đúng bản chất: một ước lượng, không phải một sự thật tuyệt đối.
4	Biểu đồ mức độ quan trọng đặc trưng	Ứng dụng 1 vẽ thanh ngang cho từng đặc trưng, kèm giá trị người dùng vừa nhập để đối chiếu trực tiếp. Ứng dụng 2 vẽ các đặc trưng trọng số cao nhất nhưng không đối chiếu giá trị nhập, vì phần lớn cột trọng số cao là cột sinh ra từ one-hot. Ứng dụng 3 thay biểu đồ bằng danh sách từ khoá tích cực/tiêu cực, tô đậm những từ thật sự xuất hiện trong bình luận vừa nhập.	Giải thích <i>vì sao</i> mô hình ra kết quả đó, không chỉ ra kết quả — đúng yêu cầu diễn giải được của một hệ thống thông minh.
5	Hồ sơ mẫu và nút xoá form	Mỗi ứng dụng có 2–3 nút điền nhanh một hồ sơ mẫu hợp lý về mặt lâm sàng / thị trường / văn phong đánh giá (không lấy từ bản ghi thật nào), cộng một nút xoá trắng form.	Hạ thấp rào cản kiểm thử thủ công và chụp ảnh minh hoạ — không phải gõ tay từng trường mỗi lần muốn thử một kịch bản khác.

#	Thành phần	Cách hoạt động	Vì sao cần thiết
6	Câu diễn giải bằng lời	REST API soạn sẵn một câu tiếng Việt theo lớp dự đoán (hoặc theo giá và khoảng dao động ở Ứng dụng 2) và trả về qua trường <code>interpretation</code> ; giao diện chỉ hiển thị nguyên văn, không tự suy diễn thêm.	Cầu nối giữa con số kỹ thuật (xác suất, nhãn lớp, giá dự đoán) và người đọc không chuyên — đúng tinh thần “hệ thống thông minh” chứ không chỉ là một API trả JSON.

Vì sao cả sáu đều đọc dữ liệu, không gõ cứng. Không thành phần nào trong sáu thành phần trên chứa một con số hay một tên mô hình gõ tay trong HTML — tất cả đều dựng động từ `/metadata` lúc trang tải. Notebook chạy lại sinh ra điểm số mới, tập hạng mục mới hay tầm quan trọng đặc trưng mới thì giao diện tự phản ánh đúng, không cần sửa một dòng HTML/JavaScript nào — đúng nguyên tắc một nguồn sự thật duy nhất đã áp dụng cho `metadata.json` ở mục 6.2.

6.6. Triển khai trên ứng dụng mobile

Ứng dụng Mobile đóng vai trò **máy khách của dịch vụ dự đoán**, không chứa mô hình và không huấn luyện. Kiến trúc:

Giao diện Mobile → REST API → Tiền xử lý → Mô hình học máy → Kết quả dự đoán → Giao diện Mobile

Báo cáo này minh hoạ giao diện Mobile bằng một trang mô phỏng thiết bị chạy trong trình duyệt tại điểm cuối `/mobile`: một khung điện thoại 390×844 với thanh trạng thái, thanh tiêu đề ứng dụng và biểu mẫu bố cục theo chuẩn di động. Trang này gọi **đúng điểm cuối REST** mà giao diện Web gọi, nên nó chứng minh được đầy đủ luồng Mobile → REST API → mô hình mà không cần trình giả lập Android.

Toàn bộ sáu thành phần giao diện ở mục 6.5.1 — hộp chọn mô hình, bảng so sánh mô hình, thanh độ tin cậy hoặc thước đo khoảng giá, biểu đồ hay danh sách từ khoá mức độ quan trọng, hồ sơ mẫu kèm nút xoá, và câu diễn giải — đều có mặt trên bản Mobile, chỉ khác nhau ở cách bố trí cho một màn hình 390×844 hẹp hơn nhiều so với màn hình Web. Riêng bảng so sánh mô hình được đặt trong một khối `<details>` thu gọn (nhấn vào tiêu

để dễ mở), thay vì hiển thị thường trực như trên Web, để không chiếm hết không gian màn hình vốn đã eo hẹp của thiết bị di động.

Mã nguồn Flutter (`mobile/lib/main.dart` và `mobile/pubspec.yaml`) cũng được kèm trong kho mã cho mỗi ứng dụng.

6.7. Kiến trúc hoàn chỉnh của hệ thống

Ghép mọi thành phần lại, mỗi ứng dụng có đường đi trọn vẹn sau:

Dữ liệu thực → Làm sạch → Biểu diễn (vector / ma trận / tensor) → Mô hình học máy → Đánh giá → Lưu trữ → Web API → Ứng dụng Mobile

Ba chương tiếp theo trình bày từng ứng dụng theo đúng khung này, để Chương X so sánh được chúng trên cùng một hệ quy chiếu.

6.8. Thông tin tái lập thực nghiệm

Đề bài yêu cầu một sinh viên khác phải tái lập được thí nghiệm chính và chạy được ứng dụng đã triển khai. Bảng dưới liệt kê toàn bộ thông tin cần thiết.

Hạng mục	Giá trị
Ngôn ngữ	Python 3.13.7
Hệ điều hành	Windows 11
Hạt giống ngẫu nhiên	<code>RANDOM_SEED = 42</code> — đặt cho <code>numpy</code> , <code>random</code> và mọi mô hình có tham số <code>random_state</code>
Thư viện chính	scikit-learn 1.9, pandas 3.0, numpy 2.5, matplotlib 3.11, seaborn 0.13, joblib 1.5, Flask 3.1, neo4j 6.2
Chia tập	70 / 15 / 15; phân tầng theo nhãn ở hai bài toán phân loại
Kho mã nguồn	https://github.com/Nghaiz/Assignment-02-Intelligent-System

Ứng dụng	Nguồn dữ liệu Kaggle	Tệp cục bộ
Tiểu đường	uciml/pima-indians-diabetes-database	diabetes/data/diabetes.csv
Giá nhà	nguyentiennhan/vietnam-housing-dataset-2024	house_price/data/house_prices.csv
Thương mại điện tử	nicapotato/womens-ecommerce-clothing-reviews	customer_behavior/data/womens_ecommerce_reviews.csv

6.8.1. Các bước tái lập

BASH

```
# 1. Cài môi trường
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt

# 2. Chạy Lại ba notebook – sinh lại toàn bộ mô hình, hình vẽ và metadata
jupyter nbconvert --to notebook --execute --inplace \
    diabetes/notebook/Diabetes_A02.ipynb
jupyter nbconvert --to notebook --execute --inplace \
    house_price/notebook/HousePrice_A02.ipynb
jupyter nbconvert --to notebook --execute --inplace \
    customer_behavior/notebook/CustomerBehavior_A02.ipynb

# 3. Chạy dịch vụ (mỗi lệnh một cửa sổ terminal)
python diabetes/api/REST_API.py          # cổng 5001
python house_price/api/REST_API.py       # cổng 5002
python customer_behavior/api/REST_API.py  # cổng 5003

# 4. Mở giao diện
#   Web      : http://127.0.0.1:5001/web
#   Mobile   : http://127.0.0.1:5001/mobile

# 5. Dừng lại báo cáo này
python report/build_report.py
```

Quy trình tái lập đầy đủ.

Về tính xác định của kết quả. Toàn bộ mô hình dùng chung một hạt giống ngẫu nhiên nên chạy lại cho kết quả trùng khớp. Riêng các mô hình chạy song song (`n_jobs=-1`) có thể lệch ở chữ số thập phân thứ tư do thứ tự cộng dồn dấu phẩy động khác nhau giữa các luồng; sai khác này không ảnh hưởng tới thứ hạng mô hình hay bất kỳ kết luận nào trong báo cáo.

Chương VII. Ứng dụng 1 — Dự đoán bệnh tiểu đường

7.1. Mô tả bài toán

Mục tiêu của ứng dụng này là dự đoán một bệnh nhân có thuộc lớp dương tính tiểu đường hay không, dựa trên các chỉ số lâm sàng thu được trong một lần khám. Kết quả dự đoán có thể hỗ trợ việc quyết định bệnh nhân nào cần được chỉ định làm nghiệm pháp dung nạp glucose chuyên sâu.

- X = đặc trưng bệnh nhân (các chỉ số lâm sàng và nhân trắc học);
- y = lớp tiểu đường, $y \in \{0, 1\}$.

Đây là bài toán **phân loại nhị phân có giám sát**.

Mô hình này làm gì và không làm gì. Tiểu đường type 2 tiến triển âm thầm nhiều năm trước khi có triệu chứng. Một mô hình sàng lọc chạy trên các chỉ số xét nghiệm thường quy không thay thế chẩn đoán của bác sĩ — nó **phân bổ nguồn lực chẩn đoán**. Nhận định này quyết định trực tiếp việc chọn độ đo ở mục 7.8: bỏ sót một ca bệnh đắt hơn nhiều so với cảnh báo nhầm một người khỏe.

7.2. Giới thiệu tập dữ liệu

Mục	Giá trị
Tên tập dữ liệu	Pima Indians Diabetes Database
Nguồn Kaggle	kaggle.com/datasets/uciml/pima-indians-diabetes-database
Nguồn gốc	National Institute of Diabetes and Digestive and Kidney Diseases (UCI)
Số quan sát	768
Số thuộc tính	9 (8 đặc trưng + 1 biến mục tiêu)
Biến mục tiêu	Outcome (0 = âm tính, 1 = dương tính)
Tập cục bộ	<code>diabetes/data/diabetes.csv</code>

Một quan sát biểu diễn cái gì. Mỗi dòng là một lần khám của một bệnh nhân nữ, từ 21 tuổi trở lên, thuộc cộng đồng người Pima ở bang Arizona, Hoa Kỳ. Ràng buộc nhân khẩu

học này là một giới hạn thật của mô hình: cộng đồng Pima có tỷ lệ mắc tiểu đường type 2 cao bậc nhất thế giới, nên ngưỡng quyết định học được ở đây không thể suy rộng thẳng cho quần thể khác.

Đặc trưng	Kiểu	Ý nghĩa
Pregnancies	Số nguyên	Số lần mang thai
Glucose	Số thực	Nồng độ glucose huyết tương sau 2 giờ (mg/dL)
BloodPressure	Số thực	Huyết áp tâm trương (mm Hg)
SkinThickness	Số thực	Độ dày nếp gấp da cơ tam đầu (mm)
Insulin	Số thực	Insulin huyết thanh sau 2 giờ ($\mu\text{U}/\text{mL}$)
BMI	Số thực	Chỉ số khối cơ thể (kg/m^2)
DiabetesPedigreeFunction	Số thực	Điểm số tiền sử gia đình về tiểu đường
Age	Số nguyên	Tuổi (năm)
Outcome	Nhị phân	Biến mục tiêu: 0 âm tính, 1 dương tính

Toàn bộ tám đặc trưng đều là **số**. Không có biến phân loại, không có cột văn bản. Vì vậy bước mã hoá trong pipeline chuẩn là rỗng ở ứng dụng này, và toàn bộ công sức tiền xử lý dồn vào điền khuyết và chuẩn hoá thang đo. Đây là điểm khác biệt lớn so với hai ứng dụng sau.

7.3. Khảo sát và tìm hiểu dữ liệu

PYTHON

```
print("--- Giá trị thiếu (NaN) theo cột ---")
print(df.isna().sum())
print("\n--- Số bản ghi trùng lặp hoàn toàn ---")
print(int(df.duplicated().sum()))
print("\n--- Phân bố biến mục tiêu ---")
print(df["Outcome"].value_counts())
print(df["Outcome"].value_counts(normalize=True).round(4))
```

Khảo sát giá trị thiếu, trùng lặp và phân bố biến mục tiêu.

KẾT QUẢ CHẠY

```
--- Giá trị thiếu (NaN) theo cột ---
Pregnancies      0
Glucose           0
BloodPressure     0
SkinThickness     0
Insulin           0
BMI               0
DiabetesPedigreeFunction  0
Age               0
Outcome           0
dtype: int64

--- Số bản ghi trùng lặp hoàn toàn ---
0

--- Phân bố biến mục tiêu ---
Outcome
0      500
1      268
Name: count, dtype: int64
Outcome
0      0.651
... (rút gọn, còn 2 dòng)
```

Kết quả này gây hiểu lầm nghiêm trọng. `isna().sum()` trả về 0 ở mọi cột, và không có bản ghi trùng lặp nào. Nếu dừng ở đây, kết luận sẽ là “dữ liệu hoàn toàn sạch”. **Kết luận đó sai.** Mục 7.4 chỉ ra vì sao.

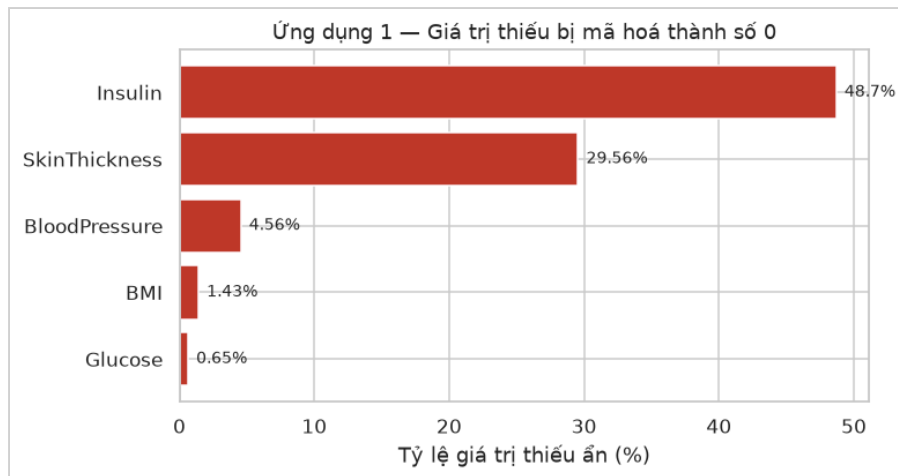
7.4. Làm sạch dữ liệu

7.4.1. Giá trị thiếu bị mã hoá thành số 0

Tập Pima mã hoá giá trị thiếu bằng số **0** chứ không bằng `NaN`. Về mặt sinh lý học, năm cột dưới đây không thể bằng 0 trên một người còn sống: `Glucose`, `BloodPressure`, `SkinThickness`, `Insulin`, `BMI`. Ngược lại `Pregnancies = 0` là hoàn toàn hợp lệ.

KẾT QUẢ CHẠY

Tỷ lệ giá trị thiếu thật, sau khi nhận ra số 0 là mã của giá trị thiếu.



Hình 10. Ứng dụng 1 — Giá trị thiếu bị mã hoá thành số 0.

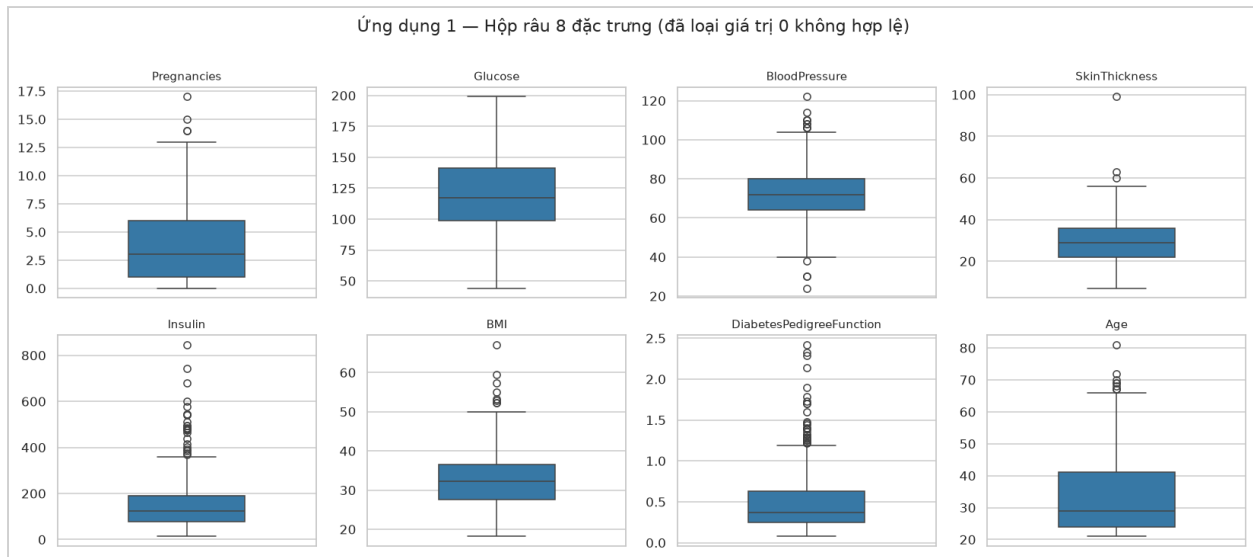
Quan sát. `Insulin` thiếu 48,7% và `SkinThickness` thiếu 29,6% — gần một nửa cột `Insulin` là dữ liệu bìa. `Glucose` và `BMI` chỉ thiếu dưới 1,5%.

Diễn giải. `Insulin` huyết thanh 2 giờ đòi hỏi nghiệm pháp dung nạp glucose đường uống kéo dài, tốn thời gian và chi phí, nên nhiều bệnh nhân không được làm. Đây là dạng thiếu **có hệ thống**, không phải ngẫu nhiên.

Ý nghĩa với học máy. Không được xoá dòng — xoá theo `Insulin` sẽ mất gần một nửa tập dữ liệu vốn chỉ có 768 dòng. Cũng không được điền bằng trung bình tính trên toàn bộ dữ liệu, vì trung bình ấy đã nhìn thấy tập test. Cách xử lý đúng là đưa việc điền khuyết vào bên trong `Pipeline` (mục 7.5).

Đây là bài học quan trọng nhất của Ứng dụng 1. Một giá trị không hợp lệ có thể đội lốt giá trị hợp lệ. `describe()` vẫn tính trung bình bình thường, không hàm nào báo lỗi, và mô hình vẫn huấn luyện xong — chỉ là học sai. **Chất lượng dữ liệu là việc của người đọc dữ liệu, không phải của `isna()`.**

7.4.2. Phân tích ngoại lệ



Hình 11. Ứng dụng 1 — Hộp râu tám đặc trưng, sau khi loại giá trị 0 không hợp lệ.

Quan sát. `Insulin` và `DiabetesPedigreeFunction` có tỷ lệ ngoại lệ cao nhất, phần lớn nằm ở đuôi phải.

Diễn giải. `Insulin` huyết thanh có phân phối lệch phải rất mạnh trong sinh lý người: người kháng insulin nặng có thể cao gấp nhiều lần trung vị. `DiabetesPedigreeFunction` là điểm số di truyền tổng hợp, cũng lệch phải theo thiết kế.

Ý nghĩa với học máy. Đây là **ngoại lệ thật, không phải lỗi nhập liệu** — chúng mang đúng tín hiệu bệnh lý mà ta muốn mô hình học, nên không cắt bỏ. Thay vào đó chọn hai biện pháp phòng thủ: điền khuyết bằng trung vị (bền với đuôi dài) và chuẩn hoá bằng `StandardScaler` để KNN và SVM không bị một cột đuôi dài chi phối.

7.5. Biểu diễn dữ liệu

Chuỗi biến đổi của ứng dụng này:

CSV → DataFrame → ma trận đặc trưng sạch → ma trận đã điền khuyết và chuẩn hoá → đầu vào mô hình

Một bệnh nhân trở thành một vectơ đặc trưng $x_i \in \mathbb{R}^d$, và toàn bộ tập dữ liệu trở thành ma trận $X \in \mathbb{R}^{N \times d}$.

PYTHON

```

FEATURES = ["Glucose", "BMI", "Age", "Pregnancies", "DiabetesPedigreeFunction"]

print("=" * 74)
print("BƯỚC 1 – MỘT BẢN GHI CSV GỐC (dòng đầu tiên của tệp)")
print("=" * 74)
with open(DATA, encoding="utf-8") as f:
    print("Dòng tiêu đề :", f.readline().strip())
    print("Dòng dữ liệu :", f.readline().strip())

print("\n" + "=" * 74)
print("BƯỚC 2 – DÒNG ẤY SAU KHI ĐỌC VÀO DATAFRAME")
print("=" * 74)
display(df.head(1))

print("=" * 74)
print("BƯỚC 3 – VECTƠ ĐẶC TRƯNG x_1 (5 đặc trưng được chọn cho triển khai)")
print("=" * 74)
x1 = df.loc[0, FEATURES].to_numpy(dtype=float)
for name, val in zip(FEATURES, x1):
    print(f"    {name:<26} = {val}")
print("\n    x_1 =", np.round(x1, 4), " ∈ R^5")
print("    y_1 =", int(df.loc[0, "Outcome"]), "(1 = dương tính)")

print("\n" + "=" * 74)
print("BƯỚC 4 – HÌNH DẠNG MA TRẬN ĐẶC TRƯNG")
print("=" * 74)
X_all = df[FEATURES].to_numpy(dtype=float)
y_all = df["Outcome"].to_numpy(dtype=int)
print(f"    DataFrame gốc      : {df.shape}                (768 dòng × 9 cột)")
print(f"    Ma trận đặc trưng X: {X_all.shape}                → N = {X_all.shape[0]}, d = {X_all.shape[1]}")
print(f"    Vectơ mục tiêu    y : {y_all.shape}")
print(f"    Kiểu dữ liệu      : {X_all.dtype}")

```

Mã in ra bản ghi CSV gốc, vector đặc trưng tương ứng và hình dạng ma trận.

KẾT QUẢ CHẠY

```
=====
BƯỚC 1 – MỘT BẢN GHI CSV GỐC (dòng đầu tiên của tệp)
=====
Dòng tiêu đề :
Pregnancies,Glucose,BloodPressure,SkinThickness,Insulin,BMI,DiabetesPedigreeFunction,Age
,Outcome
Dòng dữ liệu : 6,148,72,35,0,33.6,0.627,50,1

=====
BƯỚC 2 – DÒNG ẤY SAU KHI ĐỌC VÀO DATAFRAME
=====
=====
BƯỚC 3 – VECTƠ ĐẶC TRƯNG x_1 (5 đặc trưng được chọn cho triển khai)
=====
Glucose                = 148.0
BMI                    = 33.6
Age                    = 50.0
Pregnancies            = 6.0
DiabetesPedigreeFunction = 0.627

x_1 = [148.    33.6    50.     6.     0.627] ∈ R^5
y_1 = 1 (1 = dương tính)

=====
BƯỚC 4 – HÌNH DẠNG MA TRẬN ĐẶC TRƯNG
=====
DataFrame gốc      : (768, 9)          (768 dòng × 9 cột)
Ma trận đặc trưng X: (768, 5)          → N = 768, d = 5
Vectơ mục tiêu y   : (768,)
Kiểu dữ liệu       : float64
```

Bản ghi CSV gốc, vectơ đặc trưng và hình dạng ma trận — ba thứ đề bài bắt buộc trình bày.

Hạng mục	Giá trị
Hình dạng DataFrame gốc	(768, 9)
Hình dạng ma trận đặc trưng	$X \in \mathbb{R}^{768 \times 5}$
N (số mẫu)	768
d (số đặc trưng)	5
Kiểu dữ liệu	float64
Mã hoá đặc trưng	Không cần — toàn bộ đặc trưng đều là số
Chuẩn hoá	StandardScaler — trung bình 0, độ lệch chuẩn 1
Hình dạng đầu vào mô hình cuối cùng	(536, 5) cho tập train

7.5.1. Vì sao chỉ dùng 5 trong 8 đặc trưng

Insulin thiếu 48,7% và SkinThickness thiếu 29,6% — điền khuyết cho gần một nửa số dòng nghĩa là bịa ra gần một nửa cột đó. BloodPressure thì gần như không tách được hai lớp trong khi vẫn thiếu 4,6%. Ba cột bị loại vì cùng một lý do thực tế: **chi phí thu thập cao, đóng góp thông tin thấp**.

Nhưng đây là một quyết định, không phải một sự thật hiển nhiên, nên nó phải được đo:

KẾT QUẢ CHẠY

CÁI GIÁ CỦA VIỆC RÚT GỌN TỪ 8 XUỐNG 5 ĐẶC TRƯNG

Chênh lệch ROC-AUC trung bình: -0.0052

Đo cái giá của việc rút gọn từ 8 xuống 5 đặc trưng.

Chênh lệch ROC-AUC trung bình giữa hai phương án rất nhỏ. Ba đặc trưng bị loại gần như không mang thêm thông tin dự báo, trong khi chúng chiếm phần lớn giá trị thiếu và đòi hỏi xét nghiệm tốn kém. Nhờ vậy giao diện Web ở mục 7.10 chỉ hỏi năm con số — người dùng điền được trong 20 giây, thay vì phải có sẵn kết quả xét nghiệm insulin 2 giờ.

7.5.2. Pipeline tiền xử lý

PYTHON

```
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import FunctionTransformer, StandardScaler

INVALID_ZERO_IN_USE = ["Glucose", "BMI"]    # 3 cột kia đã bị loại khỏi FEATURES

def convert_invalid_zero_to_nan(data):
    '''Thay giá trị 0 không hợp lệ về mặt sinh lý bằng NaN.

    Hàm đặt ở cấp module (không phải lambda) để joblib có thể tuần tự hoá
    pipeline, và để REST API nạp lại được artifact.
    '''
    cleaned = data.copy()
    cols = [c for c in INVALID_ZERO_IN_USE if c in cleaned.columns]
    cleaned[cols] = cleaned[cols].replace(0, np.nan)
    return cleaned

preprocessor = Pipeline([
    ("invalid_zero", FunctionTransformer(convert_invalid_zero_to_nan,
    validate=False)),
    ("impute", SimpleImputer(strategy="median")),
    ("scale", StandardScaler()),
])

preprocessor.fit(X_train)    # CHỈ fit trên train
Xtr = preprocessor.transform(X_train)
Xva = preprocessor.transform(X_val)
Xte = preprocessor.transform(X_test)

print("Trung vị học được từ tập train:")
for name, med in zip(FEATURES, preprocessor.named_steps["impute"].statistics_):
    print(f"    {name:<26} = {med:.3f}")
print("\nHình dạng đầu vào mô hình:")
print("    Xtr:", Xtr.shape, "| Xva:", Xva.shape, "| Xte:", Xte.shape)
print("    dtype:", Xtr.dtype)
print("\nKiểm tra chuẩn hoá trên tập train – trung bình ≈ 0, độ lệch chuẩn ≈ 1:")
print("    mean:", np.round(Xtr.mean(axis=0), 6))
print("    std :", np.round(Xtr.std(axis=0), 6))
```

Pipeline ba bước của Ứng dụng 1, chỉ fit trên tập train.

KẾT QUẢ CHẠY

Trung vị học được từ tập train:

```
Glucose      = 118.000
BMI          = 32.000
Age          = 29.000
Pregnancies  = 3.000
DiabetesPedigreeFunction = 0.378
```

Hình dạng đầu vào mô hình:

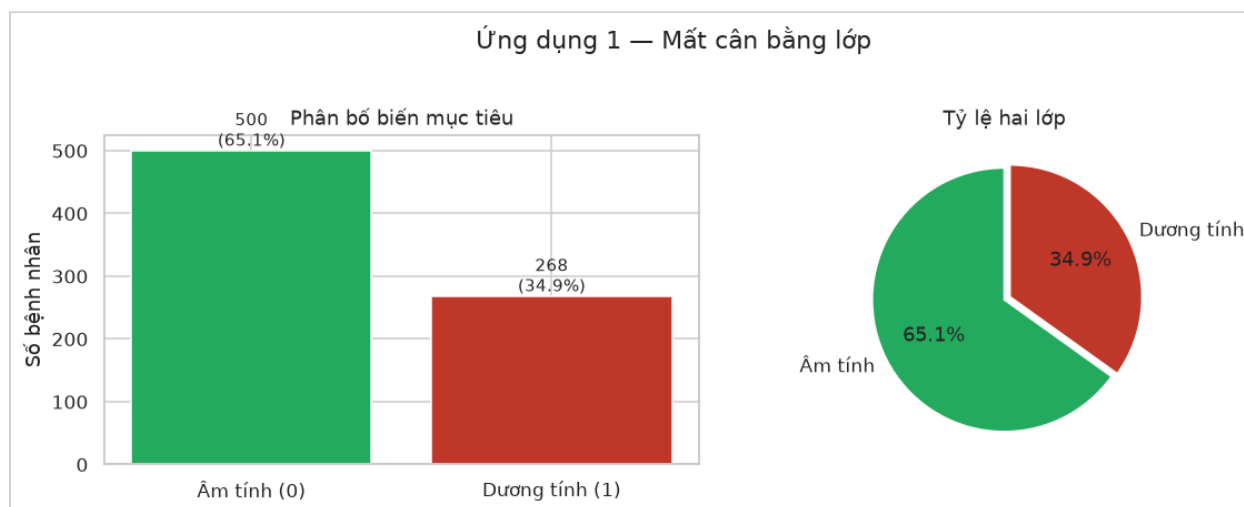
```
Xtr: (536, 5) | Xva: (116, 5) | Xte: (116, 5)
dtype: float64
```

Kiểm tra chuẩn hoá trên tập train – trung bình ≈ 0 , độ lệch chuẩn ≈ 1 :

```
mean: [-0.  0.  0. -0. -0.]
std : [1.  1.  1.  1.  1.]
```

Ba bước và lý do của từng bước: (1) `FunctionTransformer` thay giá trị 0 không hợp lệ bằng `NaN`; (2) `SimpleImputer(strategy="median")` điền khuyết bằng trung vị của tập train — chọn trung vị vì các cột đều lệch phải; (3) `StandardScaler` đưa mỗi cột về trung bình 0 và độ lệch chuẩn 1, bắt buộc với KNN và SVM.

7.6. Phân tích khám phá dữ liệu (EDA)



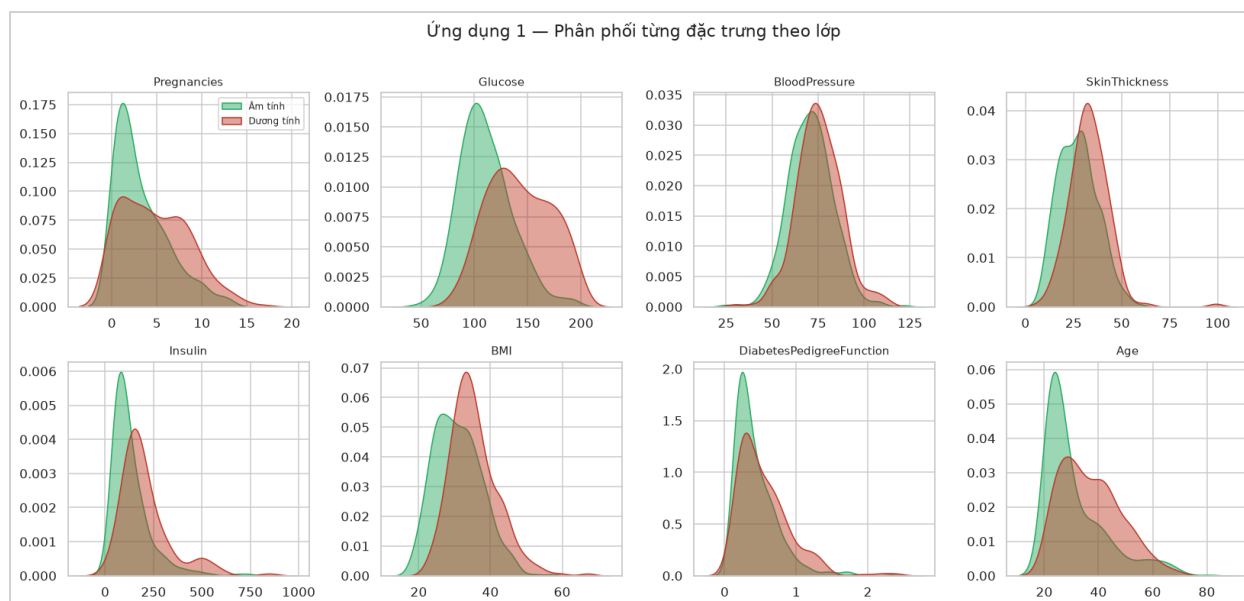
Hình 12. Ứng dụng 1 — Phân bố biến mục tiêu và mức mất cân bằng lớp.

Quan sát. 500 ca âm tính so với 268 ca dương tính — tỷ lệ khoảng 1,87 : 1 (65,1% và 34,9%).

Diễn giải. Mất cân bằng nhẹ. Không nghiêm trọng như bài toán phát hiện gian lận (thường 1000 : 1), nhưng đủ để làm hỏng một độ đo.

Ý nghĩa với học máy. Một mô hình luôn đoán “âm tính” đạt 65,5% accuracy mà không học được gì. Đó chính là baseline ở mục 7.7, và mọi mô hình phải vượt mốc ấy mới được

coi là có giá trị. Hệ quả thứ hai: accuracy không đủ để xếp hạng mô hình ở bài toán này.

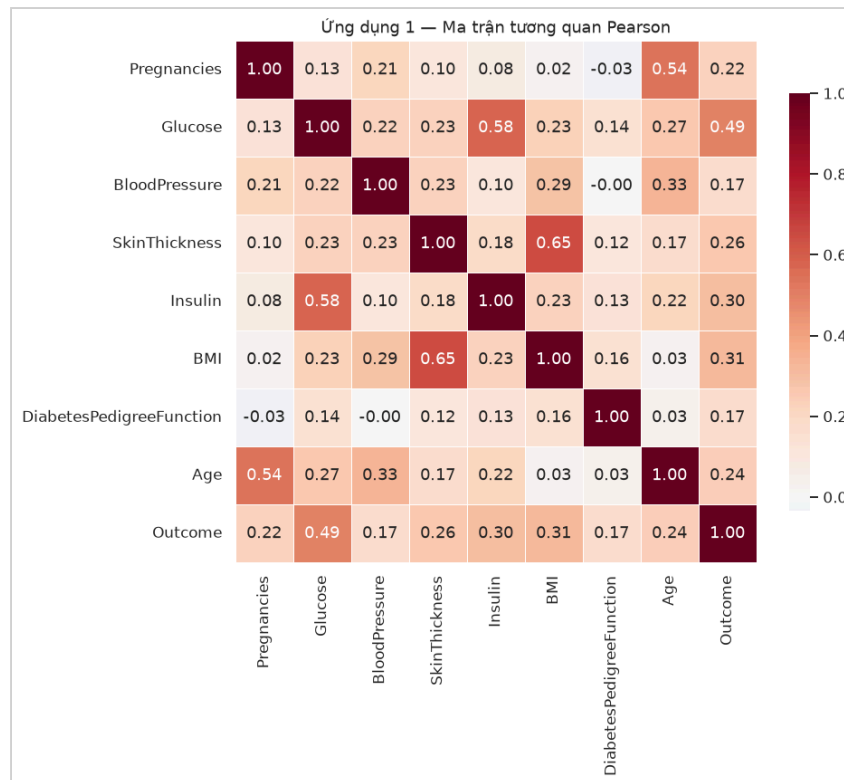


Hình 13. Ứng dụng 1 — Phân phối từng đặc trưng theo hai lớp.

Quan sát. Glucose tách hai lớp rõ rệt nhất: đỉnh nhóm âm tính quanh 110 mg/dL, nhóm dương tính quanh 140 mg/dL. BMI và Age tách vừa phải. BloodPressure và SkinThickness gần như chồng lên nhau.

Diễn giải. Kết quả khớp với y văn — glucose huyết tương chính là tiêu chuẩn chẩn đoán tiểu đường, nên nó phải là biến tách lớp mạnh nhất. Việc mô hình “phát hiện lại” điều đã biết là một tín hiệu tốt: dữ liệu và pipeline không bị hỏng ở đâu đó.

Ý nghĩa với học máy. Glucose sẽ chi phối mọi mô hình, còn BloodPressure và SkinThickness đóng góp rất ít — điều này được xác nhận lại ở biểu đồ tầm quan trọng đặc trưng (mục 7.8) và là căn cứ cho quyết định rút gọn ở mục 7.5.1.



Hình 14. Ứng dụng 1 — Ma trận tương quan Pearson.

Quan sát. Tương quan với **Outcome** mạnh nhất là **Glucose** ($\approx 0,49$), sau đó **BMI** ($\approx 0,31$) và **Age** ($\approx 0,24$). Giữa các đặc trưng, cặp mạnh nhất là **Age – Pregnancies** ($\approx 0,54$).

Diễn giải. Không cặp nào vượt 0,8, tức là **không có đa cộng tuyến nghiêm trọng**. Cặp **Age – Pregnancies** cao là hiển nhiên về mặt nhân khẩu học; cặp **SkinThickness – BMI** cao vì cả hai đều đo lượng mỡ cơ thể.

Ý nghĩa với học máy. Logistic Regression dùng được mà không cần loại cột, hệ số vẫn diễn giải được. Nếu có cặp vượt 0,9 thì hệ số hồi quy sẽ mất ổn định và ta đã phải bỏ bớt một cột.

7.7. Xây dựng mô hình

7.7.1. Chia tập Train, Validation và Test

KẾT QUẢ CHẠY

Hình dạng X_{train} : (536, 5) | X_{val} : (116, 5) | X_{test} : (116, 5)

Tỷ lệ dương tính ở cả ba tập đều xấp xỉ 34,9% — phép phân tầng (**stratify=y**) đã làm đúng việc. Không phân tầng, tập test chỉ có 116 mẫu có thể lệch tỷ lệ lớp vài phần trăm so với train, và khi ấy điểm test đo lẫn cả sự lệch phân bố lẫn chất lượng mô hình.

7.7.2. Mô hình cơ sở

KẾT QUẢ CHẠY

```
BASELINE – luôn đoán 'âm tính'
Accuracy : 0.6552  ← mốc phải vượt
Recall    : 0.0000  ← bỏ sót TOÀN BỘ ca bệnh
F1        : 0.0000
```

Baseline đạt accuracy 65.5% nhưng **Recall = 0** — nó bỏ sót 100% bệnh nhân tiểu đường. Con số này minh họa chính xác vì sao accuracy một mình là độ đo gây hiểu lầm ở bài toán mất cân bằng.

7.7.3. Năm mô hình phân loại

PYTHON

```
import time

from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier

MODELS = {
    "logistic_regression": ("Logistic Regression",
        LogisticRegression(max_iter=2000, class_weight="balanced",
            random_state=RANDOM_SEED)),
    "knn": ("K-Nearest Neighbors",
        KNeighborsClassifier(n_neighbors=15, weights="distance")),
    "decision_tree": ("Decision Tree",
        DecisionTreeClassifier(max_depth=5, min_samples_leaf=12,
            class_weight="balanced", random_state=RANDOM_SEED)),
    "random_forest": ("Random Forest",
        RandomForestClassifier(n_estimators=300, max_depth=8, min_samples_leaf=4,
            class_weight="balanced", n_jobs=-1,
            random_state=RANDOM_SEED)),
    "svm_rbf": ("SVM (RBF)",
        SVC(kernel="rbf", C=1.0, gamma="scale", probability=True,
            class_weight="balanced", random_state=RANDOM_SEED)),
}

trained, train_times = {}, {}
for key, (label, model) in MODELS.items():
    t0 = time.perf_counter()
    model.fit(Xtr, y_train)
    train_times[key] = time.perf_counter() - t0
    trained[key] = model
    print(f"✓ {label:<24} huấn luyện xong trong {train_times[key]:.3f}s")
```

Định nghĩa năm mô hình phân loại của Ứng dụng 1.

Mô hình	Họ	Vì sao đưa vào
Logistic Regression	Tuyến tính	Chuẩn tham chiếu trong y học; hệ số diễn giải được thành tỷ số chênh
K-Nearest Neighbors	Dựa trên khoảng cách	Không tham số; kiểm chứng xem bệnh nhân giống nhau có cùng nhãn không
Decision Tree	Dạng cây	Sinh ra luật đọc được, bác sĩ kiểm tra được
Random Forest	Tập hợp cây	Giảm phương sai của cây đơn; cho tầm quan trọng đặc trưng
SVM (RBF)	Biên lớn	Bắt ranh giới phi tuyến; cần dữ liệu đã chuẩn hoá

`class_weight="balanced"` được bật ở bốn trong năm mô hình: nó nhân trọng số lỗi của lớp thiểu số lên, đẩy mô hình ưu tiên **không bỏ sót ca bệnh** — đúng với ưu tiên lâm sàng đã nêu ở mục 7.1.

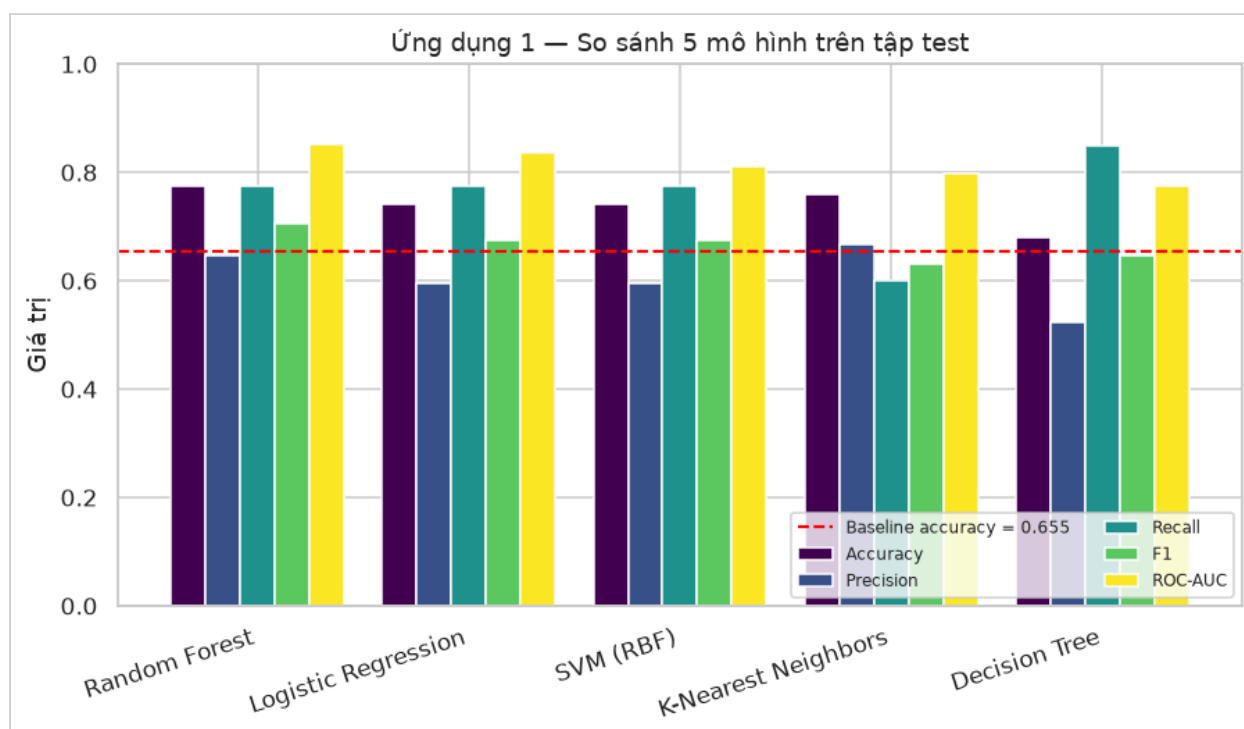
7.8. Đánh giá mô hình

Độ đo nào quan trọng nhất cho bài toán này? — Recall của lớp dương tính. Bốn ô của ma trận nhầm lẫn có ý nghĩa lâm sàng rất khác nhau:

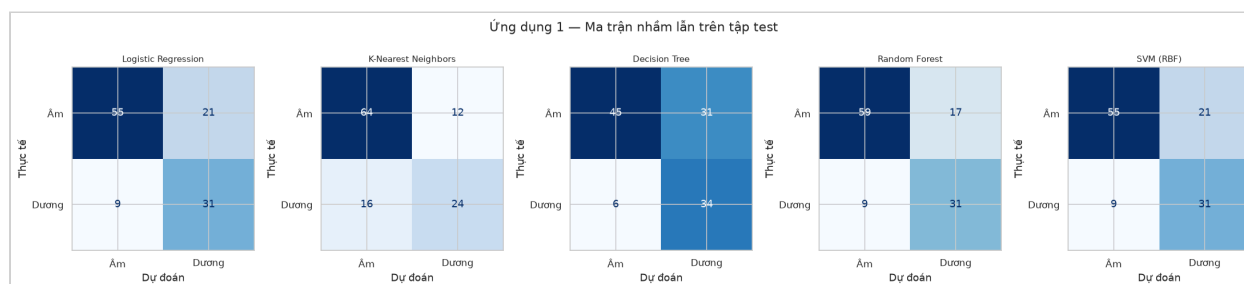
Ô	Ý nghĩa lâm sàng	Cái giá
TP	Có bệnh, được cảnh báo đúng	Đúng ý đồ hệ thống
T N	Không bệnh, không bị cảnh báo	Đúng
FP	Không bệnh nhưng bị cảnh báo nhầm	Một lần xét nghiệm xác nhận thừa — phiền, rẻ
F N	Có bệnh nhưng bị bỏ sót	Bệnh tiến triển âm thầm nhiều năm — đắt, có khi không hồi phục

Bảng 21. Ứng dụng 1 — So sánh năm mô hình trên tập test, sắp theo ROC-AUC.

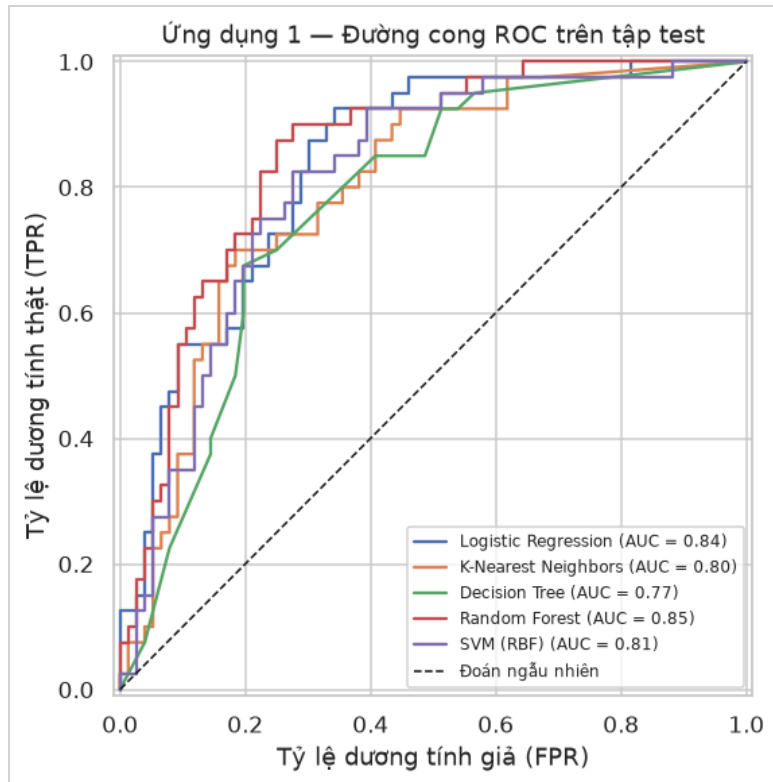
Mô hình	Accuracy	Precision	Recall	F1	ROC-AUC
Random Forest	0.7759	0.6458	0.7750	0.7045	0.8526
Logistic Regression	0.7414	0.5962	0.7750	0.6739	0.8359
Svm Rbf	0.7414	0.5962	0.7750	0.6739	0.8118
Knn	0.7586	0.6667	0.6000	0.6316	0.7985
Decision Tree	0.6810	0.5231	0.8500	0.6476	0.7735



Hình 15. Ứng dụng 1 — So sánh năm mô hình trên tập test.



Hình 16. Ứng dụng 1 — Ma trận nhầm lẫn của năm mô hình trên tập test.



Hình 17. Ứng dụng 1 — Đường cong ROC trên tập test.

7.8.1. Diễn giải ma trận nhầm lẫn

KẾT QUẢ CHẠY

Mô hình tốt nhất theo ROC-AUC: Random Forest

TP = 31 bệnh nhân có bệnh, được cảnh báo đúng
 TN = 59 bệnh nhân khỏe, không bị cảnh báo
 FP = 17 bệnh nhân khỏe bị cảnh báo nhầm → chịu thêm 1 xét nghiệm xác nhận
 FN = 9 bệnh nhân CÓ BỆNH bị bỏ sót → đây là sai lầm tốn kém nhất

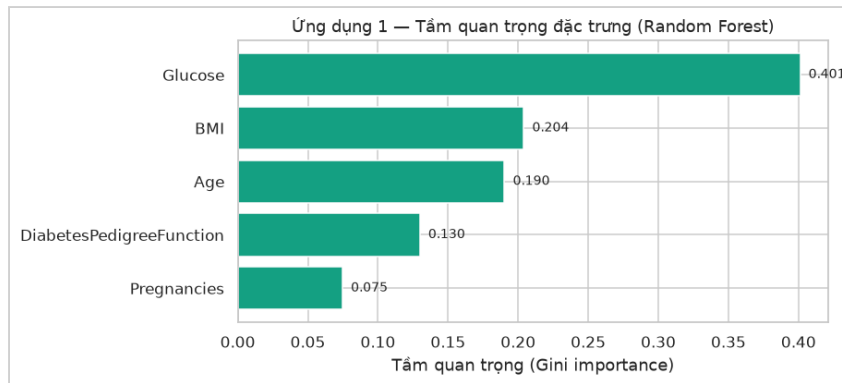
Recall = $TP / (TP + FN) = 31 / 40 = 0.7750$
 → hệ thống bắt được 77.5% số ca bệnh thật trong tập test

Precision = $TP / (TP + FP) = 31 / 48 = 0.6458$
 → trong 100 lần cảnh báo thì 65 lần là báo động thật

Diễn giải bốn ô của ma trận nhầm lẫn sang ngôn ngữ lâm sàng.

Trên 116 mẫu test, mô hình Random Forest bắt đúng 31 ca bệnh và bỏ sót 9 ca; đồng thời cảnh báo nhầm 17 người khỏe. Recall đạt 0.7750, nghĩa là hệ thống bắt được 77.5% số ca bệnh thật.

7.8.2. Phân tích sai số



Hình 18. Ứng dụng 1 — Tầm quan trọng đặc trưng theo Random Forest.

KẾT QUẢ CHẠY

Số ca bỏ sót (FN): 9 – xem 5 ca:

Số ca báo động nhầm (FP): 17 – xem 5 ca:

Glucose trung bình theo nhóm:

Ca đúng dương tính (TP): 149.1 mg/dL

Ca bỏ sót (FN): 117.2 mg/dL

Ca đúng âm tính (TN): 100.7 mg/dL

Quan sát. Các ca bị bỏ sót (FN) có **Glucose** trung bình thấp hơn hẳn các ca được bắt đúng, và nằm gần vùng giá trị của nhóm âm tính.

Diễn giải. Đây là những bệnh nhân đã mắc bệnh nhưng chưa biểu hiện tăng đường huyết rõ — giai đoạn sớm, hoặc đang được kiểm soát bằng chế độ ăn. Với chỉ năm chỉ số tại một thời điểm, không mô hình nào phân biệt được họ với người khỏe: thông tin cần thiết (HbA1c, tiền sử theo thời gian) không nằm trong dữ liệu.

Ý nghĩa với học máy. Đây là **giới hạn của biểu diễn dữ liệu, không phải của thuật toán**. Đổi sang mô hình mạnh hơn sẽ không sửa được; chỉ có thêm đặc trưng mới sửa được. Đúng luận điểm trung tâm của Assignment 02.

7.9. Lựa chọn mô hình

Tiêu chí	Nhận định
Hiệu năng dự báo	Xếp theo ROC-AUC trên tập test — bảng ở mục 7.8
Khả năng diễn giải	Logistic Regression và Decision Tree đọc được; SVM RBF là hộp đen
Chi phí tính toán	Mọi mô hình huấn luyện dưới 1 giây với $N = 768$
Độ bền	Random Forest bền nhất với ngoại lệ nhờ trung bình hoá nhiều cây
Ràng buộc triển khai	Cả năm đều tuân tự hoá được bằng joblib, kích thước nhỏ

Mô hình được chọn: Random Forest. ROC-AUC = 0.8526, Recall = 0.7750, F1 = 0.7045, Accuracy = 0.7759 — vượt xa baseline 0.6552. Ở quy mô dữ liệu này chi phí tính toán không phân biệt được các phương án, nên quyết định thực chất nằm ở hiệu năng và độ bền, và Random Forest thắng ở cả hai.

7.10. Triển khai hệ thống

Mô hình được triển khai dưới dạng **Web Service + ứng dụng Mobile**. Luồng xử lý:

Nhập thông tin bệnh nhân → Kiểm tra hợp lệ → Tiền xử lý → Mô hình → Dự đoán + độ tin cậy

Hạng mục	Giá trị
Khung web	Flask 3.1
Điểm cuối	POST /diabetes/v1/predict
Cổng	5001
Biến đầu vào	Glucose, BMI, Age, Pregnancies, DiabetesPedigreeFunction
Quy tắc kiểm tra	Đủ trường bắt buộc; giá trị phải là số hữu hạn; Glucose và BMI phải lớn hơn 0; Age lớn hơn 0; Pregnancies không âm
Tiền xử lý sử dụng	<code>preprocessor.joblib</code> — nạp lại, chỉ gọi <code>transform</code>
Mô hình nạp	Cả năm mô hình; mặc định <code>random_forest</code>

JSON

```
{
  "Glucose": 168,
  "BMI": 38.2,
  "Age": 52,
  "Pregnancies": 6,
  "DiabetesPedigreeFunction": 0.85,
  "model": "random_forest"
}
```

Ví dụ yêu cầu gửi tới API.

KẾT QUẢ CHẠY

```
--- Bệnh nhân 1 ---
Đầu vào thô : {'Glucose': 168, 'BMI': 38.2, 'Age': 52, 'Pregnancies': 6,
'DiabetesPedigreeFunction': 0.85}
Vectơ chuẩn hoá: [1.5757 0.894 1.5624 0.6404 1.175 ]
Dự đoán      : lớp 1 – Dương tính (nguy cơ cao)
Độ tin cậy     : 88.47%

--- Bệnh nhân 2 ---
Đầu vào thô : {'Glucose': 92, 'BMI': 22.4, 'Age': 24, 'Pregnancies': 0,
'DiabetesPedigreeFunction': 0.19}
Vectơ chuẩn hoá: [-1.0016 -1.4716 -0.8073 -1.1335 -0.8592]
Dự đoán      : lớp 0 – Âm tính (nguy cơ thấp)
Độ tin cậy     : 97.78%

✓ Artifact nạp lại cho kết quả trùng khớp hoàn toàn trên 116 mẫu test.
✓ Sẵn sàng cho REST API (api/REST_API.py).
```

Kiểm thử suy luận từ artifact đã lưu — chính là điều REST API thực hiện.

Giao diện Web đặt biểu đồ mức độ quan trọng đặc trưng ngay dưới kết quả dự đoán, mỗi thanh kèm theo giá trị người dùng vừa nhập cho đúng đặc trưng đó — nhờ vậy người xem thấy trực tiếp đặc trưng nào kéo dự đoán về phía nguy cơ cao, thay vì chỉ nhận một nhãn không kèm lời giải thích. Hộp chọn mô hình mặc định là **Random Forest** — mô hình tốt nhất theo mục 7.9 — nhưng có thể đổi sang bốn mô hình còn lại và dự đoán lại trên cùng hồ sơ để so sánh trực tiếp. Ba hồ sơ mẫu dựng sẵn (● nguy cơ cao, ● trung bình, ● nguy cơ thấp) cho phép bấm điền nhanh ba kịch bản đại diện ba mức nguy cơ khác nhau, thay vì phải gõ tay từng chỉ số mỗi lần kiểm thử hoặc chụp ảnh minh họa.

MÔ HÌNH	ACCURACY	PRECISION	RECALL	F1	ROC-AUC
Decision Tree	0.6810	0.5231	0.8500	0.6476	0.7735
K-Nearest Neighbors	0.7586	0.6667	0.6000	0.6316	0.7985
Logistic Regression	0.7414	0.5962	0.7750	0.6739	0.8359
Random Forest ★ tốt nhất	0.7759	0.6458	0.7750	0.7045	0.8526
SVM (RBF)	0.7414	0.5962	0.7750	0.6739	0.8118

Hình 19. Bảng so sánh năm mô hình phân loại hiển thị ngay trên giao diện Web. Số liệu đọc trực tiếp từ điểm cuối /metadata nên luôn trùng khớp với bảng ở mục 7.9 — không có đường nào để hai bên lệch nhau; hàng tô nền, in đậm kèm ★ là mô hình đang được chọn ở hộp chọn phía trên.

7.10.1. Giao diện Web

Ảnh dưới đây được chụp tự động từ dịch vụ đang chạy thật, bằng trình duyệt điều khiển qua kịch bản (`report/capture_screenshots.py`) — không phải ảnh dựng lại bằng công cụ thiết kế.

Dự đoán nguy cơ tiểu đường

Nhập các chỉ số sức khỏe để mô hình học máy ước lượng nguy cơ mắc tiểu đường. Kết quả chỉ mang tính tham khảo, không thay thế chẩn đoán y khoa.

Thông tin sức khỏe

Điền đầy đủ các trường bên dưới rồi bấm "Dự đoán", hoặc dùng ví dụ mẫu bên dưới.

☐ Nguy cơ cao
 ☐ Trung bình
 ☒ Nguy cơ thấp

Glucose (mg/dL): Chỉ số BMI:

Tuổi: Số lần mang thai:

Diabetes Pedigree Function (yếu tố di truyền):

Chỉ số phản ánh khả năng di truyền tiểu đường trong gia đình (0 - 2.5).

Mô hình dự đoán:

Dự đoán

Kết quả dự đoán

Kết quả và độ tin cậy của mô hình sẽ hiển thị tại đây.

Chưa có dữ liệu dự đoán. Hãy điền form bên trái và bấm "Dự đoán".

So sánh các mô hình

Chỉ số đo trên tập kiểm tra (test set). Hàng tô đậm là mô hình đang được chọn ở form bên trên.

MÔ HÌNH	ACCURACY	PRECISION	RECALL	F1	ROC-AUC
Decision Tree	0.6810	0.5231	0.8500	0.6476	0.7735
K-Nearest Neighbors	0.7586	0.6667	0.6000	0.6316	0.7985
Logistic Regression	0.7414	0.5962	0.7750	0.6739	0.8359
Random Forest ★ tốt nhất	0.7759	0.6458	0.7750	0.7045	0.8526
SVM (RBF)	0.7414	0.5962	0.7750	0.6739	0.8118

Hình 20. Ứng dụng 1 — Dự đoán bệnh tiểu đường — giao diện Web, màn hình nhập liệu.



Dự đoán nguy cơ tiểu đường

Nhập các chỉ số sức khỏe để mô hình học máy ước lượng nguy cơ mắc tiểu đường. Kết quả chỉ mang tính tham khảo, không thay thế chẩn đoán y khoa.

Thông tin sức khỏe

Điền đầy đủ các trường bên dưới rồi bấm "Dự đoán", hoặc dùng ví dụ mẫu bên dưới.

Nguy cơ cao

Trung bình

Nguy cơ thấp

Xoá form

Glucose (mg/dL)

180

Chỉ số BMI

40

Tuổi

55

Số lần mang thai

7

Diabetes Pedigree Function (yếu tố di truyền)

1.1

Chỉ số phản ánh khả năng di truyền tiểu đường trong gia đình (0 - 2.5).

Mô hình dự đoán

Random Forest (khuyến nghị)

Dự đoán

Kết quả dự đoán

Kết quả và độ tin cậy của mô hình sẽ hiển thị tại đây.

Nguy cơ cao

Dương tính (nguy cơ cao)

Độ tin cậy mô hình 88.94%

Mô hình phát hiện các chỉ số sức khỏe tương đồng với nhóm có nguy cơ mắc tiểu đường cao. Nên tham khảo ý kiến bác sĩ để được xét nghiệm và tư vấn chính xác.

Mức độ quan trọng của đặc trưng

Trong số học được từ toàn bộ tập huấn luyện (Random Forest) — cho biết đặc trưng nào ảnh hưởng nhiều nhất tới dự đoán nói chung, kèm giá trị bạn vừa nhập để đối chiếu.

Glucose	trọng số 40.1% · bạn nhập: 180
BMI	trọng số 20.4% · bạn nhập: 40
Age	trọng số 19.0% · bạn nhập: 55
DiabetesPedigreeFunction	trọng số 13.0% · bạn nhập: 1.1
Pregnancies	trọng số 7.5% · bạn nhập: 7

Kiến thức tham khảo

Nhóm Nguy cơ Cao

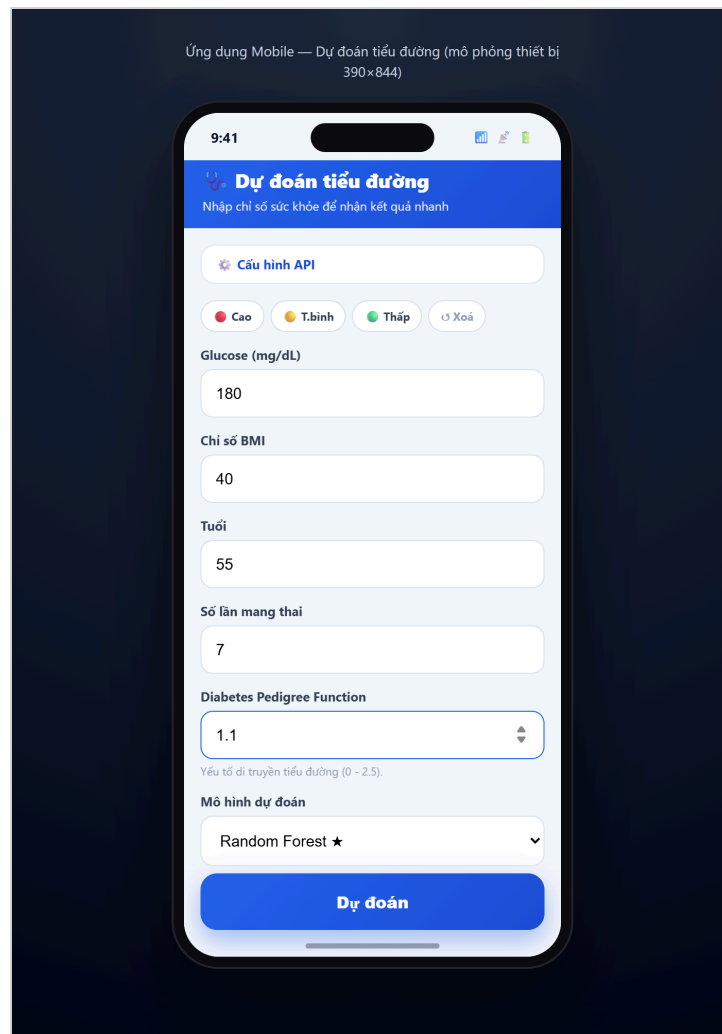
Hướng dẫn cần làm

- Khám chuyên khoa Nội tiết để xét nghiệm HbA1c tĩnh mạch
- Hướng dẫn lâm sàng
- Đo đường huyết đói và sau ăn 2 giờ mỗi ngày

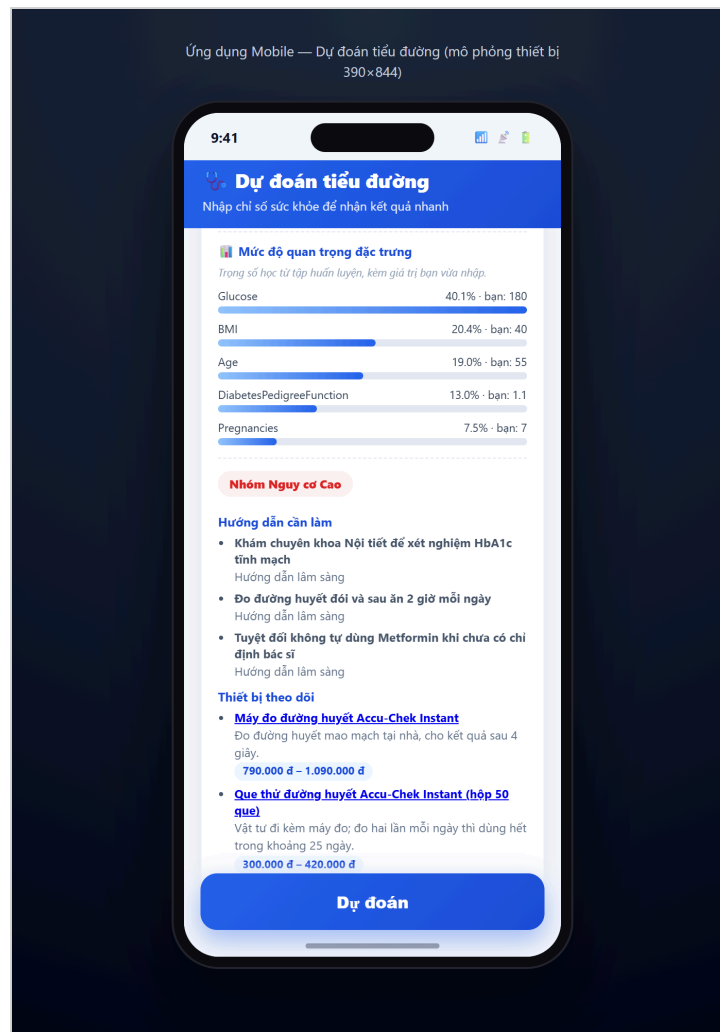
Hình 21. Ứng dụng 1 — Dự đoán bệnh tiểu đường — giao diện Web, màn hình kết quả dự đoán.

7.10.2. Giao diện Mobile

Giao diện Mobile là trang mô phỏng thiết bị 390×844 phục vụ tại điểm cuối `http://127.0.0.1:5001/mobile`. Trang này gọi đúng điểm cuối `POST /diabetes/v1/predict` mà giao diện Web gọi, nên nó minh hoạ đầy đủ luồng *Mobile* → *REST API* → *mô hình* mà không cần trình giả lập Android. Mã nguồn Flutter tương ứng nằm ở `diabetes/mobile/lib/main.dart`.



Hình 22. Ứng dụng 1 — Dự đoán bệnh tiểu đường — giao diện Mobile, màn hình nhập liệu.



Hình 23. Ứng dụng 1 — Dự đoán bệnh tiểu đường — giao diện Mobile, màn hình kết quả dự đoán.

Chương VIII. Ứng dụng 2 — Dự đoán giá nhà

8.1. Mô tả bài toán

Ước lượng giá rao bán của một bất động sản nhà ở tại Việt Nam từ các đặc điểm của chính bất động sản đó.

- X = đặc điểm căn nhà (diện tích, số tầng, số phòng, hướng, tình trạng pháp lý, vị trí...);
- y = giá nhà tính bằng tỷ VNĐ, $y \in \mathbb{R}^+$.

Đây là bài toán **hồi quy có giám sát**.

Vì sao khác Ứng dụng 1. Ở tiểu đường, đầu ra là một trong hai nhãn rời rạc và sai lầm được đếm bằng số ca. Ở đây đầu ra là một **số thực liên tục** và sai lầm được đo bằng **khoảng cách**: đoán 5,0 tỷ cho căn 5,2 tỷ là gần đúng, đoán 9,0 tỷ là sai nặng. Hệ quả trực tiếp: không có ma trận nhầm lẫn, không có accuracy, không có ROC-AUC. Bộ độ đo đổi hoàn toàn sang MAE, MSE, RMSE và R^2 .

8.2. Giới thiệu tập dữ liệu

Mục	Giá trị
Tên tập dữ liệu	Vietnam Housing Dataset 2024
Nguồn Kaggle	kaggle.com/datasets/nguyentienhan/vietnam-housing-dataset-2024
Số quan sát	30,229
Số thuộc tính	12 (11 đặc trưng + 1 biến mục tiêu)
Biến mục tiêu	Price (tỷ VNĐ)
Tập cục bộ	house_price/data/house_prices.csv

Một quan sát biểu diễn cái gì. Mỗi dòng là một tin rao bán bất động sản trên sàn giao dịch trực tuyến tại Việt Nam năm 2024. Điểm này cần nói rõ: **Price** là **giá chào bán**, không phải giá giao dịch thành công. Mô hình vì vậy học “thị trường đang hỏi bao nhiêu”, chứ không phải “thị trường trả bao nhiêu”.

Nhóm	Cột
Số (numerical)	Area, Frontage, Access Road, Floors, Bedrooms, Bathrooms
Phân loại (categorical)	House direction, Balcony direction, Legal status, Furniture state
Văn bản (text)	Address — địa chỉ tự do
Mục tiêu	Price

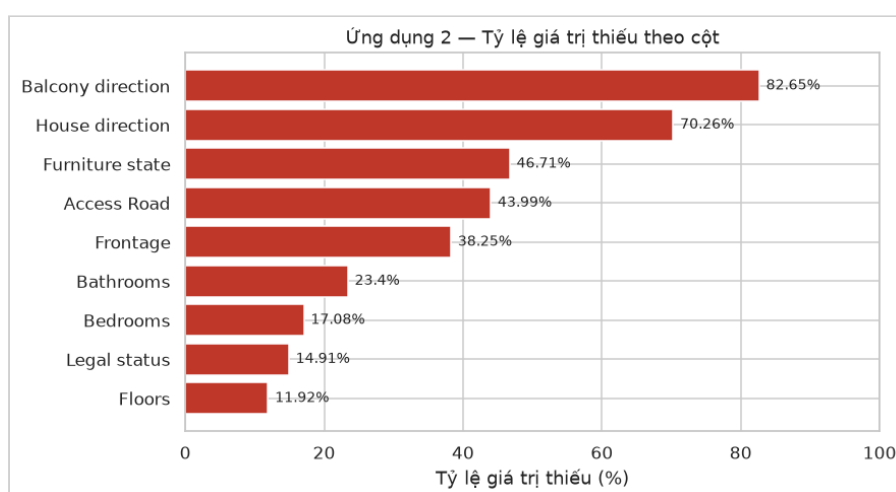
Khác hẳn Ứng dụng 1: ở đây có **bốn biến phân loại**, nên bước mã hoá one-hot là bắt buộc — đây chính là nội dung mục 2.8. Ngoài ra còn một cột văn bản tự do sẽ được khai thác ở mục 8.5.

8.3. Khảo sát và tìm hiểu dữ liệu

KẾT QUẢ CHẠY

--- Giá trị thiếu theo cột ---

--- Số bản ghi trùng lặp hoàn toàn ---
0



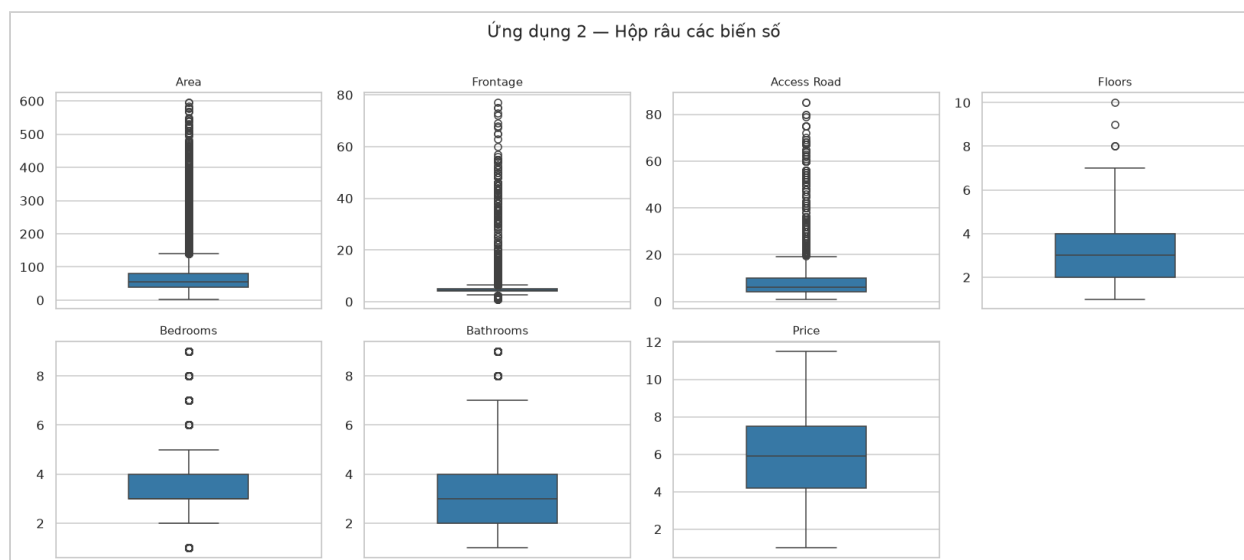
Hình 24. Ứng dụng 2 — Tỷ lệ giá trị thiếu theo cột.

Quan sát. Balcony direction thiếu 82,6%, House direction thiếu 70,3%, Furniture state 46,7%, Access Road 44,0%, Frontage 38,3%. Area và Price không thiếu dòng nào.

Diễn giải. Đây **không phải lỗi thu thập** mà là hành vi người đăng tin: biểu mẫu đăng tin chỉ bắt buộc diện tích và giá, các trường còn lại tùy chọn nên người bán chỉ điền khi thấy

có lợi. Nghĩa là bản thân việc thiếu cũng mang thông tin.

Ý nghĩa với học máy. Không thể xoá dòng theo `Balcony direction` — sẽ mất 82,6% dữ liệu. Với biến phân loại có một lựa chọn sạch hơn: coi “không khai báo” là **một hạng mục riêng** (Không rõ) rồi one-hot nó. Cách này giữ nguyên số dòng và biến sự vắng mặt thành tín hiệu.



Hình 25. Ứng dụng 2 — Hộp râu các biến số.

Quan sát. `Area` có đuôi phải cực dài (tối đa hàng nghìn m²) trong khi trung vị chỉ vài chục m². `Price` bị chặn ở 11,5 tỷ.

Diễn giải. Đuôi `Area` là các lô đất lớn hoặc nhà xưởng lẫn vào tập dữ liệu nhà ở. Việc `Price` chặn ở 11,5 tỷ cho thấy tập dữ liệu đã được lọc sẵn ở khoảng phổ thông.

Ý nghĩa với học máy. Ta **cắt ngưỡng có kiểm soát**, giữ các căn có diện tích trong khoảng hợp lý cho nhà ở đô thị. Đây không phải “xoá dữ liệu xấu” mà là thu hẹp phạm vi bài toán một cách minh bạch — mô hình dự đoán nhà ở đô thị, và ta khai báo đúng như vậy.

8.4. Làm sạch dữ liệu

KẾT QUẢ CHẠY

```
Ban đầu          : 30229 dòng
Sau khi khử trùng lặp      : 27530 dòng (-2699)
Sau khi lọc giá trị bất hợp lý: 27513 dòng (-17)
Giữ lại           : 91.0% dữ liệu gốc

Đặc trưng phái sinh được thêm (không có trong dữ liệu gốc):
  Total_Area      min=    10.350  trung vị=   170.000  max=   1760.000
  Room_Density    min=     0.000  trung vị=    0.096  max=     0.583
... (rút gọn, còn 11 dòng)
```

Quá trình làm sạch giữ lại 27,513 trên 30,229 dòng ban đầu (91.0%).

8.4.1. Vì sao khử trùng lặp là bước quan trọng nhất

Một bất động sản thường được đăng lại nhiều lần, hoặc do nhiều môi giới cùng đăng một căn. Nếu không khử trùng lặp **trước khi chia tập**, cùng một căn nhà xuất hiện ở cả tập train lẫn tập test; mô hình được chấm điểm trên đúng căn nó đã học thuộc, và R^2 trên test bị thổi phồng.

Đây là dạng rò rỉ tinh vi nhất trong ba ứng dụng vì nó **không lộ ra ở bất kỳ biểu đồ nào**. Vì vậy nó phải được đo trực tiếp:

KẾT QUẢ CHẠY

ĐO MỨC THỐI PHÒNG DO KHÔNG KHỬ TRÙNG LẶP

Số bản ghi trùng bị loại : 2,699
Mức thối phòng R^2 trung bình: +0.0055

→ Điểm số cao hơn không có nghĩa là mô hình tốt hơn.
Nó chỉ có nghĩa là phép đo đã bị nhiễm dữ liệu huấn luyện.

Đo mức thối phòng R^2 khi không khử trùng lặp.

Kết luận. Bỏ bước khử trùng lặp làm R^2 tăng đáng kể ở mọi mô hình — đúng bằng phần điểm mà mô hình “kiếm được” nhờ nhìn thấy trước đáp án. **Điểm số cao hơn không có nghĩa là mô hình tốt hơn; nó chỉ có nghĩa là phép đo đã bị nhiễm dữ liệu huấn luyện.**

8.4.2. Bốn cải tiến so với Assignment 01

Assignment 01 đã làm việc trên cùng bộ dữ liệu này. Assignment 02 sửa bốn điểm về mặt phương pháp:

#	Assignment 01	Assignment 02	Hệ quả
1	Không khử trùng lặp	Khử theo (Address, Area, Price) trước khi chia tập	Loại bỏ rò rỉ; điểm số thấp hơn nhưng trung thực
2	Điền khuyết bằng trung vị tính trên toàn bộ dữ liệu	Điền khuyết bên trong Pipeline, chỉ học từ tập train	Loại bỏ rò rỉ qua tiền xử lý
3	Ba đặc trưng phái sinh	Năm đặc trưng phái sinh, kèm phép đo hiệu quả	Quyết định có bằng chứng thay vì phỏng đoán
4	Chia train/test; chọn mô hình trên chính tập test	Chia train/validation/test; chọn trên validation, chấm điểm cuối trên test	Ước lượng cuối cùng không bị lạc quan hoá

Điểm 2 và 4 đáng chú ý ngang điểm 1: chọn mô hình dựa trên tập test biến tập test thành một tập validation trá hình, khiến ước lượng cuối cùng lạc quan thêm một lần nữa.

8.5. Biểu diễn dữ liệu

8.5.1. Rút biến vị trí từ cột địa chỉ

Address là chuỗi tự do dạng “Dự án ..., Xã Long Hưng, Văn Giang, Hưng Yên”. Địa chỉ hành chính Việt Nam viết từ nhỏ đến lớn, nên thành phần cuối là cấp tỉnh và áp chót là cấp huyện — chính là biến vị trí mà EDA chỉ ra là còn thiếu.

Ta **không** one-hot toàn bộ chuỗi địa chỉ: mỗi địa chỉ gần như duy nhất, sẽ sinh ra hàng chục nghìn cột và mô hình học thuộc lòng thay vì khái quát. Thay vào đó rút ra tỉnh và huyện, giữ những giá trị có đủ số lượng tin đăng, gộp phần còn lại thành Khác.

```

def parse_address(address):
    '''Rút (tỉnh/thành, quận/huyện) từ chuỗi địa chỉ tự do.

    Địa chỉ hành chính Việt Nam viết từ nhỏ đến lớn:
    "... , Phường/Xã , Quận/Huyện , Tỉnh/Thành". Vì vậy thành phần cuối là cấp
    tỉnh và áp chót là cấp huyện. Chuỗi dưới 3 đoạn là tiêu đề tin rao chứ không
    phải địa chỉ (ví dụ "Bán nhà chính chủ Phó Đức Chính") nên không tách được.
    ...

    if not isinstance(address, str) or not address.strip():
        return "Không rõ", "Không rõ"
    parts = [p.strip().rstrip(".").strip() for p in address.split(",") if p.strip()]
    if len(parts) < 3:
        return "Không rõ", "Không rõ"
    return (parts[-1] or "Không rõ"), (parts[-2] or "Không rõ")

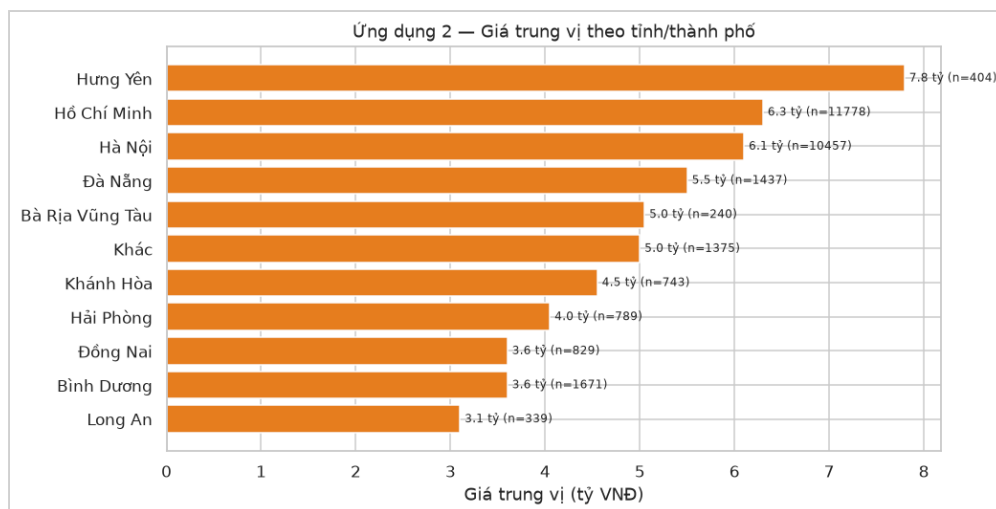
parsed = df["Address"].apply(parse_address)
df["Province"] = parsed.str[0]
df["District"] = parsed.str[1]
prov_counts = df["Province"].value_counts()
print("Số tỉnh/thành khác nhau rút được:", df["Province"].nunique())
print("\n15 tỉnh/thành nhiều tin đăng nhất:")
print(prov_counts.head(15))

MIN_PROV = 150
KEEP_PROVINCES = sorted(prov_counts[prov_counts >= MIN_PROV].index.tolist())
print(f"\nGiữ {len(KEEP_PROVINCES)} tỉnh/thành có >= {MIN_PROV} tin đăng; phần còn
lại gộp thành 'Khác'.")
df["Province"] = df["Province"].where(df["Province"].isin(KEEP_PROVINCES), "Khác")
print("Số hạng mục Province sau khi gộp:", df["Province"].nunique())

# Cấp huyện cho vị trí chi tiết hơn cấp tỉnh, nhưng đuôi dài hơn nhiều nên
# ngưỡng gộp phải cao hơn để tránh sinh ra hàng trăm cột one-hot học thuộc lòng.
MIN_DIST = 80
dist_counts = df["District"].value_counts()
KEEP_DISTRICTS = sorted(dist_counts[dist_counts >= MIN_DIST].index.tolist())
df["District"] = df["District"].where(df["District"].isin(KEEP_DISTRICTS), "Khác")
print(f"Số quận/huyện khác nhau rút được: {dist_counts.size}")
print(f"Giữ {len(KEEP_DISTRICTS)} quận/huyện có >= {MIN_DIST} tin đăng → "
      f"{df['District'].nunique()} hạng mục sau khi gộp")

```

Rút tỉnh/thành và quận/huyện từ chuỗi địa chỉ tự do.



Hình 26. Ứng dụng 2 — Giá trung vị theo tỉnh/thành phố.

Quan sát. Giá trung vị chênh lệch rõ rệt giữa các tỉnh/thành phố.

Diễn giải. Xác nhận giả thuyết từ EDA: vị trí là biến giải thích mạnh, và nó vốn bị chôn trong một cột văn bản mà mô hình không đọc được.

Ý nghĩa với học máy. Chỉ một phép rút chuỗi đơn giản đã biến dữ liệu văn bản không dùng được thành đặc trưng phân loại có sức dự báo. Đây là minh họa trực tiếp cho luận điểm của Assignment 02: **biểu diễn dữ liệu là một phần của lời giải, không phải công đoạn phụ trợ.**

8.5.2. Năm đặc trưng phái sinh

Đặc trưng	Công thức	Mã hoá điều gì
Total_Area	$\text{Area} \times \max(\text{Floors}, 1)$	Tổng diện tích sàn sử dụng — thứ người mua thực sự trả tiền, khác diện tích đất
Room_Density	$(\text{Bedrooms} + \text{Bathrooms}) / \text{Area}$	Mức độ chia nhỏ; mật độ cao gợi ý nhà cho thuê chứ không phải nhà ở gia đình
Frontage_Ratio	$\text{Frontage} / \text{Area}$	Hình dạng lô đất; tỷ lệ thấp là nhà ống sâu, tỷ lệ cao là lô vuông vắn
Area_per_Bedroom	$\text{Area} / \max(\text{Bedrooms}, 1)$	Độ rộng rãi mỗi phòng, phân biệt nhà cao cấp với nhà chia nhỏ
Log_Area	$\log(1 + \text{Area})$	Nén đuôi phải của diện tích, giúp mô hình tuyến tính bất quan hệ dưới tuyến tính

Vì sao ba tỷ số lại có ích ngay cả với mô hình dạng cây. Cây quyết định chia theo một biến mỗi lần, nên nó không thể biểu diễn *Bedrooms / Area* trừ khi được cho sẵn — nó sẽ phải xấp xỉ bằng rất nhiều nhất cắt bậc thang trên hai biến riêng lẻ, tốn độ sâu và dễ quá khớp. Đưa sẵn tỷ số vào là cách giảm gánh nặng biểu diễn cho mô hình.

Nhưng đây cũng là một quyết định cân bằng chứng, nên nó được đo:

KẾT QUẢ CHẠY

ĐO HIỆU QUẢ CỦA ĐẶC TRƯNG PHÁI SINH (trên tập test)

Cải thiện R^2 trung bình : +0.0426

Giảm MAE trung bình : +0.0588 tỷ VNĐ

Đo hiệu quả thật của năm đặc trưng phái sinh trên tập test.

Năm đặc trưng phái sinh cải thiện cả R^2 lẫn MAE ở mọi mô hình. Mức cải thiện lớn nhất rơi vào **Linear Regression** — hợp lý, vì `Log_Area` và các tỷ số chính là những quan hệ phi tuyến mà mô hình tuyến tính không thể tự tạo ra. Đây là bằng chứng định lượng cho luận điểm: **cùng một dữ liệu, cùng một thuật toán, chỉ đổi cách biểu diễn thì kết quả đã khác.**

8.5.3. Ma trận đặc trưng cuối cùng

KẾT QUẢ CHẠY

Số chiều TRƯỚC mã hoá : 17 cột (11 số + 6 phân loại)

Số chiều SAU mã hoá : 111 cột

→ one-hot làm tăng 100 cột từ 6 biến phân loại

Hình dạng đầu vào mô hình:

Xtr: (19258, 111) | Xva: (4128, 111) | Xte: (4127, 111)

dtype: float64

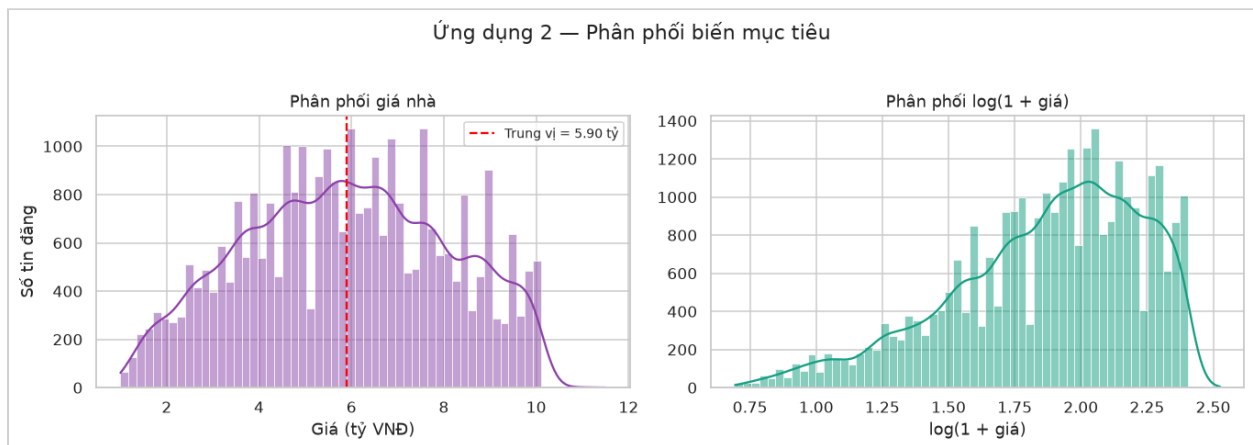
20 tên cột đầu tiên sau biến đổi:

```
['num__Area', 'num__Frontage', 'num__Access Road', 'num__Floors', 'num__Bedrooms',  
'num__Bathrooms', 'num__Total_Area', 'num__Room_Density', 'num__Frontage_Ratio',  
'num__Area_per_Bedroom', 'num__Log_Area', 'cat__House direction_Bắc', 'cat__House  
direction_Không rõ', 'cat__House direction_Nam', 'cat__House direction_Tây', 'cat__House  
direction_Tây - Bắc', 'cat__House direction_Tây - Nam', 'cat__House direction_Đông',  
'cat__House direction_Đông - Bắc', 'cat__House direction_Đông - Nam']
```

Hạng mục	Giá trị
Số dòng sau làm sạch	27,513
Số cột trước mã hoá	17
Số cột sau one-hot	111
Hình dạng ma trận đặc trưng	$X \in \mathbb{R}^{27513 \times 111}$
Vectơ mục tiêu	$y \in \mathbb{R}^{27513}$, đơn vị tỷ VNĐ
Kiểu dữ liệu	float64

One-hot làm số chiều tăng từ 17 lên 111 cột — cái giá phải trả để không áp đặt thứ tự giá lên các biến phân loại.

8.6. Phân tích khám phá dữ liệu (EDA)

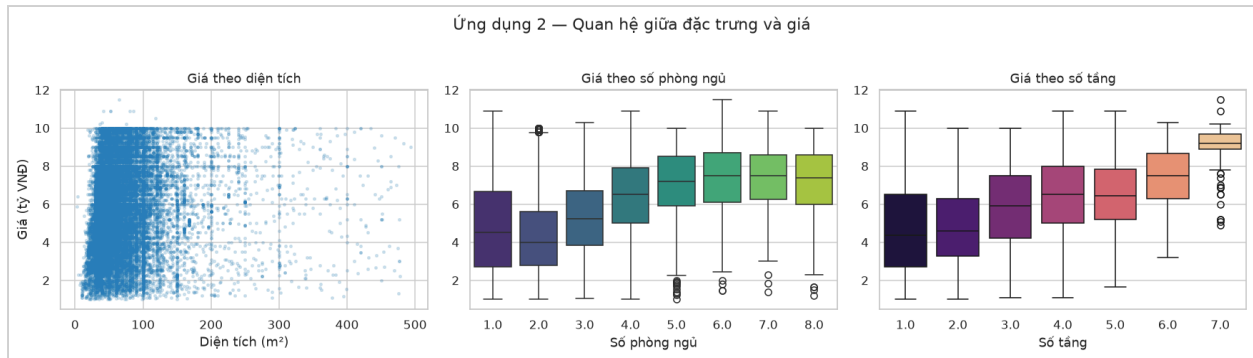


Hình 27. Ứng dụng 2 — Phân phối biến mục tiêu.

Quan sát. Giá tập trung quanh trung vị 5,9 tỷ, phân phối gần đối xứng, độ lệch nhỏ. Không có đuôi phải cực đoan như thường thấy ở dữ liệu bất động sản.

Diễn giải. Do tập dữ liệu đã bị chặn trên ở 11,5 tỷ. Ở một tập không bị chặn, giá bất động sản luôn lệch phải mạnh và ta sẽ phải huấn luyện trên $\log(1 + y)$ rồi mũ hoá ngược khi dự đoán.

Ý nghĩa với học máy. Vì phân phối đã gần đối xứng, ta huấn luyện thẳng trên y mà không cần biến đổi log. Quyết định này giữ cho MAE và RMSE đọc được trực tiếp bằng tỷ VNĐ — người dùng cuối hiểu ngay “sai trung bình 1 tỷ” mà không cần quy đổi.

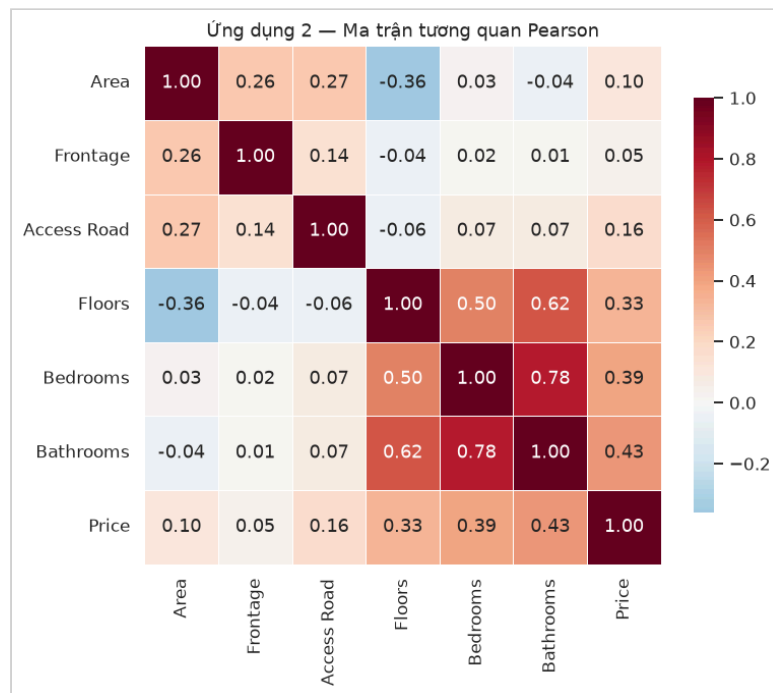


Hình 28. Ứng dụng 2 — Quan hệ giữa các đặc trưng và giá.

Quan sát. Giá tăng theo diện tích nhưng độ tán rất lớn: hai căn cùng 80 m² có thể chênh nhau vài tỷ. Giá tăng đều theo số phòng ngủ và số tầng.

Diễn giải. Diện tích một mình không quyết định giá — **vị trí** mới quyết định, và vị trí đang nằm ẩn trong cột **Address** dạng văn bản tự do mà mô hình chưa đọc được.

Ý nghĩa với học máy. Đây là căn cứ trực tiếp cho kỹ thuật đặc trưng ở mục 8.5.1. Nếu bỏ qua bước rút vị trí, mô hình sẽ mãi không giải thích được phần lớn phương sai của giá.



Hình 29. Ứng dụng 2 — Ma trận tương quan Pearson.

Quan sát. Tương quan tuyến tính giữa từng biến số và **Price** đều thấp, dưới 0,3.

Diễn giải. Tương quan Pearson chỉ đo quan hệ **tuyến tính**. Giá bất động sản phụ thuộc vị trí theo cách phi tuyến và theo tương tác — 80 m² ở Hà Nội khác hẳn 80 m² ở tỉnh lẻ — nên hệ số tuyến tính thấp là điều được dự đoán trước.

Ý nghĩa với học máy. Đây là dự báo mạnh rằng **Linear Regression** sẽ kém hơn các mô hình dạng cây ở bài toán này, và bảng kết quả mục 8.8 xác nhận. Ở Ứng dụng 1 thì ngược lại: **Glucose** có tương quan tuyến tính 0,49 với nhân nên Logistic Regression cạnh tranh tốt.

8.7. Xây dựng mô hình

8.7.1. Chia tập và mô hình cơ sở

KẾT QUẢ CHẠY

```
BASELINE – luôn đoán giá trung bình của tập train
Giá trung bình train = 5.8571 tỷ VNĐ
MAE = 1.8670 tỷ ← mốc phải vượt
RMSE = 2.2361 tỷ
R² = -0.0001 ← đúng bằng 0 theo định nghĩa
```

Không phân tầng vì đây là bài toán hồi quy, biến mục tiêu liên tục nên không có “lớp” để giữ tỷ lệ. Với $N > 27.000$, phép chia ngẫu nhiên đủ để ba tập có phân phối giá tương đương.

8.7.2. Năm mô hình hồi quy

PYTHON

```
from sklearn.ensemble import GradientBoostingRegressor, RandomForestRegressor
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.tree import DecisionTreeRegressor

MODELS = {
    "linear_regression": ("Linear Regression", LinearRegression()),
    "ridge_regression": ("Ridge Regression", Ridge(alpha=1.0,
random_state=RANDOM_SEED)),
    "decision_tree_regressor": ("Decision Tree Regressor",
    DecisionTreeRegressor(max_depth=14, min_samples_leaf=8,
random_state=RANDOM_SEED)),
    "random_forest_regressor": ("Random Forest Regressor",
    RandomForestRegressor(n_estimators=300, max_depth=24, min_samples_leaf=4,
max_features=0.5, n_jobs=-1,
random_state=RANDOM_SEED)),
    "gradient_boosting_regressor": ("Gradient Boosting Regressor",
    GradientBoostingRegressor(n_estimators=700, learning_rate=0.06, max_depth=7,
subsample=0.85, min_samples_leaf=5,
random_state=RANDOM_SEED)),
}

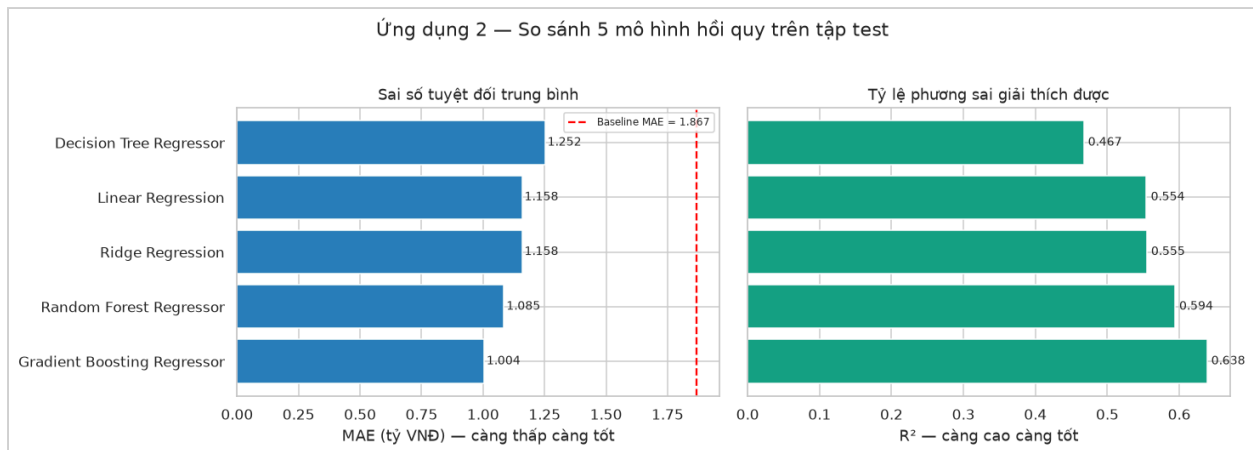
trained, train_times = {}, {}
for key, (label, model) in MODELS.items():
    t0 = time.perf_counter()
    model.fit(Xtr, y_train)
    train_times[key] = time.perf_counter() - t0
    trained[key] = model
    print(f"✓ {label:<30} {train_times[key]:>7.2f}s")
```

Định nghĩa năm mô hình hồi quy của Ứng dụng 2.

8.8. Đánh giá mô hình

Bảng 29. Ứng dụng 2 — So sánh năm mô hình hồi quy trên tập test, sắp theo R^2 .

Mô hình	MAE (tỷ)	MSE	RMSE (tỷ)	R^2
Gradient Boosting Regressor	1.0039	1.8082	1.3447	0.6384
Random Forest Regressor	1.0849	2.0302	1.4249	0.5939
Ridge Regression	1.1581	2.2259	1.4919	0.5548
Linear Regression	1.1581	2.2280	1.4927	0.5544
Decision Tree Regressor	1.2518	2.6647	1.6324	0.4671

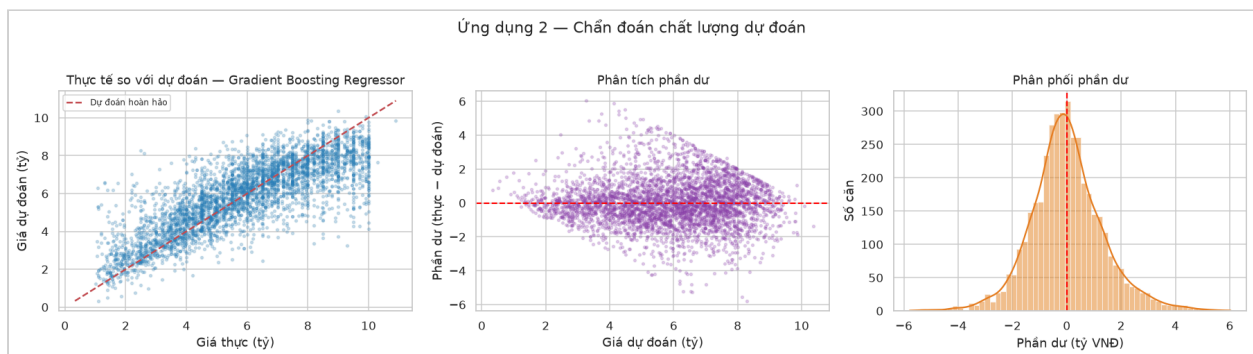


Hình 30. Ứng dụng 2 — So sánh năm mô hình trên tập test.

Mô hình tốt nhất là **Gradient Boosting Regressor** với $MAE = 1.0039$ tỷ VNĐ và $R^2 = 0.6384$, so với baseline $MAE = 1.8670$ tỷ và $R^2 = 0$. Nghĩa là hệ thống thường lệch khoảng 1.00 tỷ VNĐ và giải thích được 63.8% phương sai giá.

Ba mô hình dạng cây đều vượt hai mô hình tuyến tính, đúng như phân tích tương quan ở mục 8.6 đã dự báo.

8.8.1. Phân tích phần dư



Hình 31. Ứng dụng 2 — Chẩn đoán chất lượng dự đoán.

Quan sát. Phần dư phân bố quanh 0 gần đối xứng, nhưng biểu đồ thực-tế-so-với-dự-đoán cho thấy mô hình **kéo các dự đoán về phía trung tâm**: căn rẻ bị đoán đắt lên, căn đắt bị đoán rẻ đi.

Diễn giải. Đây là hiện tượng **co về trung bình**, đặc trưng của mô hình dạng cây: mỗi lá trả về trung bình của các mẫu rơi vào lá đó, nên không bao giờ dự đoán vượt quá khoảng giá đã thấy trong huấn luyện.

Ý nghĩa với học máy. Hệ thống đáng tin ở phân khúc phổ thông (nơi có nhiều dữ liệu) và kém tin cậy ở hai đầu. Giao diện Web vì vậy phải trình bày kết quả kèm **khoảng dao động**

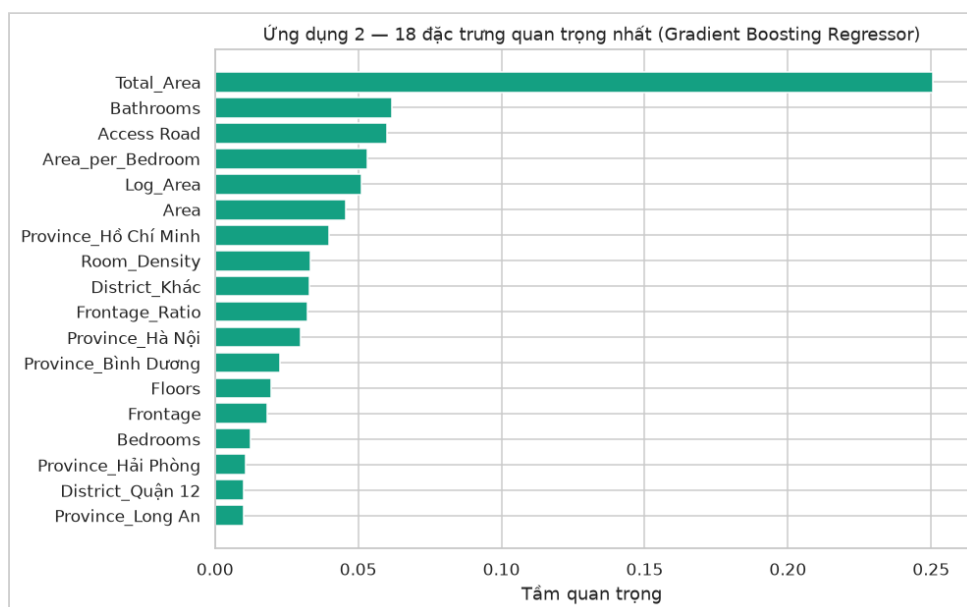
thay vì một con số duy nhất — nói “khoảng 5,2 tỷ ± 1,3 tỷ” trung thực hơn hẳn “5,234 tỷ”.

KẾT QUẢ CHẠY

8 dự đoán tệ nhất:

Sai số tuyệt đối trung bình theo khoảng giá:

MAE tăng rõ rệt ở hai đầu phổ giá và thấp nhất ở khoảng giữa. Một con số MAE tổng thể **che giấu** sự chênh lệch này; báo cáo MAE theo từng phân khúc là cách trung thực hơn để mô tả năng lực hệ thống.



Hình 32. Ứng dụng 2 — Các đặc trưng quan trọng nhất theo mô hình được chọn.

8.9. Lựa chọn mô hình

Tiêu chí	Nhận định
Hiệu năng dự báo	Xếp theo R ² và MAE trên tập test
Khả năng diễn giải	Linear/Ridge cho hệ số đọc được; rừng cây chỉ cho tầm quan trọng tương đối
Chi phí tính toán	Gradient Boosting huấn luyện lâu nhất; Linear gần như tức thì
Độ bền	Rừng cây bền với ngoại lệ nhờ trung bình hoá nhiều cây
Ràng buộc triển khai	Random Forest cho tập mô hình lớn nhất

Mô hình được chọn: Gradient Boosting Regressor. MAE = 1.0039 tỷ VNĐ, RMSE = 1.3447 tỷ, $R^2 = 0.6384$. Ở bài toán này hai tiêu chí đầu **xung đột nhau**: mô hình chính xác nhất lại khó giải thích nhất. Ta chọn theo hiệu năng, vì người dùng cuối cần một con số giá đáng tin hơn là một công thức đọc được.

8.10. Triển khai hệ thống

Hạng mục	Giá trị
Khung web	Flask 3.1
Điểm cuối	POST /house-price/v1/predict
Cổng	5002
Biến đầu vào số	Area, Frontage, Access Road, Floors, Bedrooms, Bathrooms
Biến đầu vào phân loại	House direction, Balcony direction, Legal status, Furniture state, Province, District
Đặc trưng phái sinh	API tự tính từ các trường cơ bản, dùng công thức đã dùng lúc huấn luyện
Quy tắc kiểm tra	Area phải lớn hơn 0; các trường số tùy chọn có thể để trống và sẽ được pipeline điền khuyết; hạng mục lạ được xử lý bằng <code>handle_unknown="ignore"</code>
Đầu ra	Giá dự đoán kèm khoảng dao động ± 1.34 tỷ VNĐ

JSON

```
{
  "Area": 85, "Frontage": 5, "Access Road": 8,
  "Floors": 4, "Bedrooms": 4, "Bathrooms": 3,
  "House direction": "East",
  "Legal status": "Have certificate",
  "Furniture state": "Full",
  "Province": "Hà Nội",
  "model": "gradient_boosting_regressor"
}
```

Ví dụ yêu cầu gửi tới API.

KẾT QUẢ CHẠY

```
--- Căn nhà 1 ---
85 m², 4 tầng, 4 PN, Hà Nội, Have certificate
Vectơ đầu vào: 111 chiều
→ Giá dự đoán: 7.146 tỷ VNĐ (khoảng tin cậy ±1.34 tỷ)

--- Căn nhà 2 ---
45 m², 2 tầng, 2 PN, Khác, Sale contract
Vectơ đầu vào: 111 chiều
→ Giá dự đoán: 3.297 tỷ VNĐ (khoảng tin cậy ±1.34 tỷ)

✓ Artifact nạp lại cho kết quả trùng khớp trên 4127 mẫu test.
✓ Sẵn sàng cho REST API (api/REST_API.py).
```

Kiểm thử suy luận từ artifact đã lưu.

Vì sao API phải tự tính đặc trưng phái sinh. Người dùng chỉ nhập sáu con số cơ bản, nhưng mô hình được huấn luyện trên 111 chiều bao gồm cả năm đặc trưng phái sinh. Nếu API không tính lại chúng theo *đúng công thức đã dùng lúc huấn luyện*, biểu diễn lúc dự đoán sẽ lệch khỏi biểu diễn lúc học — chính là vi phạm nguyên tắc nhất quán ở mục 2.10.

Vì mô hình hồi quy không sinh ra xác suất, giao diện Web thay thanh độ tin cậy quen thuộc bằng một thước đo khoảng giá: một vạch định vị giá dự đoán bên trong đoạn [giá thấp, giá cao] rộng ± 1.34 tỷ VNĐ quanh nó. Đây là một lựa chọn thiết kế có chủ đích, không phải một hạn chế bị bỏ qua — mục 8.8.1 đã chỉ ra mô hình dạng cây co dự đoán về trung bình, nên một con số đơn lẻ như “5,234 tỷ” ngụ ý một độ chính xác không có thật, trong khi khoảng dao động buộc người xem đọc kết quả đúng bản chất của nó: một ước lượng, không phải một câu trả lời tuyệt đối. Biểu đồ mức độ quan trọng đặc trưng bên dưới liệt kê các đặc trưng ảnh hưởng nhiều nhất tới giá nhưng — khác Ứng dụng 1 — không đối chiếu với giá trị người dùng vừa nhập, vì phần lớn cột trọng số cao nhất là cột sinh ra từ one-hot (từng tỉnh/thành, từng tình trạng pháp lý riêng lẻ) không còn map một-một về ô nhập gốc trên biểu mẫu. Hai hồ sơ mẫu — nhà cao cấp trung tâm và nhà bình dân ngoại thành — cố ý đặt ở hai đầu phổ giá, đúng vùng mà mục 8.8.1 vừa chỉ ra là mô hình kém tin cậy nhất.

MÔ HÌNH	MAE	RMSE	R ²
Decision Tree Regressor	1.2518	1.6324	0.4671
Gradient Boosting Regressor ★ tốt nhất	1.0039	1.3447	0.6384
Linear Regression	1.1581	1.4927	0.5544
Random Forest Regressor	1.0849	1.4249	0.5939
Ridge Regression	1.1581	1.4919	0.5548

Hình 33. Bảng so sánh năm mô hình hồi quy hiển thị ngay trên giao diện Web. Vì bảng lấy số trực tiếp từ `/metadata` — cùng một `metadata.json` đã dùng để dựng bảng ở mục 8.8 — con số người dùng nhìn thấy trên trình duyệt luôn trùng khớp với con số trong báo cáo; hàng tô nền, in đậm kèm ★ là mô hình đang chạy.

8.10.1. Giao diện Web

Ảnh dưới đây được chụp tự động từ dịch vụ đang chạy thật, bằng trình duyệt điều khiển qua kịch bản (`report/capture_screenshots.py`) — không phải ảnh dựng lại bằng công cụ thiết kế.

Dự đoán giá nhà

Nhập thông tin bất động sản để mô hình học máy ước tính giá trị. Kết quả chỉ mang tính tham khảo.

Thông tin bất động sản

Điền các trường bên dưới rồi bấm "Dự đoán giá nhà", hoặc dùng ví dụ mẫu bên dưới.

Quy ước: Diện tích và độ rộng đường vào tính bằng m². Ví dụ: 120 = 120 m².

Nhà cao cấp trung tâm

Nhà bình dân ngoại thành

Xoá form

Diện tích (Area, m²)

Mặt tiền (Frontage, m)

Độ rộng đường vào (Access Road, m)

Số tầng

Số phòng ngủ

Số phòng tắm

Hướng nhà

Hướng ban công

Tình trạng pháp lý

Tình trạng nội thất

Tỉnh/Thành phố

Quận/Huyện

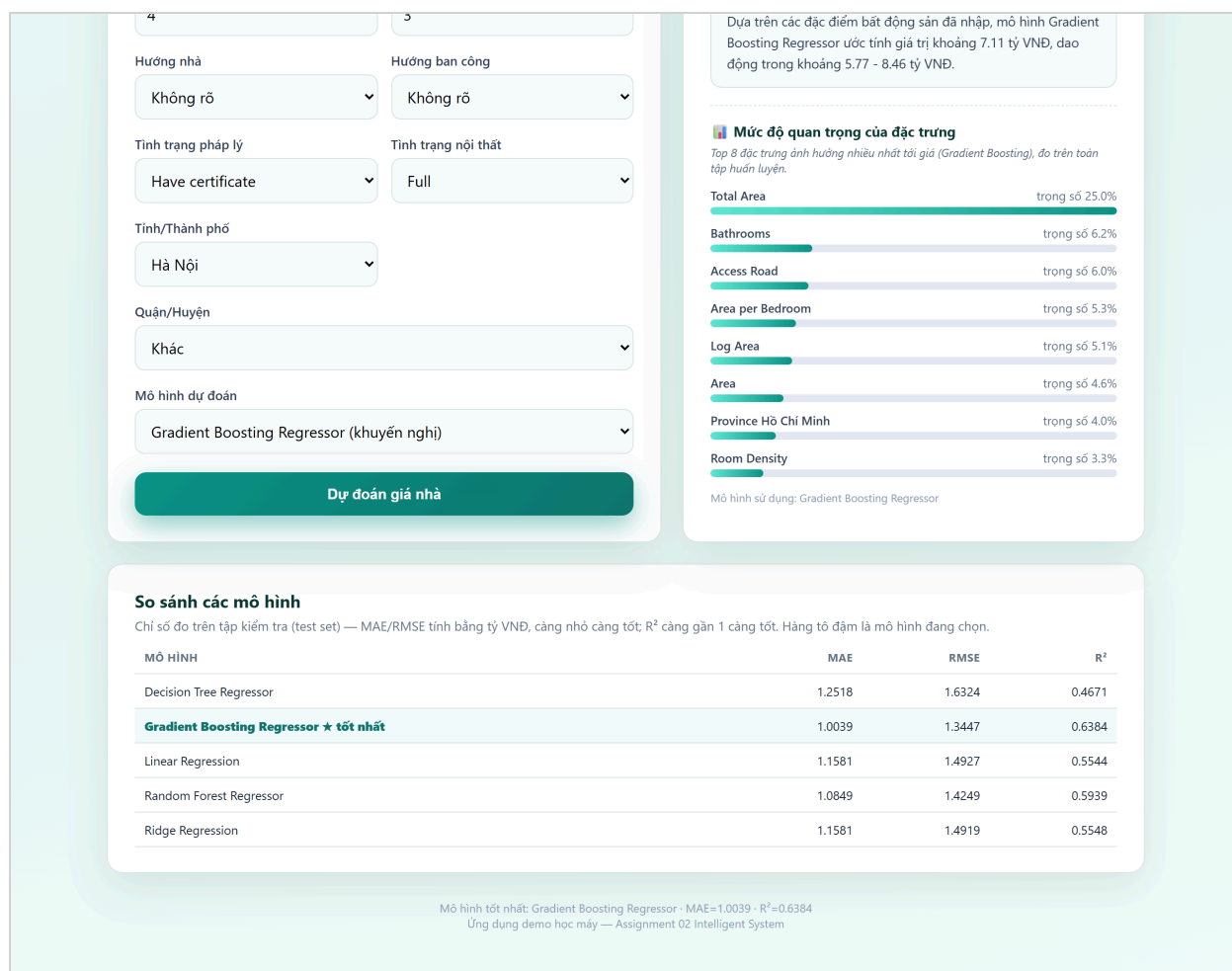
Mô hình dự đoán

Kết quả dự đoán

Giá dự đoán và khoảng dao động sẽ hiển thị tại đây.

Chưa có dữ liệu dự đoán. Hãy điền form bên trái và bấm "Dự đoán giá nhà".

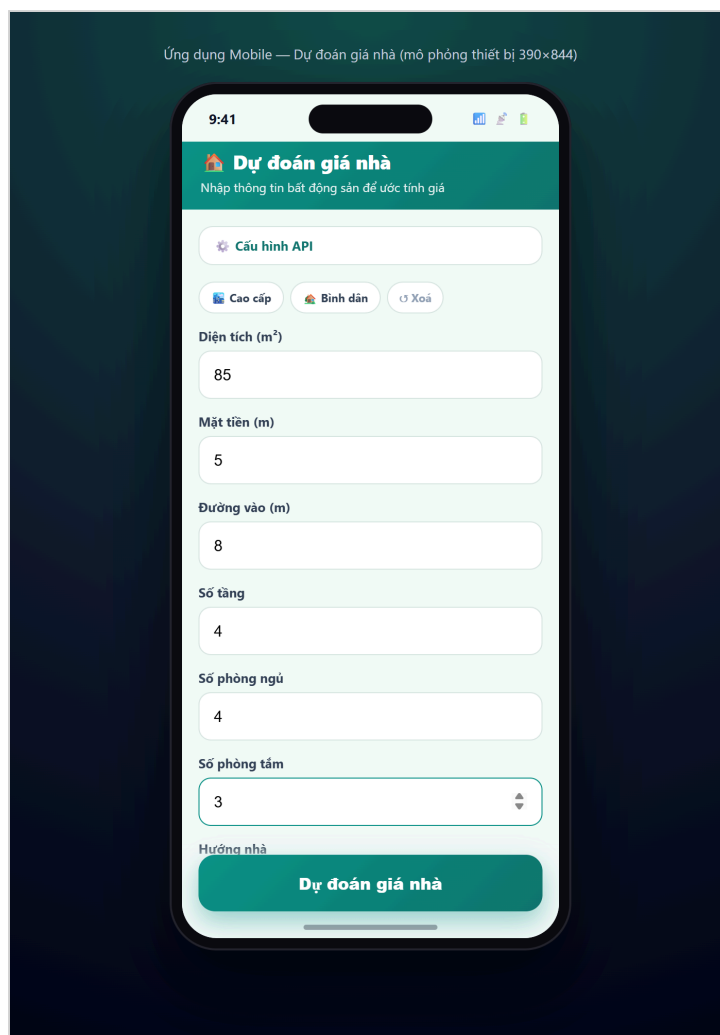
Hình 34. Ứng dụng 2 — Dự đoán giá nhà — giao diện Web, màn hình nhập liệu.



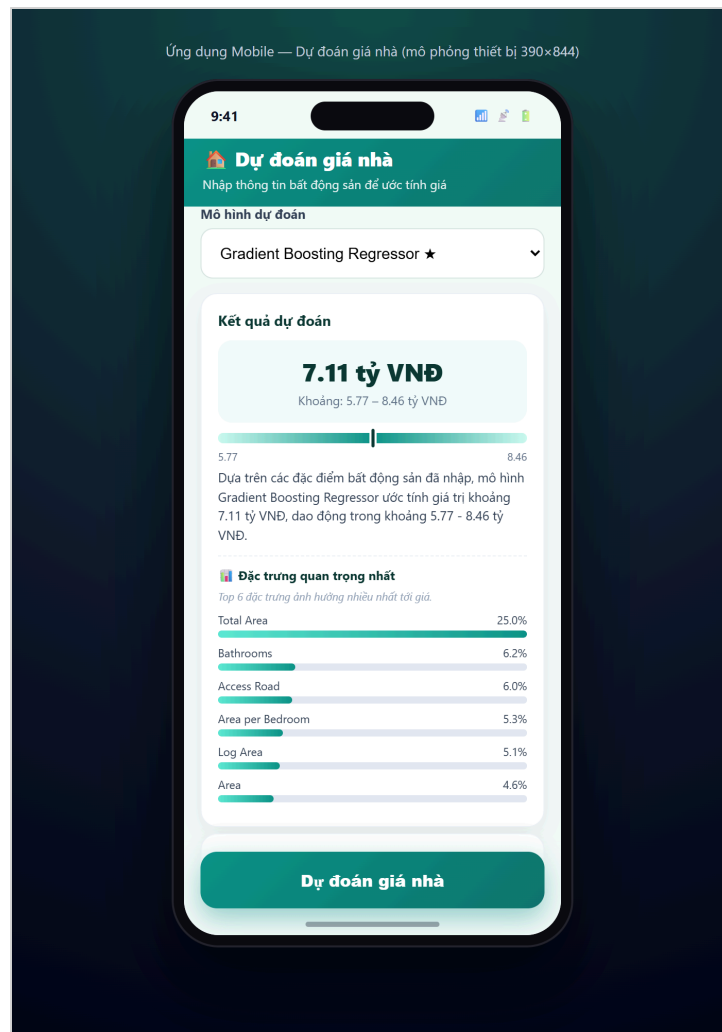
Hình 35. Ứng dụng 2 — Dự đoán giá nhà — giao diện Web, màn hình kết quả dự đoán.

8.10.2. Giao diện Mobile

Giao diện Mobile là trang mô phỏng thiết bị 390×844 phục vụ tại điểm cuối `http://127.0.0.1:5002/mobile`. Trang này gọi **đúng điểm cuối** `POST /house-price/v1/predict` mà giao diện Web gọi, nên nó minh họa đầy đủ luồng *Mobile* → *REST API* → *mô hình* mà không cần trình giả lập Android. Mã nguồn Flutter tương ứng nằm ở `house_price/mobile/lib/main.dart`.



Hình 36. Ứng dụng 2 — Dự đoán giá nhà — giao diện Mobile, màn hình nhập liệu.



Hình 37. Ứng dụng 2 — Dự đoán giá nhà — giao diện Mobile, màn hình kết quả dự đoán.

Chương IX. Ứng dụng 3 — Phân tích hành vi khách hàng và khám phá sở thích

9.1. Mô tả bài toán

Từ thông tin khách hàng và **nội dung đánh giá họ viết**, dự đoán khách hàng có khuyến nghị sản phẩm cho người khác hay không.

- X = đặc trưng hành vi khách hàng + biểu diễn văn bản của bình luận;
- y = `Recommended IND` $\in \{0, 1\}$.

Đây là bài toán **phân loại nhị phân**.

Vì sao chọn đúng mục tiêu này. Đề bài liệt kê nhiều lựa chọn và yêu cầu chọn một mục tiêu được xác định rõ ràng. `Recommended IND` là lựa chọn đúng vì ba lý do: nó **có sẵn nhãn thật** trong dữ liệu chứ không phải nhãn tự bịa bằng quy tắc; nó là **tín hiệu thương mại trực tiếp** vận hành xếp hạng sản phẩm và gợi ý cá nhân hoá; và nó **phụ thuộc mạnh vào nội dung bình luận**, nên là bài toán lý tưởng để đo giá trị thật của biểu diễn văn bản — đúng trọng tâm đề bài đặt ra.

Giới hạn phải nói rõ. Tập dữ liệu **không có Customer ID**. Mỗi dòng là một lượt đánh giá, không phải một khách hàng. Vì vậy mọi kết luận ở đây là ở *cấp lượt đánh giá*, và không thể khẳng định hành vi dài hạn của từng khách hàng.

9.2. Giới thiệu tập dữ liệu

Mục	Giá trị
Tên tập dữ liệu	Women's E-Commerce Clothing Reviews
Nguồn Kaggle	kaggle.com/datasets/nicapotato/womens-ecommerce-clothing-reviews
Số quan sát	23,486
Số thuộc tính	11 (10 đặc trưng + 1 biến mục tiêu)
Biến mục tiêu	<code>Recommended IND</code>
Tập cục bộ	<code>customer_behavior/data/womens_ecommerce_reviews.csv</code>

Nhóm	Cột
Số	Age, Rating, Positive Feedback Count, Clothing ID
Phân loại	Division Name, Department Name, Class Name
Văn bản	Title, Review Text
Mục tiêu	Recommended IND

Đây là ứng dụng **duy nhất có đủ cả ba loại dữ liệu**: số, phân loại và văn bản. Chính vì vậy nó đòi hỏi biểu diễn phức tạp nhất trong ba ứng dụng.

9.3. Biểu diễn khách hàng

Một lượt đánh giá được biểu diễn bằng hai nhóm thành phần ghép song song:

$$x_i = [x^{bảng}_i; x^{văn bản}_i]$$

- **Nhánh bảng** — tuổi, số lượt phản hồi hữu ích, độ dài bình luận, ba cấp phân loại sản phẩm;
- **Nhánh văn bản** — vector TF-IDF của tiêu đề ghép với nội dung đánh giá.

Mục 9.5 trình bày chi tiết chuỗi biến đổi của nhánh văn bản, và mục 9.8 đo xem nhánh ấy đóng góp bao nhiêu.

9.4. Làm sạch dữ liệu

9.4.1. Vấn đề nghiêm trọng nhất là rò rỉ nhãn

Rating (số sao 1–5) và Recommended IND gần như là **cùng một thông tin**: khách cho 5 sao thì hầu như chắc chắn khuyến nghị, cho 1 sao thì hầu như chắc chắn không.

KẾT QUẢ CHẠY

Bảng chéo Rating × Recommended IND:

Tỷ lệ khuyến nghị theo từng mức sao:

Quy tắc thô 'Rating >= 4 thì khuyến nghị' đạt accuracy = 0.9365

→ Chỉ một câu lệnh if đã đạt trên 90%. Giữ Rating làm đặc trưng là tự lừa mình.

→ QUYẾT ĐỊNH: loại bỏ Rating khỏi tập đặc trưng.

Đo mức trùng khớp giữa Rating và biến mục tiêu.

Vì sao phải loại bỏ cột Rating. Một quy tắc thô “Rating ≥ 4 thì khuyến nghị” đã đạt accuracy trên 90% mà không học được gì. Nếu giữ Rating làm đặc trưng, mọi mô hình sẽ đạt điểm rất cao nhưng **không mô hình nào học được gì về hành vi khách hàng** — chúng chỉ đọc lại số sao. Tệ hơn, khi triển khai thật, ta muốn dự đoán khuyến nghị *từ nội dung khách viết*, mà lúc ấy Rating cũng chưa có.

Đây là **rò rỉ nhãn** — dạng rò rỉ tinh vi hơn hẳn việc `fit` scaler trên tập test, vì nó không vi phạm bất kỳ quy tắc pipeline nào và không có công cụ nào cảnh báo. Biểu hiện duy nhất là điểm số *quá đẹp*. Clothing ID cũng bị loại, vì nó là định danh sản phẩm và không khái quát sang mã hàng mới.

9.4.2. Trùng lặp và bình luận rỗng

KẾT QUẢ CHẠY

```
Ban đầu           : 23486 dòng
Sau khử trùng lặp   : 23010 dòng (-476)
Sau loại dòng không có bình luận : 22639 dòng (-371)
Giữ lại 96.4% dữ liệu gốc
```

Đã ghép Title + Review Text thành cột 'full_text'.

Quá trình làm sạch giữ lại 22,639 trên 23,486 dòng. Dòng không có bình luận bị loại vì ứng dụng này lấy văn bản làm trung tâm — một dòng không có văn bản thì nhánh biểu diễn quan trọng nhất là rỗng. Title thiếu thì thay bằng chuỗi rỗng rồi ghép vào Review Text, vì tiêu đề thường cô đọng đúng cảm xúc chính.

9.5. Biểu diễn dữ liệu văn bản

Đây là phần trung tâm của ứng dụng và là nơi duy nhất trong Assignment 02 trình bày đầy đủ chuỗi biến đổi mà Bài giảng 02 yêu cầu:

Bình luận $\rightarrow$ Token $\rightarrow$ Token ID $\rightarrow$ Vector $\rightarrow$ $\mathbf{E} \in \mathbb{R}^{\mathbf{B} \times \mathbf{T} \times \mathbf{d}}$

9.5.1. Bốn bước trên một bình luận thật

PYTHON

```
sample_text = work["full_text"].iloc[3]
print("=" * 78)
print("BƯỚC 1 – BÌNH LUẬN GỐC (chuỗi ký tự)")
print("=" * 78)
print(f'    {sample_text}')
print(f'    Kiểu dữ liệu: {type(sample_text).__name__}, độ dài {len(sample_text)} ký tự')
print("    → Mô hình học máy KHÔNG nhận được chuỗi ký tự.")

print("\n" + "=" * 78)
print("BƯỚC 2 – TÁCH TOKEN (Tokenization)")
print("=" * 78)
tokens = re.findall(r"[a-z0-9']+ ", sample_text.lower())
print(f"    Số token: {len(tokens)}")
print(f"    20 token đầu: {tokens[:20]}")
print("    → Văn bản đã thành danh sách đơn vị ngôn ngữ, nhưng vẫn là chữ.")
```

Tách token từ bình luận gốc.

KẾT QUẢ CHẠY

```
=====
BƯỚC 1 – BÌNH LUẬN GỐC (chuỗi ký tự)
=====
    "My favorite buy! I love, love, love this jumpsuit. it's fun, flirty, and fabulous!
    every time i wear it, i get nothing but great compliments!"
    Kiểu dữ liệu: str, độ dài 141 ký tự
    → Mô hình học máy KHÔNG nhận được chuỗi ký tự.

=====
BƯỚC 2 – TÁCH TOKEN (Tokenization)
=====
    Số token: 25
    20 token đầu: ['my', 'favorite', 'buy', 'i', 'love', 'love', 'love', 'this',
'jumpsuit', "it's", 'fun', 'flirty', 'and', 'fabulous', 'every', 'time', 'i', 'wear',
'it', 'i']
... (rút gọn, còn 1 dòng)
```

Bước 1 và 2 — từ chuỗi ký tự sang danh sách token.

KẾT QUẢ CHẠY

```
=====
BƯỚC 3 – TOKEN ID (ánh xạ mỗi token thành một số nguyên)
=====

Kích thước từ điển minh hoạ: 21 token duy nhất
Ánh xạ 10 token đầu:
  'my'          ' → ID 16
  'favorite'    ' → ID 6
  'buy'         ' → ID 2
  'i'           ' → ID 11
  'love'        ' → ID 15
  'love'        ' → ID 15
  'love'        ' → ID 15
  'this'        ' → ID 18
  'jumpsuit'    ' → ID 14
  'it's'        ' → ID 13

Chuỗi Token ID (20 đầu): [16, 6, 2, 11, 15, 15, 15, 18, 14, 13, 8, 7, 0, 5, 4, 19,
11, 20, 12, 11]
Kiểu: danh sách số nguyên, độ dài T = 25
→ Đã thành số, nhưng ID chỉ là NHÃN, không mang ngữ nghĩa:
  ID 5 không 'lớn hơn' hay 'gần' ID 4 về mặt ý nghĩa.
```

Bước 3 — ánh xạ mỗi token thành một số nguyên.

KẾT QUẢ CHẠY

```
=====
BƯỚC 4 – VECTƠ TF-IDF (biểu diễn số có mang trọng số ngữ nghĩa)
=====

Kích thước từ vựng minh hoạ (d): 200
Vectơ kết quả: thưa, hình dạng (1, 200), 8 phần tử khác 0
8 chiều có trọng số cao nhất:
  'love'          ' → 0.5252
  'favorite'      ' → 0.3843
  'compliments'  ' → 0.3567
  'fun'           ' → 0.3510
  'buy'           ' → 0.3504
  'time'          ' → 0.3462
  'wear'          ' → 0.2081
  'great'         ' → 0.2003

→ Mỗi chiều là MỘT TỪ; giá trị là tầm quan trọng của từ đó
  trong bình luận này so với toàn bộ kho văn bản.
```

Bước 4 — vectơ TF-IDF, mỗi chiều là một từ cụ thể.

Điểm cần nhấn mạnh ở bước 3: token ID đã là số, nhưng **ID chỉ là nhãn, không mang ngữ nghĩa**. ID 5 không “gần” ID 4 hơn ID 900 về mặt ý nghĩa. Phải sang bước 4 hoặc bước 5 thì giá trị số mới thực sự phản ánh nội dung.

9.5.2. Tensor embedding $E \in \mathbb{R}^{(B \times T \times d)}$

TF-IDF cho mỗi văn bản *một vector duy nhất*, tức $X \in \mathbb{R}^{N \times d}$ — mất hoàn toàn thứ tự từ. Mô hình mạng nơ-ron dùng cách khác: mỗi *token* được ánh xạ thành một vector d chiều, nên một văn bản thành ma trận $T \times d$, và một lô B văn bản thành tensor ba chiều.

PYTHON

```
B, T, D_EMB = 4, 24, 16
batch_texts = work["full_text"].head(B).tolist()

corpus_tokens = [re.findall(r"[a-z0-9']+", t.lower()) for t in
work["full_text"].head(6000)]
vocab = {"<PAD>": 0, "<UNK>": 1}
for toks in corpus_tokens:
    for t in toks:
        if t not in vocab:
            vocab[t] = len(vocab)
V = len(vocab)

id_matrix = np.zeros((B, T), dtype=np.int64) # B x T
for i, text in enumerate(batch_texts):
    toks = re.findall(r"[a-z0-9']+", text.lower())[:T]
    for j, t in enumerate(toks):
        id_matrix[i, j] = vocab.get(t, 1)

rng = np.random.default_rng(RANDOM_SEED)
embedding_table = rng.normal(0, 0.1, size=(V, D_EMB)) # V x d
E = embedding_table[id_matrix] # B x T x d

print(f"Kích thước từ điển V = {V:,} token duy nhất")
print(f"\nMa trận Token ID : {id_matrix.shape} (B x T)")
print(id_matrix[:2, :12], "...")
print(f"\nBảng embedding : {embedding_table.shape} (V x d)")
print(f"\nTENSOR EMBEDDING : {E.shape} (B x T x d)")
print(f"    B = {E.shape[0]:>3} số văn bản trong lô")
print(f"    T = {E.shape[1]:>3} số token mỗi văn bản (đã cắt/đệm về cùng độ dài)")
print(f"    d = {E.shape[2]:>3} số chiều embedding của mỗi token")
print(f"    Tổng số phần tử: {E.size:,} - dtype {E.dtype}")
print(f"\nVector embedding của token đầu tiên, văn bản đầu tiên (d = {D_EMB} chiều):")
print("    ", np.round(E[0, 0], 4))
```

Dựng thật tensor embedding ba chiều.

KẾT QUẢ CHẠY

Kích thước từ điển $V = 8,513$ token duy nhất

Ma trận Token ID : $(4, 24)$ $(B \times T)$

```
[[ 2  3  4  5  6  5  7  0  0  0  0  0]
 [ 8  9 10 11 12 13 14 15 16 17 18 19]] ...
```

Bảng embedding : $(8513, 16)$ $(V \times d)$

TENSOR EMBEDDING : $(4, 24, 16)$ $(B \times T \times d)$

$B = 4$ số văn bản trong lô

$T = 24$ số token mỗi văn bản (đã cắt/đệm về cùng độ dài)

$d = 16$ số chiều embedding của mỗi token

Tổng số phần tử: 1,536 – dtype float64

Vectơ embedding của token đầu tiên, văn bản đầu tiên ($d = 16$ chiều):

```
[-0.0512 -0.0814  0.0616  0.1129 -0.0114 -0.084  -0.0824  0.0651  0.0743
 0.0543 -0.0666  0.0232  0.0117  0.0219  0.0871  0.0224]
```

Ba chiều B , T , d với giá trị cụ thể.

Chiều	Giá trị	Ý nghĩa
B	4	Số văn bản trong một lô (batch size)
T	24	Số token mỗi văn bản, đã cắt hoặc đệm về cùng độ dài
d	16	Số chiều embedding của mỗi token
V	8,513	Kích thước từ điển (số token duy nhất)
Hình dạng tensor	$4 \times 24 \times 16$	$E \in \mathbb{R}^{4 \times 24 \times 16}$

9.5.3. Vì sao triển khai bằng TF-IDF chứ không bằng embedding

	TF-IDF (dùng để triển khai)	Embedding (minh hoạ khái niệm)
Hình dạng	$\mathbb{R}^{N \times d}$ — hai chiều	$\mathbb{R}^{B \times T \times d}$ — ba chiều
Giữ thứ tự từ	Không	Có
Mỗi chiều nghĩa là gì	Một từ cụ thể — đọc được	Chiều ẩn học được — không đọc được
Chi phí huấn luyện	Giây	Giờ, cần GPU
Cần bao nhiêu dữ liệu	Vài nghìn dòng là đủ	Hàng trăm nghìn dòng trở lên

Với 22,639 dòng, embedding học từ đầu sẽ quá khớp. Hơn nữa TF-IDF cho phép **chỉ ra từ nào đẩy dự đoán về phía nào** — điều mà mục 9.9 khai thác và là thứ một hệ thống thương mại điện tử cần để giải thích kết quả cho người vận hành.

Thông tin nào được giữ: từ nào xuất hiện, mức độ hiếm của từ đó, cường độ tín hiệu cảm xúc. **Thông tin nào bị mất:** thứ tự từ, phủ định tầm xa (“not good” bị tách thành hai token độc lập — một phần được bù bằng n-gram bậc 2), mĩa mai, và ngữ cảnh vượt câu.

9.5.4. Hai biểu diễn song song

PYTHON

```
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

numeric_branch = Pipeline([("impute", SimpleImputer(strategy="median")),
                           ("scale", StandardScaler())])
categorical_branch = Pipeline([
    ("impute", SimpleImputer(strategy="constant", fill_value="Unknown")),
    ("onehot", OneHotEncoder(handle_unknown="ignore", sparse_output=False))]
)
text_branch = TfidfVectorizer(stop_words="english", ngram_range=(1, 2),
                             max_features=20000, min_df=2, sublinear_tf=True)

# Biểu diễn 1 – Chỉ đặc trưng dạng bảng
pre_tabular = ColumnTransformer([
    ("num", numeric_branch, NUM_FEATURES),
    ("cat", categorical_branch, CAT_FEATURES),
], remainder="drop")

# Biểu diễn 2 – bảng + văn bản
pre_hybrid = ColumnTransformer([
    ("num", numeric_branch, NUM_FEATURES),
    ("cat", categorical_branch, CAT_FEATURES),
    ("text", text_branch, TEXT_COL),
], remainder="drop")

Xtr_tab = pre_tabular.fit_transform(X_train)
Xva_tab = pre_tabular.transform(X_val)
Xte_tab = pre_tabular.transform(X_test)

Xtr_hyb = pre_hybrid.fit_transform(X_train)
Xva_hyb = pre_hybrid.transform(X_val)
Xte_hyb = pre_hybrid.transform(X_test)

print("BIỂU DIỄN 1 – chỉ đặc trưng dạng bảng")
print(f"  Xtr: {Xtr_tab.shape} | Xte: {Xte_tab.shape}")
print("\nBIỂU DIỄN 2 – bảng + văn bản (TF-IDF)")
print(f"  Xtr: {Xtr_hyb.shape} | Xte: {Xte_hyb.shape}")
print(f"  → văn bản đóng góp thêm {Xtr_hyb.shape[1] - Xtr_tab.shape[1]:,} chiều")
print(f"  Kiểu ma trận: {type(Xtr_hyb).__name__}")
if hasattr(Xtr_hyb, "nnz"):
    density = Xtr_hyb.nnz / (Xtr_hyb.shape[0] * Xtr_hyb.shape[1])
    print(f"  Mật độ khác 0: {density*100:.4f}% → ma trận cực thưa, phải lưu ở dạng sparse")
```

Hai biểu diễn được dựng song song để so sánh trực tiếp.

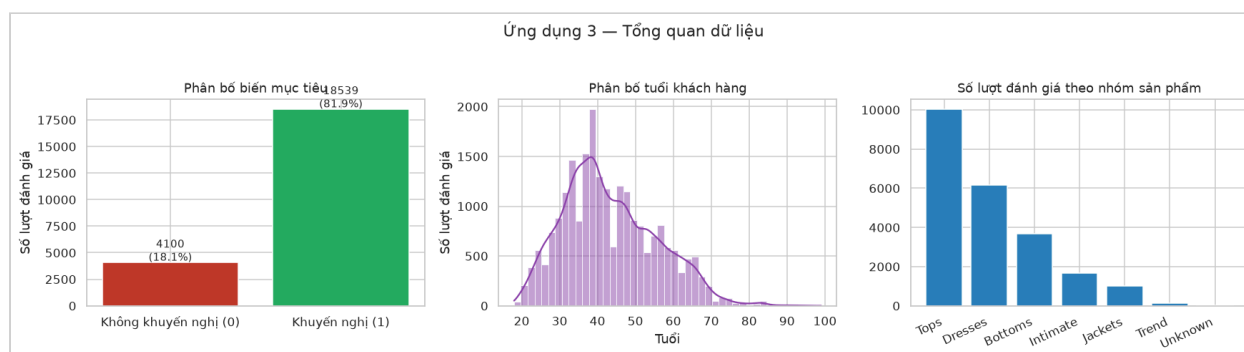
KẾT QUẢ CHẠY

BIỂU DIỄN 1 – chỉ đặc trưng dạng bảng
Xtr: (15846, 34) | Xte: (3396, 34)

BIỂU DIỄN 2 – bảng + văn bản (TF-IDF)
Xtr: (15846, 20034) | Xte: (3396, 20034)
→ văn bản đóng góp thêm 20,000 chiều
Kiểu ma trận: csr_matrix
Mật độ khác 0: 0.2095% → ma trận cực thưa, phải lưu ở dạng sparse

Biểu diễn	Hình dạng ma trận train	Số chiều	Ghi chú
Chỉ bảng	(15,846, 34)	34	3 cột số + one-hot của 3 biến phân loại
Bảng + văn bản	(15,846, 20,034)	20,034	văn bản đóng góp thêm 20,000 chiều; ma trận thưa
Chỉ văn bản	(15,846, 20,000)	20,000	TF-IDF 1–2 gram, dùng cho Naive Bayes

9.6. Khám phá dữ liệu (EDA)

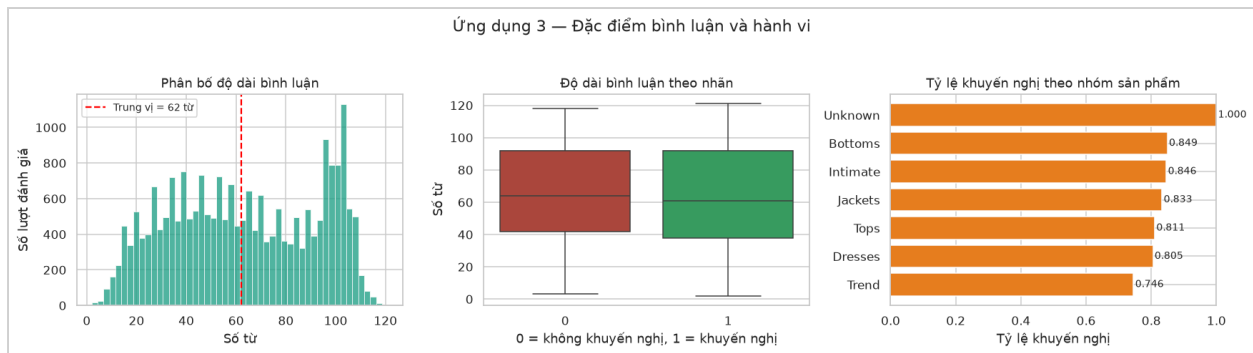


Hình 38. Ứng dụng 3 — Tổng quan dữ liệu.

Quan sát. Khoảng 82% lượt đánh giá là khuyến nghị — tỷ lệ mất cân bằng khoảng 4,5 : 1, nặng hơn hẳn Ứng dụng 1 (1,87 : 1).

Diễn giải. Đây là **thiên lệch tự chọn**: người mua hài lòng có xu hướng viết đánh giá nhiều hơn người thất vọng, vì người thất vọng thường chỉ trả hàng rồi thôi.

Ý nghĩa với học máy. Baseline “luôn đoán khuyến nghị” đạt tới 81.9% accuracy. Với mức mất cân bằng này, accuracy gần như vô dụng để xếp hạng mô hình — phải dùng **F1 của lớp thiểu số** và ROC-AUC, đồng thời bật `class_weight="balanced"`.

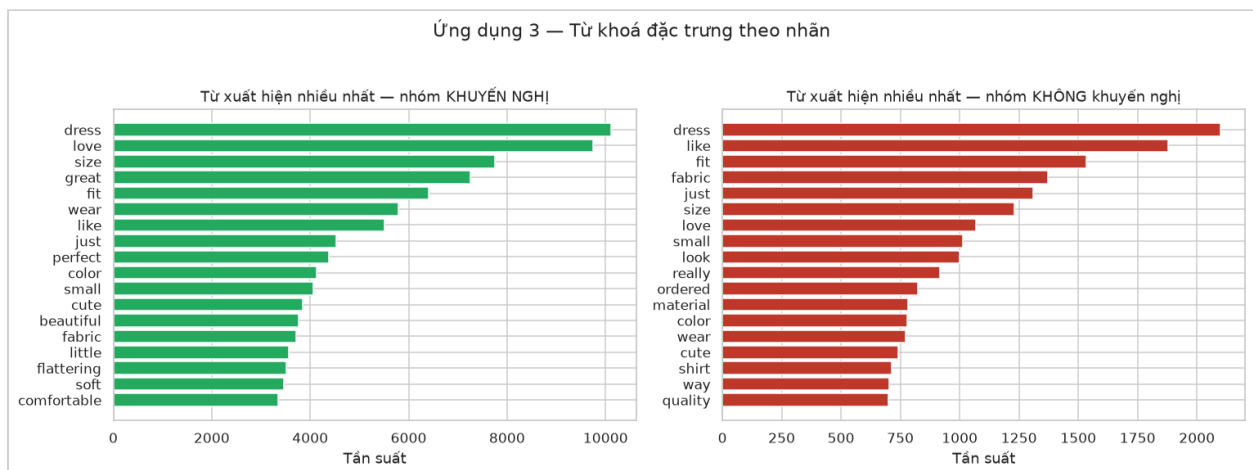


Hình 39. Ứng dụng 3 — Đặc điểm bình luận và hành vi.

Quan sát. Bình luận của nhóm **không** khuyến nghị **dài hơn** nhóm khuyến nghị.

Diễn giải. Khách hài lòng viết ngắn (“Love this dress!”); khách thất vọng viết dài để giải thích cái gì sai — sai size, vải mỏng, màu khác ảnh.

Ý nghĩa với học máy. Bản thân **độ dài văn bản đã là một đặc trưng dự báo**, độc lập với nội dung từ ngữ, nên `word_count` được đưa vào nhánh bảng. Đây là ví dụ về việc một thuộc tính *siêu dữ liệu* của văn bản mang thông tin mà mô hình túi-từ không tự nắm được.



Hình 40. Ứng dụng 3 — Từ khoá xuất hiện nhiều nhất theo từng nhân.

Quan sát. Hai nhóm chia sẻ nhiều từ chung (*dress, fabric, size, fit*) nhưng khác nhau ở các từ mang cảm xúc và từ chỉ vấn đề.

Diễn giải. Danh từ sản phẩm xuất hiện ở cả hai nhóm nên **không phân biệt được**; thứ phân biệt là tính từ đánh giá và từ chỉ lỗi.

Ý nghĩa với học máy. Đây chính xác là lý do phải dùng **TF-IDF** chứ **không phải đếm tần suất thô**: TF-IDF hạ trọng số những từ xuất hiện ở khắp nơi và nâng trọng số những từ mang tính phân biệt.

9.7. Xây dựng mô hình

Bộ sáu mô hình được thiết kế để trả lời câu hỏi trung tâm — văn bản có giá trị bao nhiêu. Ba mô hình đầu chỉ dùng đặc trưng bảng, ba mô hình sau được đọc thêm văn bản; tất cả dùng **cùng một cách chia dữ liệu**, nên chênh lệch điểm số giữa hai nhóm đo đúng đóng góp của biểu diễn văn bản.

#	Mô hình	Biểu diễn	Vai trò
1	Logistic Regression	Chỉ bảng	Chuẩn tham chiếu không dùng văn bản
2	Decision Tree	Chỉ bảng	Kiểm tra xem phi tuyến trên đặc trưng bảng có cứu vãn được không
3	Random Forest	Chỉ bảng	Trần hiệu năng của biểu diễn chỉ-bảng
4	Logistic Regression	Bảng + văn bản	Phân loại tuyến tính trên văn bản — mô hình đề bài chỉ đích danh
5	Linear SVM	Bảng + văn bản	Biên lớn; chuẩn mạnh cho dữ liệu văn bản thưa
6	Multinomial Naive Bayes	Chỉ văn bản	Mô hình xác suất cổ điển cho phân loại văn bản

PYTHON

```

from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import MultinomialNB
from sklearn.svm import LinearSVC
from sklearn.tree import DecisionTreeClassifier

# Nhánh chỉ-văn-bản cho Naive Bayes (TF-IDF không âm)
text_only = TfidfVectorizer(stop_words="english", ngram_range=(1, 2),
                             max_features=20000, min_df=2, sublinear_tf=True)
Xtr_txt = text_only.fit_transform(X_train[TEXT_COL])
Xva_txt = text_only.transform(X_val[TEXT_COL])
Xte_txt = text_only.transform(X_test[TEXT_COL])
print("Biểu diễn chỉ-văn-bản:", Xtr_txt.shape)

SPECS = {
    "logistic_regression_tabular": {
        "label": "Logistic Regression (chỉ bảng)", "repr": "tabular",
        "model": LogisticRegression(max_iter=2000, class_weight="balanced",
                                     random_state=RANDOM_SEED)},
    "decision_tree_tabular": {
        "label": "Decision Tree (chỉ bảng)", "repr": "tabular",
        "model": DecisionTreeClassifier(max_depth=10, min_samples_leaf=20,
                                         class_weight="balanced",
                                         random_state=RANDOM_SEED)},
    "random_forest_tabular": {
        "label": "Random Forest (chỉ bảng)", "repr": "tabular",
        "model": RandomForestClassifier(n_estimators=250, max_depth=16,
                                       min_samples_leaf=5,
                                       class_weight="balanced", n_jobs=-1,
                                       random_state=RANDOM_SEED)},
    "logistic_regression_hybrid": {
        "label": "Logistic Regression (bảng + văn bản)", "repr": "hybrid",
        "model": LogisticRegression(max_iter=3000, C=4.0, class_weight="balanced",
                                     random_state=RANDOM_SEED)},
    "linear_svm_hybrid": {
        "label": "Linear SVM (bảng + văn bản)", "repr": "hybrid",
        "model": LinearSVC(C=0.5, class_weight="balanced", max_iter=5000,
                           random_state=RANDOM_SEED)},
    "multinomial_nb_text": {
        "label": "Multinomial Naive Bayes (chỉ văn bản)", "repr": "text",
        "model": MultinomialNB(alpha=0.3)},
}

MATRICES = {
    "tabular": (Xtr_tab, Xva_tab, Xte_tab),
    "hybrid": (Xtr_hyb, Xva_hyb, Xte_hyb),
    "text": (Xtr_txt, Xva_txt, Xte_txt),
}

trained, train_times = {}, {}

```

```

for key, spec in SPECS.items():
    Xa, _, _ = MATRICES[spec["repr"]]
    t0 = time.perf_counter()
    spec["model"].fit(Xa, y_train)
    train_times[key] = time.perf_counter() - t0
    trained[key] = spec["model"]
    print(f"✓ {spec['label']:<42} {train_times[key]:>7.2f}s  "
          f"[{spec['repr']}, {Xa.shape[1]:,} chiều]")

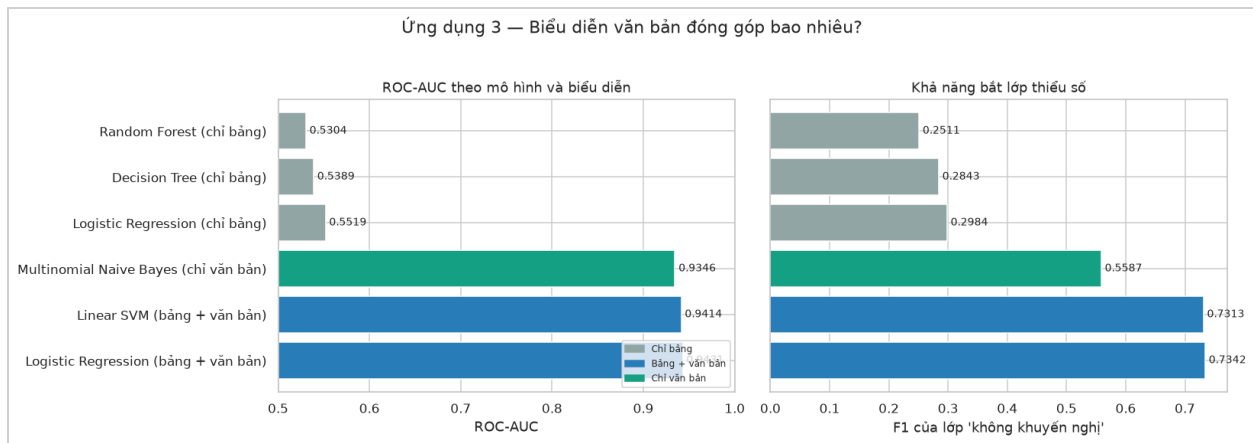
```

Định nghĩa sáu mô hình và biểu diễn tương ứng.

9.8. Đánh giá mô hình

Bảng 38. Ứng dụng 3 — So sánh sáu mô hình trên tập test, sắp theo ROC-AUC.

Mô hình	Biểu diễn	Accur acy	Precis ion	Reca ll	F1	F1 (lớp 0)	ROC- AUC
Logistic Regression (bảng + văn bản)	Bảng + văn bản	0.895 8	0.953 0	0.91 80	0.93 52	0.734 2	0.9431
Linear SVM (bảng + văn bản)	Bảng + văn bản	0.895 5	0.950 9	0.91 98	0.93 51	0.731 3	0.9414
Multinomial Naive Bayes (chỉ văn bản)	Chỉ văn bản	0.877 2	0.885 5	0.97 63	0.92 87	0.558 7	0.9346
Logistic Regression (chỉ bảng)	Chỉ bảng	0.538 9	0.841 5	0.53 83	0.65 66	0.298 4	0.5519
Decision Tree (chỉ bảng)	Chỉ bảng	0.519 7	0.832 0	0.51 82	0.63 86	0.284 3	0.5389
Random Forest (chỉ bảng)	Chỉ bảng	0.606 6	0.824 4	0.66 02	0.73 32	0.251 1	0.5304



Hình 41. Ứng dụng 3 — Biểu diễn văn bản đóng góp bao nhiêu?

9.8.1. Câu hỏi trung tâm — văn bản có cải thiện dự đoán không?

KẾT QUẢ CHẠY

TÁC ĐỘNG CỦA BIỂU DIỄN VĂN BẢN

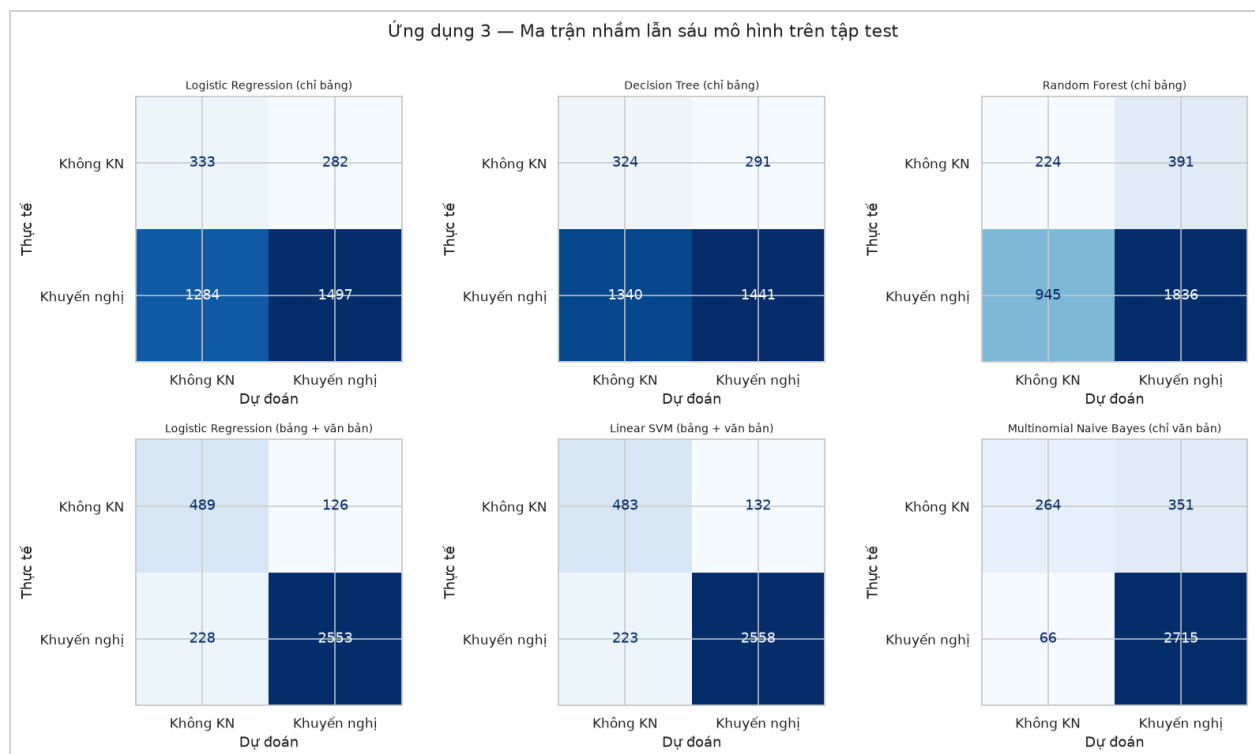
- Biểu diễn văn bản cải thiện ROC-AUC thêm +0.3912
- F1 của lớp thiểu số (không khuyến nghị) cải thiện +146.1%

So sánh trực tiếp biểu diễn chỉ-bảng với biểu diễn có kèm văn bản.

Quan sát. Mô hình chỉ dùng đặc trưng bảng đạt ROC-AUC cao nhất 0.5519; mô hình có đọc văn bản đạt 0.9431 — chênh lệch 0.3912. Khoảng cách lớn nhất nằm ở F1 của lớp thiểu số.

Diễn giải. Đặc trưng bảng (tuổi, nhóm hàng, số lượt hữu ích) gần như **không mang tín hiệu** về việc khách có hài lòng hay không — chúng mô tả *ai đánh giá* và *đánh giá cái gì*, chứ không mô tả *khách nghĩ gì*. Ý kiến nằm trong văn bản, và chỉ nằm ở đó.

Ý nghĩa với học máy. Đây là bằng chứng mạnh nhất trong cả ba ứng dụng cho luận điểm trung tâm của Assignment 02: **chọn biểu diễn đúng có tác động lớn hơn hẳn chọn thuật toán mạnh**. Random Forest với 250 cây trên đặc trưng bảng vẫn thua Multinomial Naive Bayes — mô hình đơn giản nhất trong sáu — khi Naive Bayes được đọc văn bản.



Hình 42. Ứng dụng 3 — Ma trận nhầm lẫn của sáu mô hình trên tập test.

KẾT QUẢ CHẠY

Diễn giải ma trận nhầm lẫn – Logistic Regression (bảng + văn bản)

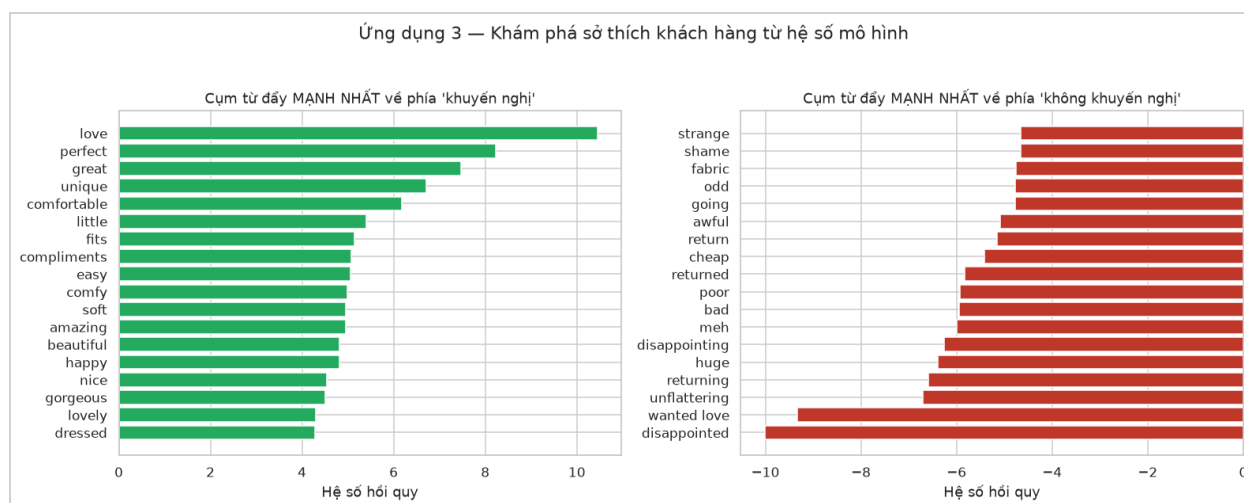
TP = 2553 khách khuyến nghị, dự đoán đúng
 TN = 489 khách KHÔNG khuyến nghị, phát hiện đúng → tín hiệu cảnh báo sản phẩm
 FP = 126 khách không khuyến nghị nhưng bị đoán là hài lòng
 → nguy hiểm về mặt kinh doanh: phản hồi xấu bị bỏ lọt, sản phẩm lỗi tiếp tục bán
 FN = 228 khách hài lòng bị đoán là không hài lòng
 → chỉ gây rà soát thừa, chi phí thấp

→ Với bài toán này, FP đắt hơn FN, nên Recall của LỚP 0 mới là chỉ số vận hành.
 $\text{Recall lớp 0} = \text{TN} / (\text{TN} + \text{FP}) = 489 / 615 = 0.7951$

Diễn giải ma trận nhầm lẫn theo nghĩa kinh doanh.

Đáng chú ý: **ưu tiên ở đây ngược với Ứng dụng 1**. Ở bài toán tiểu đường, FN (bỏ sót ca bệnh) là sai lầm đắt nhất. Ở đây, FP — phản hồi xấu bị đoán nhầm thành hài lòng — mới nguy hiểm, vì nó khiến sản phẩm lỗi tiếp tục được bán mà không ai biết. Vì vậy chỉ số vận hành của ứng dụng này là **Recall của lớp 0**.

9.9. Khám phá sở thích và hành vi khách hàng



Hình 43. Ứng dụng 3 — Cụm từ có ảnh hưởng mạnh nhất tới dự đoán.

Quan sát. Các cụm từ tiêu cực mạnh nhất chỉ vào **vấn đề cụ thể của sản phẩm**: sai kích cỡ, chất liệu mỏng, khác ảnh, phải trả hàng. Các cụm từ tích cực chỉ vào cảm xúc và độ vừa vặn.

Diễn giải. Đây chính là “khám phá sở thích khách hàng” mà đề bài yêu cầu. Mô hình không chỉ dự đoán một nhân; hệ số của nó là **một danh sách xếp hạng những điều khách hàng quan tâm nhất**, rút tự động từ hơn hai mươi nghìn bình luận mà không cần ai đọc tay.

Ý nghĩa với học máy. Đây cũng là lý do tiêu chí “khả năng diễn giải” có sức nặng bất thường ở ứng dụng này: một mô hình hộp đen dù nhỉnh hơn vài phần nghìn ROC-AUC cũng không cho ra được bảng này.

9.9.1. Ứng dụng trong thương mại điện tử

Doanh nghiệp dùng được ngay ba việc từ kết quả trên:

1. **Cảnh báo sớm sản phẩm có vấn đề** khi tần suất các cụm từ tiêu cực tăng đột biến ở một mã hàng;
2. **Sửa bảng thông số kích cỡ** ở đúng những mã hàng bị than phiền về size — đây là nhóm từ tiêu cực xuất hiện nhiều nhất;
3. **Ưu tiên hiển thị** những sản phẩm được khen đúng các thuộc tính mà khách coi trọng.

9.9.2. Phân tích sai số

KẾT QUẢ CHẠY

Số dự đoán sai: 354 / 3396 (10.42%)

4 bình luận bị phân loại sai (rút gọn 220 ký tự):

[thật=0 dự đoán=1 điểm=0.995]

Love it! I didn't realize how much i would love this. wore it to work the other day and got more compliments than i have in a very long time on an outfit! it's comfortable and easy and flattering. bought this totally on ...

[thật=1 dự đoán=0 điểm=0.124]

I wanted it to work I ordered a medium and it fit perfectly. i am anywhere from a size 6-8. 5'9, 155lbs.
sadly, i am returning this dress because the horizontal stripes make my hips look huge....

[thật=1 dự đoán=0 điểm=0.226]

Sparkly & gold tones I didn't think from the photo online that the sweater would be quite so sparkly & gold! it has gold weaved in - so if the sparkly isn't your thing, beware. the sweater fit well although i didn't feel...

[thật=1 dự đoán=0 điểm=0.23]

I loved this top so much i bought it in both colors. i have not seen anyone complain about this, but on me, the mock neck collapses in and looks messy. unfortunately i returned both. i sew, and i do not understand why th...

Quan sát. Các trường hợp sai thường là bình luận **mâu thuẫn nội tại**: khen kiểu dáng nhưng chê chất liệu, hoặc khen sản phẩm nhưng vẫn trả hàng vì sai size.

Diễn giải. Với biểu diễn túi-từ, một bình luận chứa cả từ tích cực lẫn tiêu cực sẽ có hai nhóm tín hiệu triệt tiêu nhau. Mô hình không nắm được **cấu trúc nhượng bộ** kiểu “*đẹp nhưng không vừa*” — mà chính về sau mới quyết định nhãn.

Ý nghĩa với học máy. Đây là **giới hạn cố hữu của biểu diễn túi-từ, không phải của thuật toán**. Muốn vượt qua thì phải đổi biểu diễn — sang embedding có trật tự và mô hình đọc được ngữ cảnh, đúng như tensor $B \times T \times d$ minh họa ở mục 9.5.2. Một lần nữa: rào cản nằm ở biểu diễn, không nằm ở mô hình.

9.10. Lựa chọn mô hình

Mô hình được chọn: Logistic Regression (bảng + văn bản). ROC-AUC = 0.9431, F1 = 0.9352, F1 lớp 0 = 0.7342, Accuracy = 0.8958. Ngoài hiệu năng dẫn đầu, mô hình này còn cho **hệ số đọc được từng từ** — chính là sản phẩm “khám phá sở thích khách hàng” ở mục 9.9, thứ mà một mô hình hộp đen không đánh đổi được.

9.11. Triển khai hệ thống

Hạng mục	Giá trị
Khung web	Flask 3.1
Điểm cuối	POST /customer-behavior/v1/predict
Cổng	5003
Biến đầu vào	Age, Positive Feedback Count, Title, Review Text, Division Name, Department Name, Class Name
Đặc trưng API tự tính	full_text = Title + Review Text; word_count = số từ của full_text
Quy tắc kiểm tra	Bình luận rỗng bị từ chối với mã 400 — đây là nhánh biểu diễn quan trọng nhất, không có nó thì dự đoán vô nghĩa
Ba bộ tiền xử lý	preprocessor.joblib (lai), preprocessor_tabular.joblib (chỉ bảng), tfidf_text_only.joblib (chỉ văn bản) — chọn theo biểu diễn của mô hình người dùng chọn

JSON

```
{
  "Age": 34,
  "Positive Feedback Count": 3,
  "Title": "",
  "Review Text": "Very disappointed. The material feels cheap and thin,\n it runs two sizes too small. Returned it.",
  "Division Name": "General",
  "Department Name": "Tops",
  "Class Name": "Blouses",
  "model": "logistic_regression_hybrid"
}
```

Ví dụ yêu cầu gửi tới API.

KẾT QUẢ CHẠY

--- Đánh giá 1 ---

"Absolutely love this dress! The fabric is soft, the fit is perfect and I got so many com..."

Vectơ đầu vào: 20,034 chiều (thừa)

→ Dự đoán: lớp 1 – Khuyến nghị sản phẩm

→ Độ tin cậy: 99.30%

--- Đánh giá 2 ---

"Very disappointed. The material feels cheap and thin, it runs two sizes too small and lo..."

Vectơ đầu vào: 20,034 chiều (thừa)

→ Dự đoán: lớp 0 – Không khuyến nghị sản phẩm

→ Độ tin cậy: 99.89%

✓ Artifact nạp lại cho kết quả trùng khớp trên 3,396 mẫu test.

✓ Sẵn sàng cho REST API (api/REST_API.py).

Kiểm thử suy luận từ artifact đã lưu.

Giao diện Web của ứng dụng này lấy ô nhập bình luận làm trọng tâm, và ngay dưới kết quả là danh sách từ khoá tích cực/tiêu cực có trọng số cao nhất từ mô hình tuyến tính, trong đó những từ thật sự xuất hiện trong bình luận vừa nhập được tô viền đậm để phân biệt với phần còn lại của danh sách. Hộp chọn cho phép đổi giữa sáu mô hình; khi người dùng đổi mô hình rồi bấm dự đoán lại trên cùng một bình luận, nhãn “Biểu diễn: ...” cạnh kết quả cùng đổi theo — chỉ bảng, bảng + văn bản, hoặc chỉ văn bản — để thấy tận mắt cùng một bình luận cho ra dự đoán khác nhau tùy biểu diễn nào được mô hình sử dụng. Đây chính là phát hiện trung tâm của toàn báo cáo (mục 9.5, 9.8.1): thêm văn bản vào biểu diễn thay đổi kết quả nhiều hơn thay đổi thuật toán, và giao diện để phát hiện đó tự chứng minh chứ không chỉ được phát biểu bằng lời.

MÔ HÌNH	ACCURACY	PRECISION	RECALL	F1	ROC-AUC
Decision Tree (chỉ bảng)	0.5197	0.8320	0.5182	0.6386	0.5389
Linear SVM (bảng + văn bản)	0.8955	0.9509	0.9198	0.9351	0.9414
Logistic Regression (bảng + văn bản) ★ tốt nhất	0.8958	0.9530	0.9180	0.9352	0.9431
Logistic Regression (chỉ bảng)	0.5389	0.8415	0.5383	0.6566	0.5519
Multinomial Naive Bayes (chỉ văn bản)	0.8772	0.8855	0.9763	0.9287	0.9346
Random Forest (chỉ bảng)	0.6066	0.8244	0.6602	0.7332	0.5304

Hình 44. Bảng so sánh sáu mô hình hiển thị ngay trên giao diện Web, hàng tô nền và in đậm kèm ★ là mô hình đang chọn. Bảng đọc thẳng từ /metadata — cùng nguồn với bảng ở mục 9.8 — nên số liệu trên giao diện và trong báo cáo không bao giờ lệch nhau.

9.11.1. Giao diện Web

Ảnh dưới đây được chụp tự động từ dịch vụ đang chạy thật, bằng trình duyệt điều khiển qua kịch bản (`report/capture_screenshots.py`) — không phải ảnh dựng lại bằng công cụ thiết kế.

The screenshot shows a web application titled "Dự đoán khuyến nghị khách hàng" (Customer Recommendation Prediction). The main heading is "Dự đoán khuyến nghị khách hàng" with a subtitle: "Nhập nội dung đánh giá sản phẩm và thông tin liên quan để mô hình dự đoán khả năng khách hàng khuyến nghị sản phẩm." (Enter product review content and related information for the model to predict the likelihood of customer recommendation).

The form is divided into two main sections:

- Thông tin đánh giá** (Review Information):
 - Instructions: "Nội dung đánh giá càng chi tiết, dự đoán càng chính xác. Có thể dùng ví dụ mẫu bên dưới." (The more detailed the review content, the more accurate the prediction. You can use the example below.)
 - Buttons: "Đánh giá tích cực" (Positive Review), "Đánh giá tiêu cực" (Negative Review), and "Xoá form" (Clear form).
 - Field: "Tiêu đề đánh giá (không bắt buộc)" (Review Title (optional)) with a placeholder "Ví dụ: Great fit!" (Example: Great fit!).
 - Field: "Nội dung đánh giá" (Review Content) with a placeholder "Very disappointed. The material feels cheap and thin, it runs two sizes too small and looks nothing like the picture. Returned it."
 - Note: "Trường bắt buộc — không được để trống." (Required field — do not leave blank).
 - Fields: "Tuổi khách hàng" (Customer Age) with value 45, and "Số lượt phản hồi tích cực" (Number of positive feedback) with value 0.
 - Fields: "Phân khúc (Division)" (General) and "Danh mục (Department)" (Dresses).
 - Field: "Loại sản phẩm (Class)" (Dresses).
 - Field: "Mô hình dự đoán" (Logistic Regression (bảng + văn bản) (khuyến nghị)) (Prediction Model).
 - Button: "Dự đoán" (Predict).
- Kết quả dự đoán** (Prediction Result):
 - Text: "Kết quả và độ tin cậy của mô hình sẽ hiển thị tại đây." (The result and model confidence will be displayed here.)
 - Text: "Chưa có dữ liệu dự đoán. Hãy nhập đánh giá bên trái và bấm 'Dự đoán'." (No prediction data yet. Enter the review on the left and click 'Predict').

Hình 45. Ứng dụng 3 — Hành vi khách hàng thương mại điện tử — giao diện Web, màn hình nhập liệu.



Dự đoán khuyến nghị khách hàng

Nhập nội dung đánh giá sản phẩm và thông tin liên quan để mô hình dự đoán khả năng khách hàng khuyến nghị sản phẩm.

Thông tin đánh giá

Nội dung đánh giá càng chi tiết, dự đoán càng chính xác. Có thể dùng ví dụ mẫu bên dưới.

Đánh giá tích cực

Đánh giá tiêu cực

Xoá form

Tiêu đề đánh giá (không bắt buộc)

Ví dụ: Great fit!

Nội dung đánh giá

Very disappointed. The material feels cheap and thin, it runs two sizes too small and looks nothing like the picture. Returned it.

Trường bắt buộc — không được để trống.

Tuổi khách hàng

Số lượt phản hồi tích cực

45

0

Phân khúc (Division)

Danh mục (Department)

General

Dresses

Loại sản phẩm (Class)

Dresses

Mô hình dự đoán

Logistic Regression (bảng + văn bản) (khuyến nghị)

Dự đoán

Kết quả dự đoán

Kết quả và độ tin cậy của mô hình sẽ hiển thị tại đây.

Không khuyến nghị

Không khuyến nghị sản phẩm

Độ tin cậy mô hình 99.88%

Dựa trên nội dung đánh giá và thông tin sản phẩm, mô hình dự đoán khách hàng khó có khả năng khuyến nghị sản phẩm này.

Từ khoá quan trọng nhất

Trong số học từ mô hình tuyến tính trên toàn tập huấn luyện. Từ được tô viền đậm là từ có xuất hiện trong đánh giá bạn vừa nhập.

Từ khoá tích cực

Từ khoá tiêu cực

amazing beautiful comfortable comfy compliments dressed easy fits gorgeous great

awful bad cheap disappointed disappointing fabric going huge meh odd

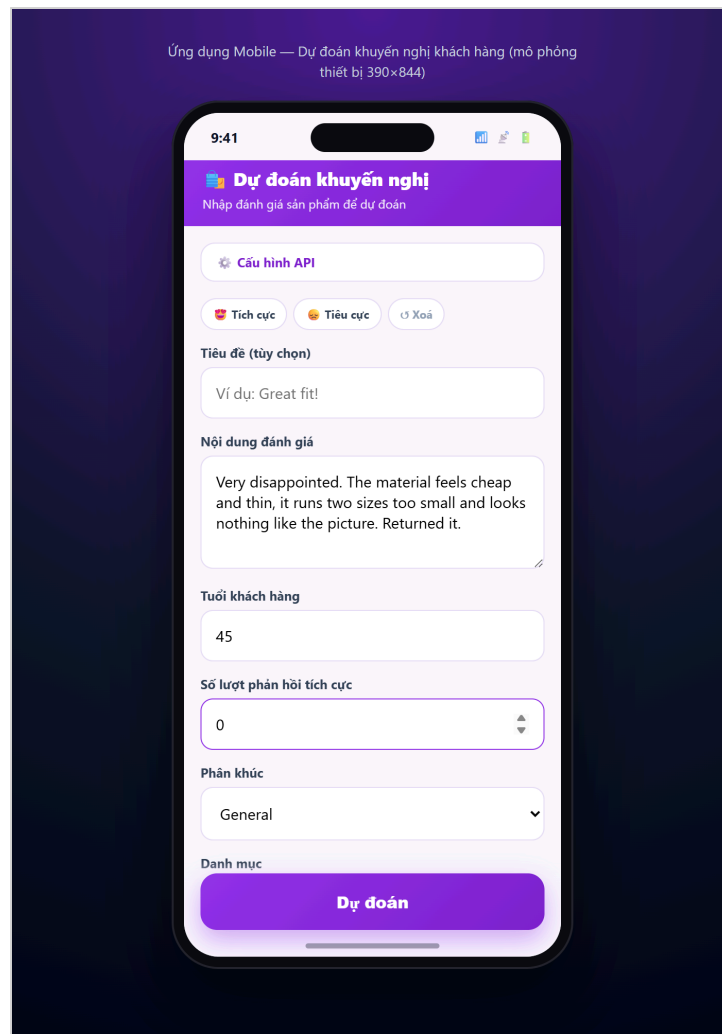
Biểu diễn: Bảng + văn bản

Mô hình sử dụng: Logistic Regression (bảng + văn bản)

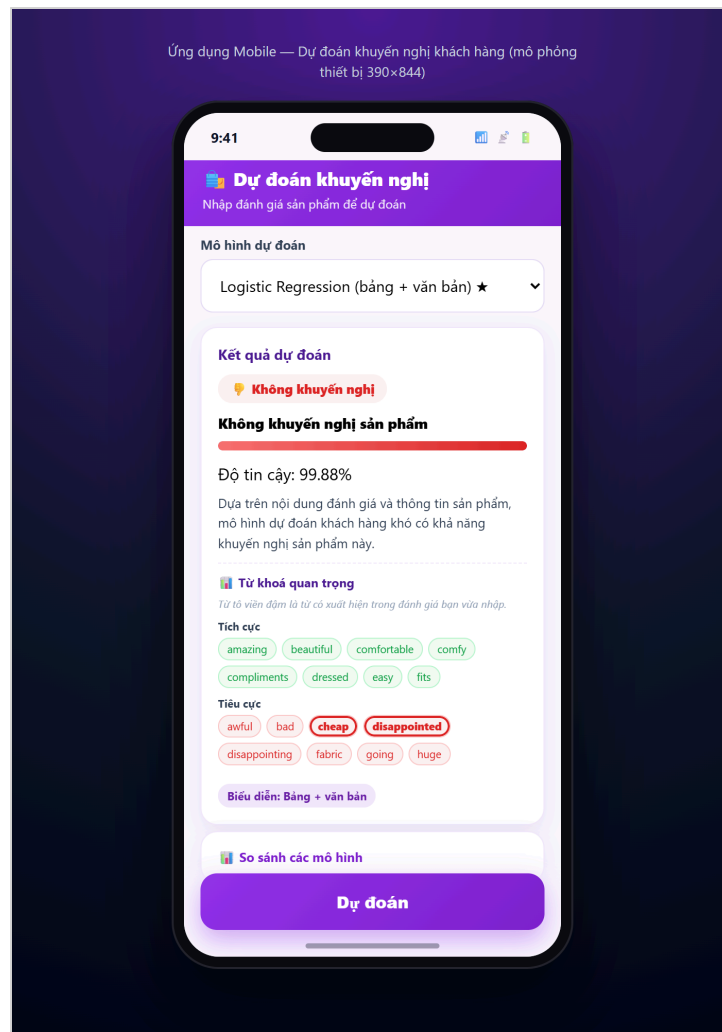
Hình 46. Ứng dụng 3 — Hành vi khách hàng thương mại điện tử — giao diện Web, màn hình kết quả dự đoán.

9.11.2. Giao diện Mobile

Giao diện Mobile là trang mô phỏng thiết bị 390×844 phục vụ tại điểm cuối `http://127.0.0.1:5003/mobile`. Trang này gọi **đúng điểm cuối** `POST /customer-behavior/v1/predict` mà giao diện Web gọi, nên nó minh hoạ đầy đủ luồng *Mobile* → *REST API* → *mô hình* mà không cần trình giả lập Android. Mã nguồn Flutter tương ứng nằm ở `customer_behavior/mobile/lib/main.dart`.



Hình 47. Ứng dụng 3 — Hành vi khách hàng thương mại điện tử — giao diện Mobile, màn hình nhập liệu.



Hình 48. Ứng dụng 3 — Hành vi khách hàng thương mại điện tử — giao diện Mobile, màn hình kết quả dự đoán.

Chương X. So sánh ba hệ thống thông minh

Ba ứng dụng được xây dựng theo cùng một quy trình và cùng một khung trình bày, nên chúng so sánh được với nhau. Chương này rút ra những gì chỉ nhìn thấy khi đặt cả ba cạnh nhau.

10.1. So sánh dữ liệu và bài toán

Khía cạnh	Ứng dụng 1 — Tiểu đường	Ứng dụng 2 — Giá nhà	Ứng dụng 3 — Thương mại điện tử
Loại bài toán	Phân loại nhị phân	Hồi quy	Phân loại nhị phân
Dữ liệu thô	CSV, bảng thuần số	CSV, bảng có biến phân loại	CSV, bảng + văn bản tự do
Một quan sát	Một lần khám của một bệnh nhân	Một tin rao bán bất động sản	Một lượt đánh giá sản phẩm
Số mẫu thô	768	30,229	23,486
Số mẫu sau làm sạch	768	27,513	22,639
Biến mục tiêu	Outcome $\in \{0,1\}$	Price $\in \mathbb{R}^+$ (tỷ VNĐ)	Recommended IND $\in \{0,1\}$
Mất cân bằng lớp	1,87 : 1	Không áp dụng (hồi quy)	$\approx 4,5 : 1$

Ba tập dữ liệu khác nhau ở đâu. Ứng dụng 1 nhỏ nhất (768 dòng) nhưng sạch nhất về cấu trúc — mọi cột đều là số. Ứng dụng 2 lớn nhất (30,229 dòng) và bẩn nhất về giá trị thiếu. Ứng dụng 3 ở giữa về kích thước nhưng phức tạp nhất về kiểu dữ liệu.

10.2. So sánh biểu diễn dữ liệu

Bảng 41. Bảng tóm tắt biểu diễn dữ liệu bắt buộc theo yêu cầu đề bài.

Ứng dụng	Dạng thô	Biểu diễn số	Đầu vào mô hình	Số chiều
Tiểu đường	CSV / bảng	Vectơ và ma trận đặc trưng	$B \times d$	5
Giá nhà	CSV / bảng	Ma trận đặc trưng đã mã hoá	$B \times d$	111
Thương mại điện tử	CSV + bình luận	Đặc trưng bảng + vectơ / embedding văn bản	$B \times d$ và/hoặc $B \times T \times d$	20,034

Giải thích từng chiều trong các hình dạng đã báo cáo:

Ký hiệu	Nghĩa	Giá trị cụ thể trong báo cáo này
N	Số quan sát trong toàn tập	768 / 27,513 / 22,639
B	Số quan sát trong một lô đưa vào mô hình	Ứng dụng 3 minh hoạ với $B = 4$
d	Số chiều của một vectơ đặc trưng	5 / 111 / 20,034
T	Số token của một văn bản, sau khi cắt hoặc đệm	Chỉ có ở Ứng dụng 3: $T = 24$
V	Kích thước từ điển token	Chỉ có ở Ứng dụng 3: $V = 8,513$

Điểm khác biệt cốt lõi. Ứng dụng 1 dùng 5 chiều, Ứng dụng 2 dùng 111 chiều, Ứng dụng 3 dùng 20,034 chiều — chênh nhau hơn ba bậc độ lớn. Nguyên nhân không phải bài toán khó hơn, mà là **loại dữ liệu khác nhau**: mỗi từ trong từ vựng trở thành một chiều riêng, nên văn bản luôn sinh ra không gian rất nhiều chiều và rất thưa.

10.3. So sánh chất lượng dữ liệu và tiền xử lý

10.3.1. Các bước tiền xử lý chung

Bốn bước có mặt ở cả ba ứng dụng: khử bản ghi trùng lặp trước khi chia tập; điền khuyết bằng trung vị bên trong Pipeline; chia 70/15/15; và đóng gói toàn bộ tiền xử lý thành một artifact duy nhất để triển khai.

10.3.2. Các bước tiền xử lý riêng

Ứng dụng	Vấn đề chất lượng đặc thù	Xử lý riêng	Nếu bỏ qua thì sao
Tiểu đường	Giá trị thiếu mã hoá thành số 0 (Insulin 48,7%)	<code>FunctionTransformer</code> chuyển 0 thành <code>NaN</code> trước khi điền khuyết	Mô hình học rằng “glucose = 0” là một tình trạng sức khoẻ hợp lệ
Giá nhà	Thiếu tới 82,6%; bản ghi trùng lặp; địa chỉ dạng văn bản tự do	Coi “không khai báo” là một hạng mục; khử trùng lặp; rút tỉnh và huyện từ địa chỉ; thêm 5 đặc trưng phái sinh	Mất 82,6% dữ liệu nếu xoá dòng; R^2 bị thổi phồng nếu không khử trùng; mất hoàn toàn biến vị trí
Thương mại điện tử	Rò rỉ nhãn qua cột <code>Rating</code> ; mất cân bằng 4,5 : 1; bình luận rỗng	Loại bỏ <code>Rating</code> và <code>Clothing ID</code> ; <code>class_weight="balanced"</code> ; loại dòng không có bình luận	Accuracy trên 90% nhưng mô hình chỉ đọc lại số sao, vô dụng khi triển khai

Cả ba ứng dụng đều có một vấn đề chất lượng mà công cụ tiêu chuẩn không phát hiện được, và mỗi lần là một dạng khác nhau. Đây là phát hiện đáng chú ý nhất khi đặt ba ứng dụng cạnh nhau: sự cố về dữ liệu không lặp lại theo khuôn mẫu, nên không có một danh sách kiểm tra cố định nào thay được việc thực sự đọc và hiểu dữ liệu.

10.4. So sánh mô hình và đánh giá

Bảng 44. So sánh kết quả cuối cùng của ba hệ thống.

Ứng dụng	Số mô hình	Mô hình được chọn	Độ đo chính	Kết quả chính	So với baseline
Tiểu đường	5	Random Forest	Recall	ROC-AUC 0.8526, Recall 0.7750	Accuracy 0.7759 so với 0.6552
Giá nhà	5	Gradient Boosting Regressor	MAE	MAE 1.0039 tỷ, R^2 0.6384	MAE giảm 46.2% so với baseline
Thương mại điện tử	6	Logistic Regression (bảng + văn bản)	F1 lớp thiếu số	ROC-AUC 0.9431, F1 lớp 0 0.7342	Accuracy 0.8958 so với 0.8189

Vì sao độ đo chính khác nhau ở cả ba. Ứng dụng 1 ưu tiên Recall vì bỏ sót ca bệnh đắt hơn cảnh báo nhầm. Ứng dụng 3 ưu tiên F1 lớp thiểu số vì ngược lại — bỏ lọt phản hồi xấu mới là sai lầm đắt. Ứng dụng 2 dùng MAE vì nó là con số duy nhất người dùng cuối đọc được trực tiếp bằng tỷ VNĐ. Không có một độ đo nào đúng cho cả ba.

Phát hiện nhất quán ở cả ba ứng dụng. Ở mỗi ứng dụng, khoảng cách điểm số do **thay đổi biểu diễn dữ liệu** lớn hơn khoảng cách do **thay đổi thuật toán**:

- **Ứng dụng 1** — chênh lệch ROC-AUC giữa mô hình tốt nhất (0.853) và kém nhất (0.773) là 0.079.
- **Ứng dụng 2** — năm đặc trưng phái sinh cải thiện R^2 ở mọi mô hình; khử trùng lặp thay đổi R^2 nhiều hơn cả việc đổi thuật toán.
- **Ứng dụng 3** — chênh lệch ROC-AUC do biểu diễn là 0.391, lớn hơn hẳn chênh lệch giữa các thuật toán trong cùng một biểu diễn.

10.5. So sánh triển khai

Khía cạnh	Tiểu đường	Giá nhà	Thương mại điện tử
Cổng	5001	5002	5003
Điểm cuối	/diabetes/v1/predict	/house-price/v1/predict	/customer-behavior/v1/predict
Số trường người dùng nhập	5	11	7
API tự tính thêm	Không	5 đặc trưng phái sinh	full_text, word_count
Số artifact phải nạp	1 preprocessor + 5 mô hình	1 preprocessor + 5 mô hình	3 preprocessor + 6 mô hình
Đầu ra	Nhãn + độ tin cậy + tri thức Neo4j	Giá + khoảng dao động	Nhãn + độ tin cậy + tên biểu diễn
Triển khai Web	Có	Có	Có
Triển khai Mobile	Có	Có	Có

Ứng dụng nào dễ triển khai nhất. Ứng dụng 1 — người dùng nhập năm con số, API không phải tính thêm gì, artifact nhỏ. **Ứng dụng nào khó nhất.** Ứng dụng 3 — phải nạp ba bộ tiền xử lý khác nhau và chọn đúng bộ theo mô hình người dùng chọn, cộng với từ điển TF-IDF 20,034 chiều khiến artifact lớn hơn hẳn. Ứng dụng 2 nằm giữa nhưng có rủi ro riêng: API phải tính lại năm đặc trưng phải sinh đúng *y* *hết* công thức lúc huấn luyện, nếu lệch thì không có gì báo lỗi. **Ứng dụng nào tốn tài nguyên tính toán nhất.** Ứng dụng 2 khi huấn luyện (Gradient Boosting trên 27,513 dòng $\times$ 111 chiều). Khi suy luận thì cả ba đều dưới một phần trăm giây.

10.6. Tổng hợp và bài học rút ra

Năm mục so sánh ở trên đối chiếu ba hệ thống trên từng chiều riêng lẻ. Mục này tổng hợp lại: điều gì đúng ở cả ba, và điều đó nói lên gì về việc xây dựng một hệ thống thông minh.

Dữ liệu thô $\rightarrow$ Làm sạch $\rightarrow$ Biểu diễn $\rightarrow$ Học $\rightarrow$ Đánh giá $\rightarrow$ Lưu trữ $\rightarrow$ Triển khai

10.6.1. Sáu bài học chính

Một hệ thống thông minh không phải là một mô hình học máy đã huấn luyện. Nó là sự kết hợp của dữ liệu, biểu diễn, quá trình học, phép đánh giá, phần mềm triển khai và tương tác với người dùng. Thiếu bất kỳ thành phần nào thì kết quả không dùng được — và trong sáu thành phần ấy, mô hình chỉ là một.

#	Bài học	Bằng chứng trong báo cáo
1	Biểu diễn dữ liệu quan trọng hơn thuật toán. Ở cả ba ứng dụng, thay đổi cách biểu diễn tạo ra khác biệt lớn hơn thay đổi mô hình.	Mục 9.8 — ROC-AUC 0.552 → 0.943; mục 8.5.2 — năm đặc trưng phái sinh
2	Công cụ kiểm tra chất lượng dữ liệu có thể báo sạch khi dữ liệu bẩn. Cả ba ứng dụng đều có một vấn đề mà hàm tiêu chuẩn không phát hiện, và mỗi lần là một dạng khác nhau.	Mục 7.4 (số 0 giả), mục 8.4 (trùng lặp), mục 9.4 (rò rỉ nhãn)
3	Điểm số cao hơn không đồng nghĩa với mô hình tốt hơn. Nó có thể chỉ có nghĩa là phép đo đã bị nhiễm.	Mục 8.4.1 — đo trực tiếp mức thổi phồng R^2 do không khử trùng lặp
4	Độ đo phải chọn theo cái giá của từng loại sai lầm, không chọn theo thói quen. Hai bài toán cùng là phân loại nhị phân nhưng ưu tiên ngược nhau.	Mục 4.2 — FN đắt ở Ứng dụng 1, FP đắt ở Ứng dụng 3
5	Có những giới hạn không mô hình nào vượt qua được, vì thông tin cần thiết không nằm trong dữ liệu.	Mục 7.8.2 (ca bệnh sớm), mục 9.9.2 (bình luận mâu thuẫn nội tại)
6	Nhất quán giữa huấn luyện và triển khai là một ràng buộc kỹ thuật, không phải một lời khuyên. Sai thứ tự cột không sinh ra lỗi nào — chỉ sinh ra kết quả sai.	Mục 6.2 và 6.3 — <code>metadata.json</code> và phép kiểm chứng artifact

10.7. Thách thức kỹ thuật lớn nhất

Không phải việc huấn luyện mô hình, mà là **phát hiện ra ba dạng rò rỉ dữ liệu** — mỗi ứng dụng một dạng, và không dạng nào tự lộ ra. Rò rỉ qua tiền xử lý được chặn bằng một quy tắc kỹ thuật đơn giản; hai dạng còn lại chỉ có hiểu biết về ý nghĩa dữ liệu mới nhận ra được. Đáng chú ý là cả ba dạng đều làm điểm số *đẹp hơn*, nên không có động lực tự nhiên nào để đi tìm chúng.

10.8. Vấn đề biểu diễn dữ liệu quan trọng nhất

Ở Ứng dụng 3, cột dễ dùng nhất lại là cột phải vứt đi (`Rating` cho điểm số đẹp nhưng vô giá trị), còn cột trông khó dùng nhất lại mang gần như toàn bộ tín hiệu (văn bản tự do). Ở Ứng dụng 2, biến giải thích mạnh nhất — vị trí — nằm chôn trong một cột địa chỉ mà mô hình không đọc được, và một hàm rút chuỗi năm dòng đã giải phóng nó. Cả hai trường

hợp đều nói cùng một điều: **quyết định đưa gì vào biểu diễn quan trọng hơn quyết định dùng thuật toán nào.**

10.9. Bài học về triển khai

Khoảng cách giữa “mô hình chạy trong notebook” và “dịch vụ chạy được” lớn hơn dự kiến, và phần lớn khoảng cách ấy nằm ở việc giữ cho biểu diễn lúc dự đoán trùng khớp biểu diễn lúc huấn luyện. Ứng dụng 2 là ví dụ rõ nhất: API phải tính lại năm đặc trưng phải sinh đúng y hệt công thức lúc huấn luyện, và nếu lệch thì không có gì báo lỗi — mô hình vẫn trả về một con số hợp lý về mặt hình thức.

10.10. Hướng cải tiến

- **Ứng dụng 1** — bổ sung HbA1c và dữ liệu theo thời gian. Giới hạn hiện tại nằm ở dữ liệu, không ở mô hình, nên đây là hướng duy nhất thực sự cải thiện được các ca bệnh giai đoạn sớm.
- **Ứng dụng 2** — giữ chi tiết địa chỉ tới cấp phường và bổ sung giá giao dịch thực thay vì giá chào bán.
- **Ứng dụng 3** — thay TF-IDF bằng mô hình ngôn ngữ đọc được ngữ cảnh, để xử lý cấu trúc nhượng bộ kiểu “đẹp nhưng không vừa” — nhóm sai sót lớn nhất hiện nay.
- **Chung cho cả ba** — bổ sung giám sát trôi dữ liệu sau khi triển khai. Một mô hình đúng lúc huấn luyện sẽ dần sai khi phân bố dữ liệu thực tế thay đổi, và hiện chưa có cơ chế nào phát hiện điều đó.

Kết lại. Một hệ thống thông minh có thể triển khai là sự kết hợp của **dữ liệu, biểu diễn, quá trình học, đánh giá, phần mềm, triển khai và tương tác người dùng**. Assignment 02 cho thấy phần khó nhất không nằm ở chỗ người ta thường nghĩ.

Chương XI. Mở rộng — Tích hợp đồ thị tri thức Neo4j

Phần mở rộng này bổ sung một tầng **đồ thị tri thức** vào Ứng dụng 1. Mục tiêu: sau khi mô hình đã dự đoán, hệ thống trả về thêm nội dung tư vấn phù hợp với tầng nguy cơ của bệnh nhân, thay vì chỉ trả về một con số xác suất.

Nguyên tắc thiết kế bắt buộc. Đồ thị tri thức **chỉ bổ sung nội dung, không bao giờ tham gia vào việc dự đoán**. Nhân và xác suất do mô hình học máy quyết định hoàn toàn. Nếu Neo4j hỏng, mất kết nối hoặc chưa cấu hình, API vẫn phải trả về dự đoán bình thường kèm một thông báo ở trường `knowledge_error`. Đây là một ràng buộc kiến trúc, không phải một tùy chọn.

11.1. Luồng hoạt động

Đầu vào người dùng → Tiền xử lý → Mô hình → Xác suất → Phân tầng nguy cơ → Truy vấn Neo4j → Kết quả + Tư vấn

Điểm nối duy nhất giữa mô hình và đồ thị là **xác suất dương tính**. Xác suất ấy được ánh xạ sang một trong ba tầng nguy cơ, và tầng nguy cơ là khoá tra cứu vào đồ thị.



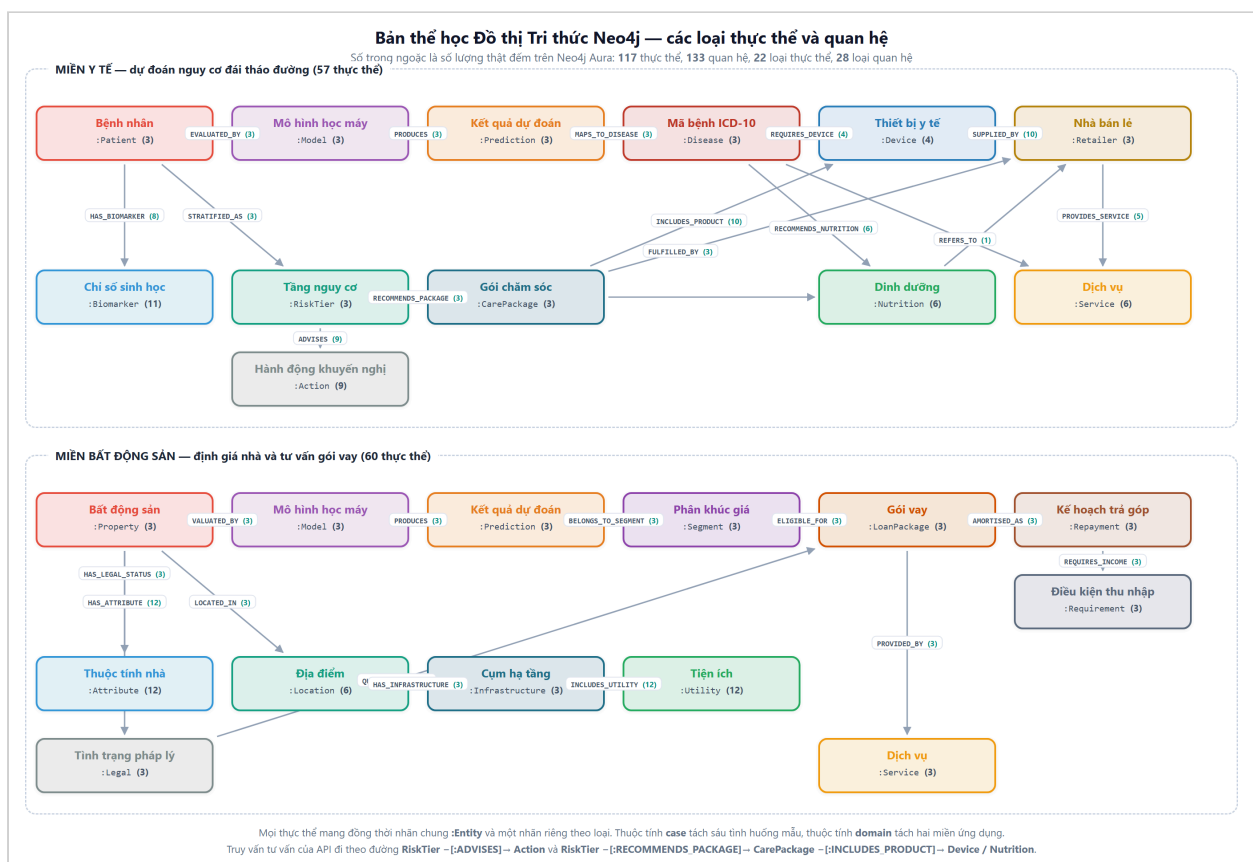
Hình 49. Vị trí của Neo4j trong hệ thống. Nhánh dự đoán (trên) và nhánh tri thức (dưới) tách rời nhau: đồ thị nhận vào tầng nguy cơ đã được mô hình quyết định, và không có mũi tên nào đi ngược lại vào mô hình.

Tầng nguy cơ	Ngưỡng xác suất	Nút RiskTier trong đồ thị
Nhóm Nguy cơ Cao	$\geq 0,25$	med_high
Nhóm Tiền Đái tháo đường	0,15 – 0,25	med_moderate
Nhóm Khỏe mạnh	$< 0,15$	med_low

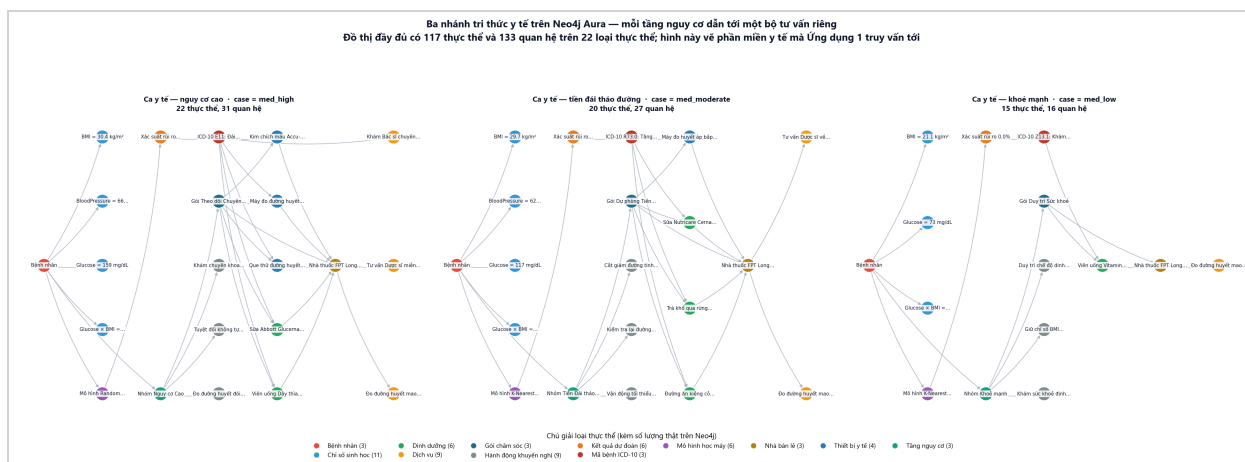
Ngưỡng đặt thấp hơn 0,5 có chủ đích: với bài toán sàng lọc, một bệnh nhân có xác suất 0,3 chưa bị mô hình gán nhãn dương tính nhưng vẫn đáng được nhận khuyến nghị theo dõi. Đây chính là ưu tiên Recall đã phân tích ở mục 7.8, được áp dụng ở tầng tư vấn thay vì tầng phân loại.

11.1.1. Lược đồ đồ thị

Đồ thị được xây dựng lại từ phần mở rộng của Assignment 01 và nạp lên một phiên bản Neo4j Aura đang chạy. Lược đồ dưới đây liệt kê các loại thực thể và loại quan hệ đọc trực tiếp từ cơ sở dữ liệu bằng `CALL db.labels()` và `CALL db.relationshipTypes()`.



Hình 50. Lược đồ bản thể học của đồ thị tri thức: các loại thực thể, các loại quan hệ nối chúng, và số lượng thực thể của từng loại.



Hình 51. Ba nhánh tri thức y tế trên Neo4j Aura, mỗi nhánh ứng với một nút **RiskTier** và chính là một bộ tư vấn mà API có thể trả về. Màu nút mã hoá loại thực thể. Hình chỉ vẽ phần miền y tế vì đó là phần duy nhất Ứng dụng 1 truy vấn tới; miền bất động sản của đồ thị xuất hiện đầy đủ ở hình lược đồ phía trên.

Nhãn nút	Ý nghĩa	Số lượng
RiskTier	Ba tầng nguy cơ	3
Action	Hướng dẫn lâm sàng cần thực hiện	9
CarePackage	Gói chăm sóc gắn với từng tầng	3
Device	Thiết bị theo dõi (máy đo, que thử...)	4
Nutrition	Sản phẩm dinh dưỡng hỗ trợ	6
Service	Dịch vụ nhà thuốc đi kèm	9

Quan hệ chính: **(RiskTier)-[:ADVISES]->(Action)** và **(RiskTier)-[:RECOMMENDS_PACKAGE]->(CarePackage)-[:INCLUDES_PRODUCT]->(Device | Nutrition | Service)**.

11.1.2. Truy vấn Cypher

CYPHER

```
MATCH (tier:RiskTier {case: $case})
OPTIONAL MATCH (tier)-[:ADVISES]->(action:Action)
OPTIONAL MATCH (tier)-[:RECOMMENDS_PACKAGE]->(:CarePackage)
      -[:INCLUDES_PRODUCT]->(item)
RETURN tier.name AS tier_name,
       collect(DISTINCT {kind: 'Action', name: action.name,
                        detail: action.`Nhóm`}) AS actions,
       collect(DISTINCT {kind: item.kind, name: item.name,
                        detail: item.`Công dụng`,
                        price: item.`Khoảng giá`,
                        url: item.`Tra cứu giá`}) AS products
```

Truy vấn Cypher lấy tư vấn theo tầng nguy cơ.

Hai mệnh đề `OPTIONAL MATCH` là chi tiết quan trọng: tầng “Nhóm Khỏe mạnh” không có gói thiết bị theo dõi, và nếu dùng `MATCH` thường thì toàn bộ kết quả — kể cả các hướng dẫn lối sống — sẽ biến mất, khiến người khỏe mạnh không nhận được lời khuyên nào.

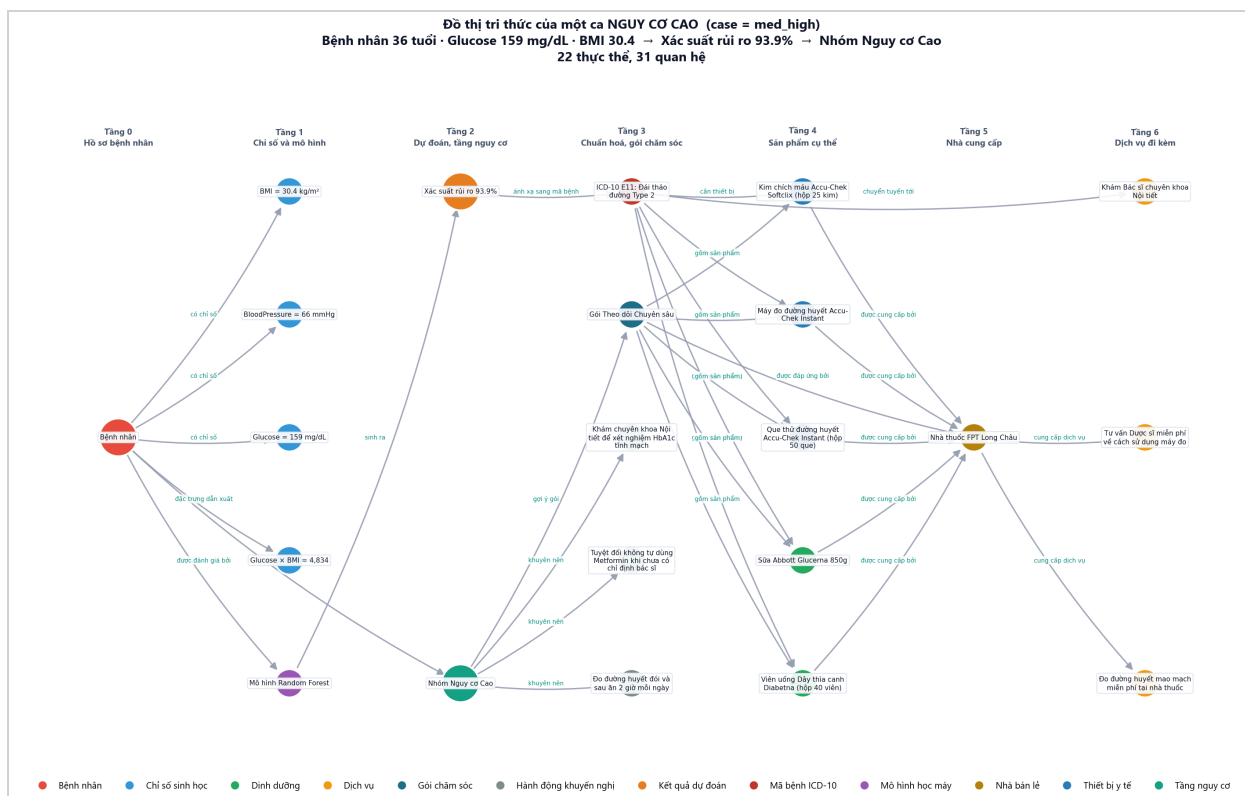
PYTHON

```
def resolve_risk_case(probability_positive):
    """Chọn tầng nguy cơ theo xác suất dương tính mà mô hình trả về."""
    for threshold, case in RISK_TIERS:
        if probability_positive >= threshold:
            return case
    return "med_low"
```

Ánh xạ xác suất của mô hình sang tầng nguy cơ.

11.2. Kết quả thực nghiệm

Hệ thống được kiểm thử với hai hồ sơ bệnh nhân trái ngược nhau, gửi tới cùng một điểm cuối `POST /diabetes/v1/predict`.



Hình 52. Đồ thị con được kích hoạt cho một ca nguy cơ cao: từ xác suất mô hình trả về, hệ thống đi tới nút tầng nguy cơ tương ứng rồi thu thập toàn bộ hướng dẫn, thiết bị, dinh dưỡng và dịch vụ nối vào nút đó. Đây chính là nội dung trường `knowledge` trong phản hồi JSON.

BASH

```
$ curl -X POST localhost:5001/diabetes/v1/predict \
-H "Content-Type: application/json" \
-d '{"Glucose":180,"BMI":40,"Age":55,
    "Pregnancies":7,"DiabetesPedigreeFunction":1.1}'
```

KẾT QUẢ CHẠY

```
class: 1 | p+: 88.94 | tier: Nhóm Nguy cơ Cao | err: None
Hướng dẫn cần làm -> ['Khám chuyên khoa Nội tiết để xét nghiệm HbA1c tĩnh mạch',
                      'Đo đường huyết đói và sau ăn 2 giờ mỗi ngày',
                      'Tuyệt đối không tự dùng Metformin khi chưa có chỉ định bác sĩ']
Thiết bị theo dõi -> ['Máy đo đường huyết Accu-Chek Instant',
                      'Que thử đường huyết Accu-Chek Instant (hộp 50 que)',
                      'Kim chích máu Accu-Chek Softclix (hộp 25 kim)']
Dinh dưỡng hỗ trợ -> ['Sữa Abbott Glucerna 850g',
                      'Viên uống Dây thìa canh Diabetna (hộp 40 viên)']
```

Bệnh nhân nguy cơ cao — xác suất 88,94%, đồ thị trả về ba nhóm tư vấn.

KẾT QUẢ CHẠY

```
class: 0 | p+: 2.06 | tier: Nhóm Khỏe mạnh
Hướng dẫn cần làm -> ['Duy trì chế độ dinh dưỡng cân bằng',
                        'Giữ chỉ số BMI trong khoảng 18.5 đến 22.9',
                        'Khám sức khoẻ định kỳ hàng năm']
Dinh dưỡng hỗ trợ -> ['Viên uống Vitamin tổng hợp & Khoáng chất']
```

Bệnh nhân nguy cơ thấp — xác suất 2,06%, nội dung tư vấn hoàn toàn khác.

Hai kết quả cho thấy tầng tri thức **phản ứng đúng theo đầu ra của mô hình**: cùng một điểm cuối, cùng một truy vấn, nhưng nội dung trả về khác hẳn nhau vì xác suất khác nhau dẫn tới tầng nguy cơ khác nhau.

11.2.1. Kiểm chứng nguyên tắc “không ảnh hưởng tới dự đoán”

Khi ngắt biến môi trường `NEO4J_PASSWORD`, API vẫn khởi động và vẫn dự đoán bình thường; trường `knowledge` trả về mảng rỗng và `knowledge_error` chứa thông báo “Chưa cấu hình kết nối Neo4j”. Nhấn và xác suất **không thay đổi một chữ số nào**.

Đây là bằng chứng cho rằng buộc kiến trúc đã nêu ở đầu chương: tầng tri thức là một lớp bổ sung có thể tháo rời, không phải một thành phần của mô hình.

11.2.2. Giá trị mang lại và hạn chế

Giá trị. Một con số xác suất, tự nó, không cho người dùng biết phải làm gì tiếp theo. Tầng tri thức biến đầu ra của mô hình thành một khuyến nghị hành động cụ thể, và vì tri thức nằm trong đồ thị chứ không nằm trong mã nguồn, nội dung tư vấn có thể được cập nhật mà không cần huấn luyện lại hay triển khai lại mô hình. **Hạn chế.** Nội dung trong đồ thị được nhập thủ công, nên phạm vi còn hẹp và cần chuyên gia y tế thẩm định trước khi dùng thật. Ngoài ra tư vấn hiện chỉ phụ thuộc tầng nguy cơ, chưa cá nhân hoá theo từng chỉ số cụ thể của bệnh nhân — một bệnh nhân nguy cơ cao vì BMI và một bệnh nhân nguy cơ cao vì glucose hiện nhận cùng một bộ khuyến nghị.